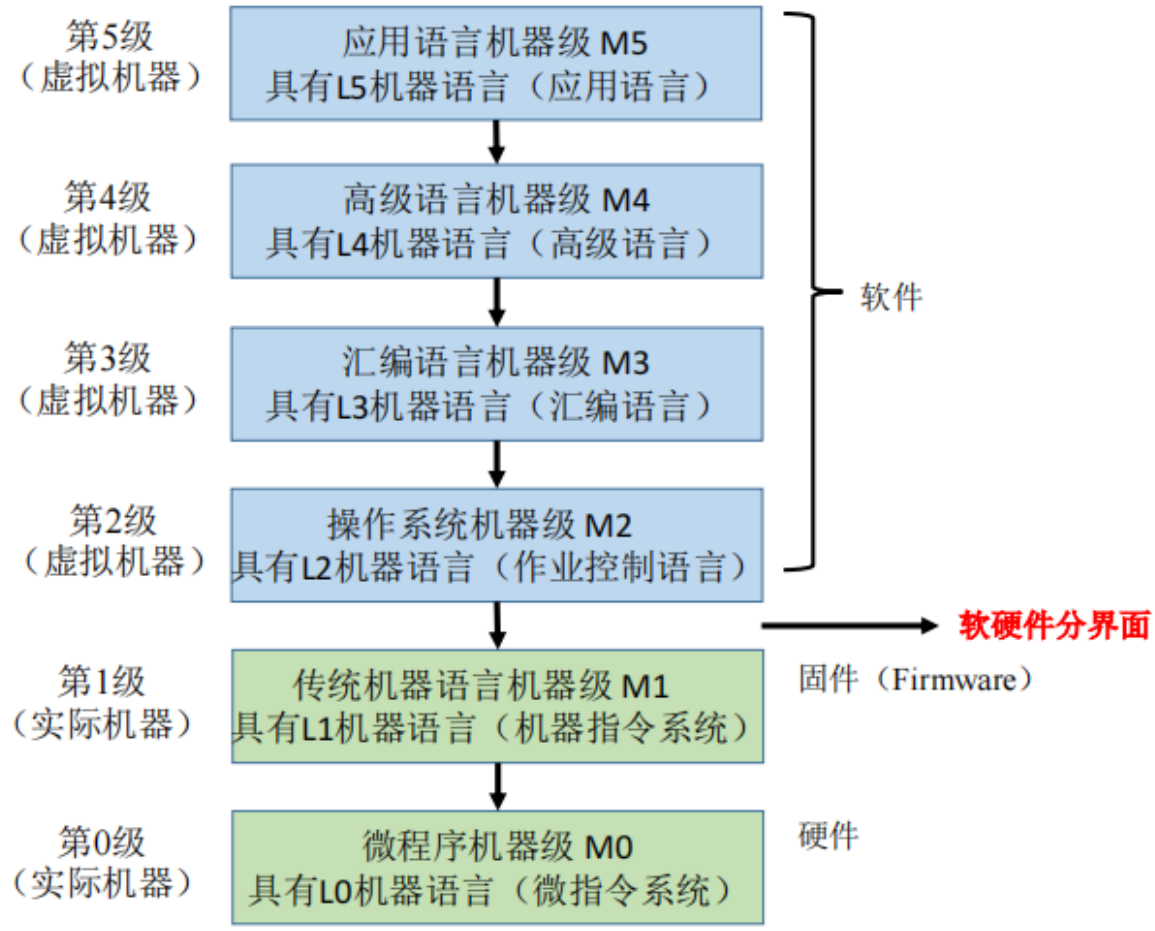
计算机体系结构复习提纲

第一章 基本概念

1. 计算机系统的层次结构



语言实现的两种基本技术：

- 翻译：先把 `N+1` 级程序全部转换成 `N` 级程序后，再去执行新产生的 `N` 级程序，在执行过程中 `N+1` 级程序不再被访问。
- 解释：每当一条 `N+1` 级指令被译码后，就直接去执行一串等效的 `N` 级指令，然后再去取下一条 `N+1` 级的指令，依此重复进行。
- 解释执行比编译后再执行所花的时间多，但占用的存储空间较少。

2. 计算机系统结构的定义

1964 年，`Amdahl` 将计算机系统结构定义为由程序设计者所看到的计算机系统的属性，即概念性结构和功能特性。

- 程序员：系统程序员（包括：汇编语言、机器语言、编译程序、操作系统）；
- 看到的：编写出能在机器上正确运行的程序所必须了解到的；
- 概念性结构：计算机底层系统的总线图表示；

- 功能特性：指令系统及其执行模式，例如数据表示、寻址技术、寄存器组织、指令系统、中断系统、存储系统、IO 系统；
- 此定义侧重于硬件子系统；

透明性概念：在计算机技术中，一种本来是存在的事物或属性，但从某种角度上看起来似乎不存在。

正经定义：

- 计算机系统结构研究的是 **软、硬件之间的功能分配** 以及 **对传统机器级界面的确定** ，为机器语言、汇编语言程序设计者或编译程序生成系统提供使生成的程序能在机器上正确运行而应看到和遵循的计算机属性。

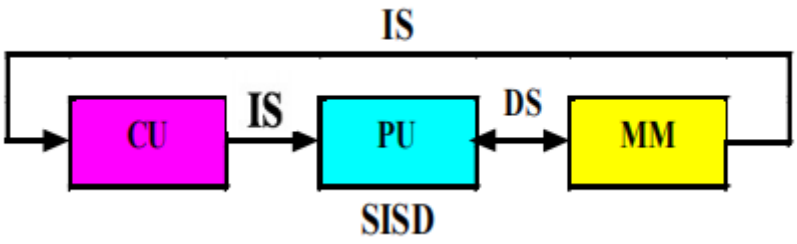
3. 计算机系统的分类

`Flynn` 分类法：`1966` 年 `Flynn` 提出了如下定义：

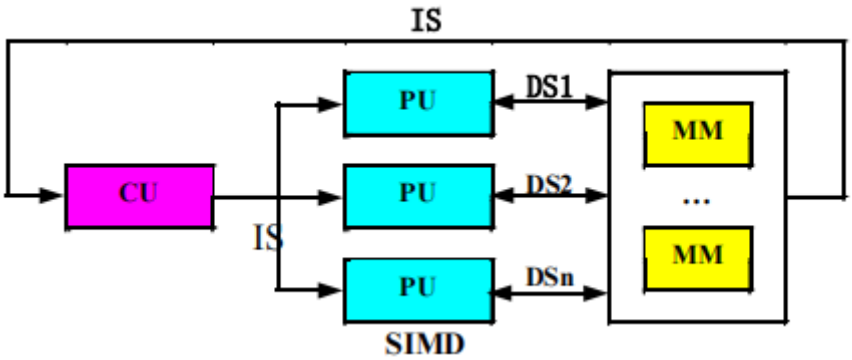
- 指令流：机器执行的指令序列；
- 数据流：由指令流调用的数据序列；
- 多倍性：在系统性能瓶颈部件上同时处于同一执行阶段的指令或数据的最大可能个数；

他按照指令流和数据流的不同组织方式，把计算机系统结构划分为以下四类：

- 单指令流单数据流 SISD：典型顺序处理计算机；

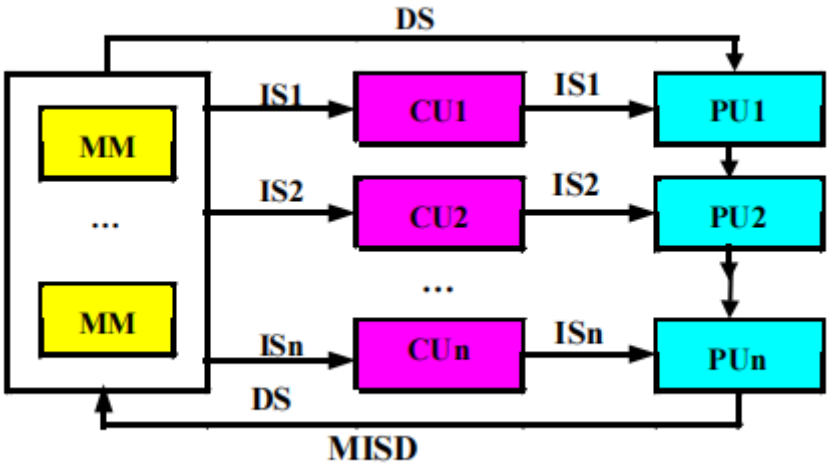


- 单指令流多数据流 SIMD：并行处理机、阵列处理机、向量处理机、相联处理机、超标量处理机、超流水线处理机；

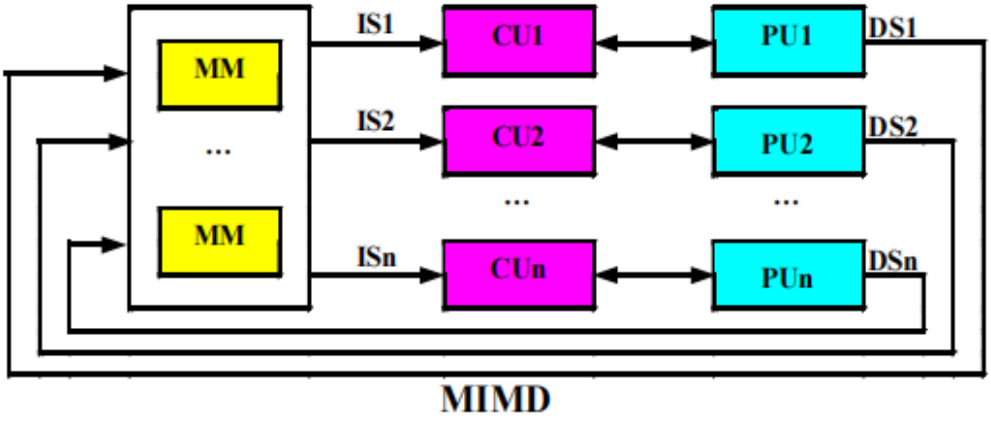


  多个PU按一定方式互连，在同一个CU控制下，对各自的数据完成同一条指令规定的操作；从CU看指令顺序执行，从PU看数据并行执行。

- 多指令流单数据流 MISD：几条指令对同一个数据进行不同的处理；



- 多指令流多数据流 MIMD：多处理机系统；



4. Amdahl 定律及其应用

系统中某一部件由于采用更快的执行方式后，整个系统性能的提高与这种执行方式的使用频率或占总执行时间的比例有关。

$$\text{可改进部分的比例：}F_e = \frac{\text{可改进部分的执行时间}}{\text{改进前整个任务的执行时间}}$$

$$\text{改进部分的加速比：}S_e = \frac{\text{改进前改进部分的执行时间}}{\text{改进后改进部分的执行时间}}$$

假设 `T0` 为改进前整个任务的执行时间，则改进后整个任务的执行时间为：

$$T_n = T_0 \cdot (1 - F_e + \frac{F_e}{S_e})$$

改进后整个系统的加速比为：

$$S_n = \frac{T_0}{T_n} = \frac{1}{1 - F_e + \frac{F_e}{S_e}}$$

例：某部件的处理时间仅为整个运行时间的 `40%`，如果将该部件的处理速度加快到 `10` 倍，则采用加快措施后能使整个系统的性能提高多少?

解：由题意可知：Fe = 0.4, Se = 10， 根据Amdahl定律，加速比为：

$$S_n = \frac{1}{1 - 0.4 + \frac{0.4}{10}} = \frac{1}{0.64} = 1.56$$

例：采用哪种实现技术来求浮点数平方根FPSQR的操作对系统的性能影响较大。假设FPSQR操作占整个测试程序执行时间的20%。一种实现方法是采用FPSQR硬件，使FPSQR操作的速度加快到10倍。另一种实现方法是使所有浮点数据指令的速度加快，使FP指令的速度加快到2倍，还假设FP指令占整个执行时间的50%。请比较这两种设计方案。

$$S_{FPSQR} = \frac{1}{(1 - 0.2) + 0.2/10} = \frac{1}{0.82} = 1.22$$

$$S_{FP} = \frac{1}{(1 - 0.5) + 0.5/2} = \frac{1}{0.75} = 1.33$$

可见，使所有 `FP` 指令的速度提高这一方案更好。

5. CPU 性能公式

- IC：程序执行的总指令条数；
- CPI：平均每条指令的时钟周期数；
- fc：时钟主频，取决于具体硬件；

一个程序所花的 `CPU` 时间为：

$$T_{CPU} = IC \times CPI \times \frac{1}{f_c}$$

如果有 `n` 种指令，每种指令的时钟周期数为 `CPIi`，出现次数为 `Ii`，公式为：

$$T_{CPU} = \frac{\sum_{i=1}^{n}(CPI_i \times I_i)}{f_c}$$

平均指令时钟周期数为：

$$CPI = \frac{\sum_{i=1}^{n}(CPI_i \times I_i)}{IC}$$

例：如果FP操作的比例为25%，FP操作的平均CPI=4.0，其它指令的平均CPI为1.33；FPSQR操作的比例为2%， FPSQR的CPI为20。假设有两种设计方案，分别把FPSQR操作的CPI和所有FP操作的CPI减为2。试利用CPU性能公式比较这两种设计方案哪一个更好(只改变CPI而时钟频率和指令条数保持不变)。

原系统的CPI=25%×4+75%×1.33=2

方案1：使FPSQR操作的CPI为2

$$CPI=CPI_{原}-2\%\times(20-2)=2-2\%\times18=1.64$$

方案2：提高所有FP指令的处理速度

$$CPI=CPI_{原}-25\%\times(4-2)=2-25\%\times2=1.5$$

显然，提高所有FP指令处理速度的方案要比提高FPSQR处理速度的方案要好。方案2的加速比=2/1.5= 1.33

6. 程序访问的局部性规律

局部性分时间局部性和空间局部性：

- 时间局部性：程序中近期被访问的信息项很可能马上将被再次访问；
- 空间局部性：指那些在访问地址上相邻近的信息项很可能会被一起访问；
- 例如，程序执行时间的90%都是在执行程序中10%的代码；存储器体系的构成就是以访问的局部性原理为基础的。

7. 计算机系统的评价标准

- **MIPS**：Million Instructions Per Second

$$MIPS=\frac{\text{指令条数}}{\text{执行时间}\times10^6}=\frac{\text{指令条数}}{\text{指令条数}\times CPI\times\text{时钟长度}\times10^6}=\frac{\text{时钟频率}}{CPI\times10^6}$$

  - 优点：直观、方便，目前还经常使用；
  - 缺点：①依赖于指令集，只适合指令集相同的不同机器；②在同一台机器上，由于指令使用频度差别很大，MIPS会因程序不同而变化；③由于硬件的优化，MIPS可能与性能相反。
- **MFLPS**：Million Floating Point Operations Per Second

$$MFLOPS=\frac{\text{程序中的浮点操作次数}}{\text{执行时间}\times10^6}$$

  - 只能反映机器执行浮点操作的性能，并不能反映机器的整体性能；
  - 会随着整数和浮点数的比例、快速浮点操作与慢速浮点操作的比例不同而不同；
  - 一般认为 1 MFLOPS ≈ 13MIPS。
- **基准测试程序**

  计算机的性能通常用 **峰值性能** 和 **持续性能** 来评价；

- 峰值性能是指在理想情况下计算机系统可获得的最高理论性能值；
- 持续性能又称为实际性能，它的值往往是峰值性能的 5%~30%；

持续性能的表示（算数性能平均值）：

算数性能平均值A_m是n道程序运算速度或运算时间的算数平均值

以速度评价

$$A_m = \frac{1}{n}\sum_{i=1}^{n} R_i = \frac{1}{n}\sum_{i=1}^{n} \frac{1}{T_i}$$

以执行时间评价

$$A_m = \frac{1}{n}\sum_{i=1}^{n} T_i$$

加权算数平均（各个程序出现的比例不同）

$$A_m = \sum_{i=1}^{n} \alpha_i R_i = \frac{1}{n}\sum_{i=1}^{n} \frac{\alpha_i}{T_i}$$

第二章 指令系统

1. 数据表示与数据类型

数据表示：硬件能直接识别，可以被指令系统直接调用的那些数据类型，数据表示是数据类型中最常用，也是最容易用硬件实现的几种，例如定点数、浮点数、布尔数、字符、字符串、堆栈和向量；

数据结构：研究的是面向系统软件和应用领域的各种高级的数据类型，并有相应算法；

数据类型：文件、堆栈、向量、阵列、链表、整型、字符等软件要处理的各种数据形式；

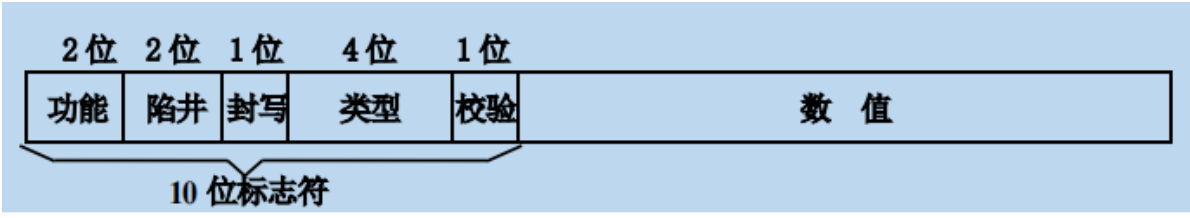
区别：数据表示和数据结构都是数据类型的子集，数据结构要通过软件映像，变换成机器所具有的数据表示来实现；不同的数据表示可为数据结构的实现提供不同的支持，数据结构和数据表示实际上是软硬件的交界面，需要在系统结构设计时予以确定。

系统结构的设计者首先要做的，就是确定哪些数据类型全部用硬件表示，即数据表示，哪些数据类型用软件实现，即数据结构。

2. 标志符数据表示、数据描述符

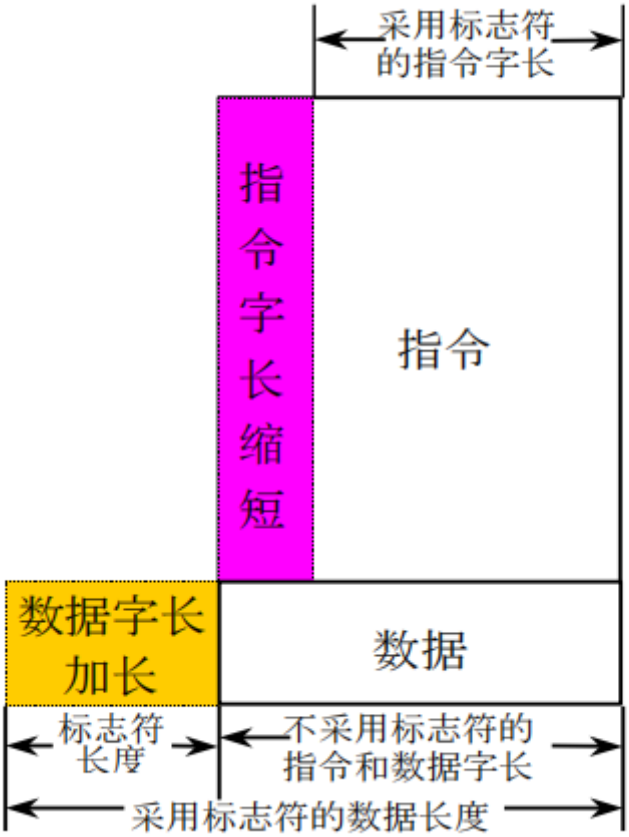
标志符数据表示：

为缩短高级语言与机器语言之间的语义差距，让机器中的每个数据都加上类型标志位；



- **功能位**：操作数、指令、地址、控制字；
- **封写位**：指定数据是只读的还是可读可写；
- **类型位**：二进制、十进制、定点数、浮点数、字符串等。

常规数据表示方法与带标志符数据表示方法的比较：



假设X处理机的数据不带标志符，其指令字长和数据字长均为32位；Y处理机的数据带标志符，数据字长增加至35位，其中3位是标志符，其指令字长由32位减少至30位。并假设一条指令平均访问两个操作数，每个操作数平均被访问R次。分别计算这两种不同类型的处理机中程序所占用的存储空间。

解：X和Y处理机程序占用的存储空间总和分别为

$$B_X = 32I + \frac{2\times 32I}{R}, B_Y = 30I + \frac{2\times 35I}{R}$$

程序占用存储空间的比值为

$$\frac{B_Y}{B_X} = \frac{15\mathrm{R} + 35}{16R + 32}$$

当$R > 3$时，有$\frac{B_Y}{B_X} < 1$;而实际应用中经常是$R > 10$。所以，**标志符的处理机所占用的存储空间通常要小**。

采用标志符数据表示方法的主要优点：

- 简化了指令系统；
- 由硬件实现一致性检查和数据类型转换；
- 简化程序设计，缩小了人与计算机之间的语义差距；

- 简化编译器，使高级语言与机器语言之间的语义差距大大缩短；
- 支持数据库系统，一个软件不加修改就可适用于多种数据类型；
- 方便软件调试，在每个数据中都有陷井位。

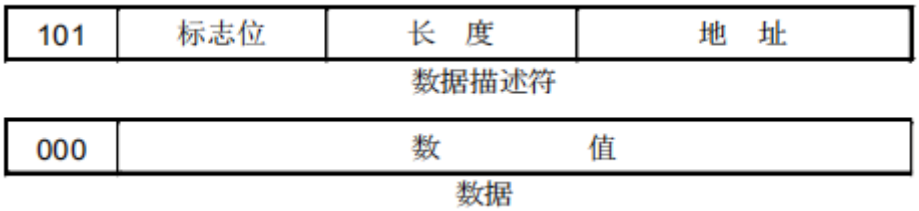
采用标志符数据表示方法的主要缺点：

- 数据和指令的长度可能不一致；
  - 可以通过精心设计指令系统来解决
- 指令的执行速度降低；
  - 但是，程序的设计时间、编译时间和调试时间可以缩短；
- 硬件复杂度增加；
  - 由硬件实现一致性检查和数据类型的转换。

数据描述符：为进一步减少标志符所占用的存储空间，对向量、数据、记录等数据，由于元素属性相同，采用数据描述符。

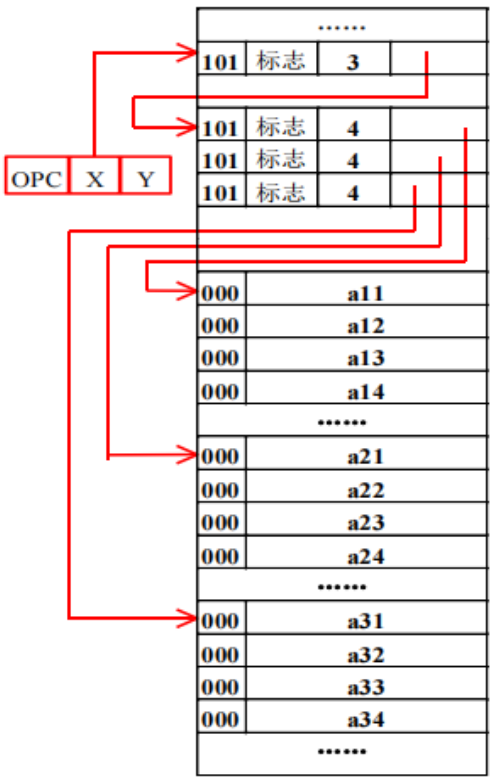
- 数据描述符与标志符的区别：标志符只作用于一个数据，而数据描述符要作用于一组数据。标志符一般与数值一起存放在同一个数据单元中，数据描述符一般单独存放，独立占据一个存储单元。

通常，最高三位为 `101` 时表示数据描述符，最高三位为 `000` 时表示数据：



例如，用数据描述符表示方法表示如下一个 `3×4` 的矩阵：

$$A=\begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \end{bmatrix}$$

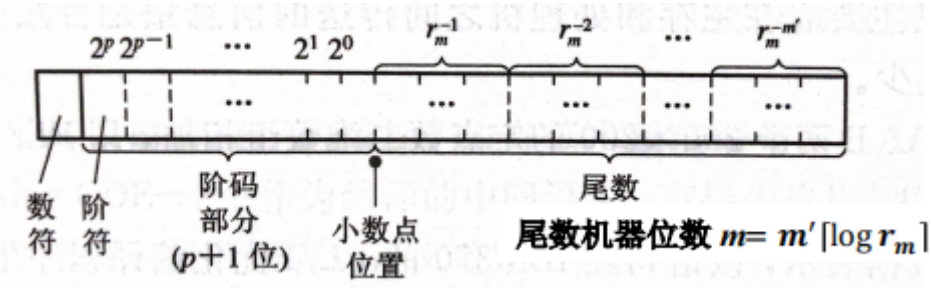


其中，标志位表示了描述的是单个数据还是数据块，是连续存放还是分段存放，是可读还是可读写等，如果描述的是数据块，还要给出长度和首地址。

3. 浮点数的表示方式

一个浮点数 N 可用如下方式表示：

$$N = M \times r_m^E$$



M：尾数值，一般采用纯小数和原码表示；

E：阶码，一般采用整数和移码表示；

r_m：尾数基，通常有二进制、四进制、八进制；

m：尾数机器尾数，实际用多少位表示；

m'：尾数位数，如 m = 8，r_m = 2，则 8 = m' × $\log_2 2$，m' = 8；如 m = 8，r_m = 4，则 8 = m' × $\log_2 4$，m' = 4，表示每 2 位机器数表示一个基为 4 的尾数；

p：阶码长度，阶码部分的二进制位数；

浮点数的表数范围（**当 r_m 为 2 的整数次幂时，$r_m^{m'} = 2^m$**）：

- 最小尾数（小数点后第一个 r_m 末位为 1）：r_m^{-1}；
- 最大尾数（尾数全 1）：$1 - r_m^{-m'}$；

  推导过程：

$$(r_m - 1) \cdot (r_m^{-1} + r_m^{-2} + \ldots + r_m^{-m'}) = (r_m - 1) \cdot \frac{r_m^{-1} \cdot (1 - r_m^{-m'})}{1 - r_m^{-1}} = 1 - r_m^{-m'}$$

- 最小阶：-2^p；
- 最大阶（阶码全 1）：$2^p - 1$；
- **可表示最小值**：$r_m^{-1} \times r_m^{-2^p}$；
- **可表示最大值**：$(1 - r_m^{-m'}) \times r_m^{2^p - 1}$；（**= $r_m^{2p-1} \times (1 - 2^{-m})$**）
- 可表示尾数个数：$(r_m - 1) \times r_m^{m'-1}$；

  推导过程：

$$(r_m - 1) \times r_m \times \ldots \times r_m = (r_m - 1) \times r_m^{m'-1}$$

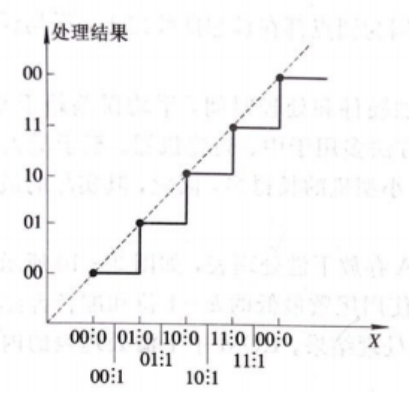
- 可表示阶的个数：2^p；
- **可表示数的个数**：$2^p \times (r_m - 1) \times r_m^{m'-1}$；（**= $2^{p+m} \times (r_m - 1) / r_m$**）

结论：尾数基值增大，会扩大浮点数表示范围，增加可表示数的个数，因此可以减少计算中的移位次数，降低右移造成的精度损失，提高运算速度，但也会降低数据的表示精度，数值的分布变稀疏。

尾数下溢的处理方法有：

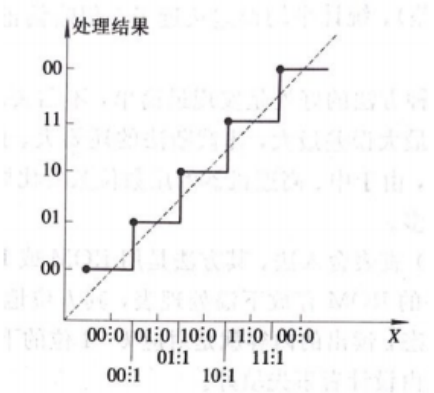
- **截断法**：将尾数超出机器字长的部分截去。优点是实现简单，不增加硬件，不需要处理时间；缺点是平均误差较大且无法调节。

		尾数有效字长 p 位	有效字长之外 g 位	误差情况
正数区	舍入前	0. xxx. xx	00. 000	$\delta=0$
		0. xxx. xx	00. 001	$\delta=-2^{-p-g}$
		0. xxx. xx	00. 010	$\delta=-2^{-p-g+1}$
		. . . . .	. . . . .	. . . . .
		0. xxx. xx	11. 111	$\delta=-2^{-p}(1-2^{-g})$
	舍入后	0. xxx. xx		$\sigma=-2^{-p-1}(2^{g}-1)$
负数区	舍入前	−0. xxx. xx	00. 000	$\delta=0$
		−0. xxx. xx	00. 001	$\delta=+2^{-p-g}$
		−0. xxx. xx	00. 010	$\delta=+2^{-p-g+1}$
		. . . . .	. . . . .	. . . . .
		−0. xxx. xx	11. 111	$\delta=+2^{-p}(1-2^{-g})$
	舍入后	−0. xxx. xx		$\sigma=+2^{-p-1}(2^{g}-1)$



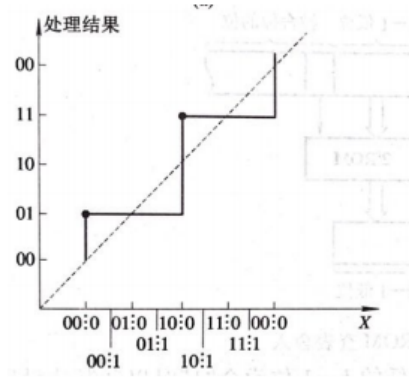
- **舍入法**：在机器运算的规定字长之外增设一位附加位，存放溢出部分的最高位，每当进行尾数下溢处理时，将附加位加1。优点是实现简单，增加硬件很少，最大误差小，平均误差接近于零；缺点是处理速度慢，需要花费在附加位上加1以及因此产生的进位时间。

正数区	尾数有效字长 p 位	有效字长之外 g 位	误差δ
舍入前	0. xxx. xx	00. 000	0
	0. xxx. xx	00. 001	-2^{-p-g}
	0. xxx. xx	00. 010	-2^{-p-g+1}
	. . . . .	. . . . .	. . . . .
	0. xxx. xx	01. 111	$-2^{-p-1}(1-2^{-g})$
	0. xxx. xx	10. 000	$+2^{-p-1}$—积累误差
	0. xxx. xx	10. 001	$+2^{-p-1}(1-2^{-g})$
	. . . . .		. . . . .
	0. xxx. xx	11. 110	$+2^{-p-g+1}$
	0. xxx. xx	11. 111	$+2^{-p-g}$
舍入后	0.xxx. . . . .xx 0.xxx. . . . .xx+2^{-p}	最高位为 **0** 最高位为 **1**	积累误差$\sigma=+2^{-p-1}$



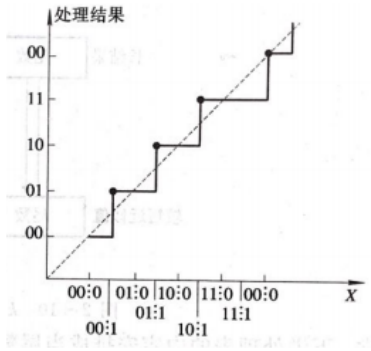
- **恒置"1"法**：把有效字长的最低一位置成 r_m / 2，优点是实现简单，不需要增加硬件和处理时间，平均误差接近0；缺点是最大误差较大。

正数区	尾数有效字长 p 位	有效字长外 g 位	误差情况
舍入前	0. xxx. xx0	00. 000	$+2^{-p}$ —— 误差积累
	0. xxx. xx0	00. 001	$+2^{-p}(1-2^{-g})$
	0. xxx. xx0	00. 010	$+2^{-p}(1-2^{-g+1})$
	. . . . .	. . . . .	. . . . .
	0. xxx. xx0	11. 111	$+2^{-p-g}$
	0. xxx. xx1	00. 000	0
	0. xxx. xx1	00. 001	-2^{-p-g}
			. . . . .
	0. xxx. xx1	11. 110	$-2^{-p}(1-2^{-g+1})$
	0. xxx. xx1	11. 111	$-2^{-p}(1-2^{-g})$
舍入后	0. xxx. xx1		积累误差$\sigma=2^{-p}$



- **查表舍入法**：用ROM或者PLA存放下溢处理表。优点是速度快，平均误差可以调节到0；缺点是硬件量大。

ROM地址	舍入前(p+g位)	舍入后(p位)	误差情况	
000	xx…x00|0xx…x	xx…x00	$-2^{-p-1}(1-2^{-g+1})\sim 0$	$+2^{-p-1}$
001	xx…x00|1xx…x	xx…x01	$+2^{-p-g}\sim 2^{-p-1}$	
010	xx…x01|0xx…x	xx…x01	$-2^{-p-1}(1-2^{-g+1})\sim 0$	$+2^{-p-1}$
011	xx…x01|1xx…x	xx…x10	$+2^{-p-g}\sim 2^{-p-1}$	
100	xx…x10|0xx…x	xx…x10	$-2^{-p-1}(1-2^{-g+1})\sim 0$	$+2^{-p-1}$
101	xx…x10|1xx…x	xx…x11	$+2^{-p-g}\sim 2^{-p-1}$	
110	xx…x11|0xx…x	xx…x11	$-2^{-p-1}(1-2^{-g+1})\sim 0$	$-2^{-p-1}(2^{g}-1)$
111	xx…x11|1xx…x	xx…x11	$-2^{-p}(1-2^{-g})\sim -2^{-p-1}$	

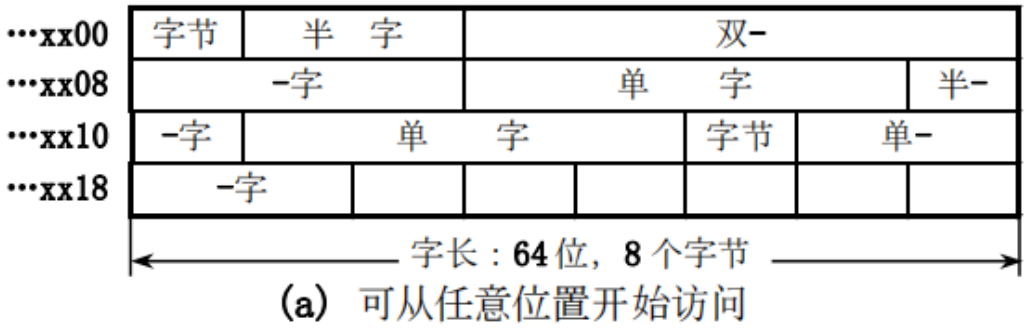


4. 编址、寻址和定位方式

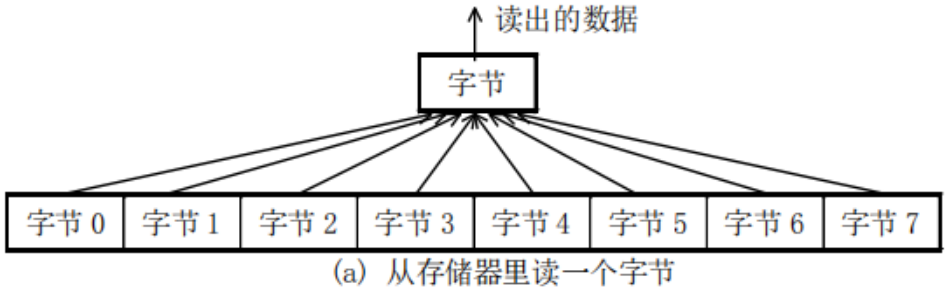
编址方式

常用的编址单位：

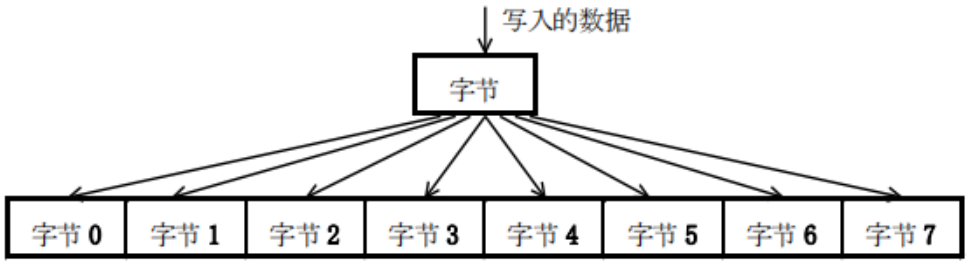
- 位编址：按位编址；
- 块编址：按块编址；
- 字编址：每个编址单位与设备的访问单位一致，实现简单，地址信息和存储容量没有任何浪费，但是没有对非数值计算操作提供支持，因为非数值计算的基本寻址单位是字节，目前以很少用；
- **字节编址**：编制单位与信息的基本单位一致，但是会产生地址信息浪费问题，例如 32 位机器浪费低 2 位地址，64 位机器浪费低 3 位地址，并且还有读写逻辑复杂、大端小端问题。存储器是按字访问数据的，因此产生了数据如何在存储器中存放的问题。
    - 可从任意位置开始访问：



(a) 可从任意位置开始访问



(a) 从存储器里读一个字节

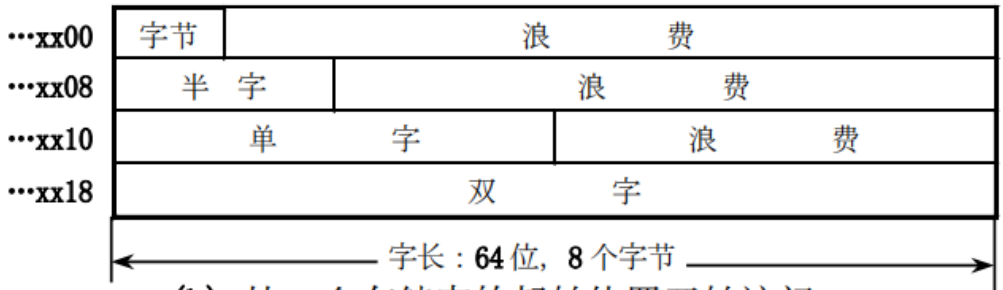


(b) 写一个字节到存储器

当从主存中读一个字节时，首先用除去低 3 位之外的地址读主存，此时读出了 8 个字节，然后再用低 3 位地址控制一套多路开关去读字节，写操作同理。

读入和写出共两次操作在 DRAM 中只需要一个存储周期就能完成，因为 DRAM 是一种破坏性读出存储器，当从一个存储单元读出数据后，这个单元就被清空，为了下次还能从该单元读出原来的数据，必须把刚刚读出的那个数据重新写入这个单元。因此，向一个存储单元中写入一个数据，涉及到读出和写入两次操作，但是实际上只需要一个存储周期。

- 从一个存储字的起始位置开始访问：



(b) 从一个存储字的起始位置开始访问

此方法的优点是，无论访问什么数据都可以在一个存储周期内完成，读写数据的控制逻辑也比较简单；

缺点是，浪费了宝贵的存储器资源，存储器空间的利用率约只有 50%。

- 从地址的整倍数位置开始访问：

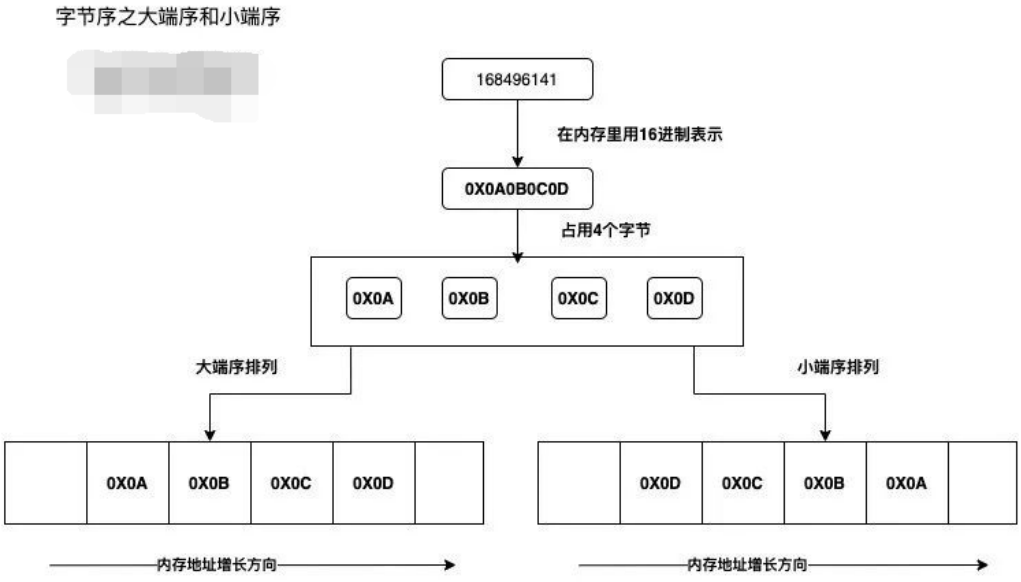


(c) 从地址的整倍数位置开始访问

此方法的优点是，双字地址最末三个二进制位必须为000，单字地址最末两位必须为00， 半字地址最末一位必须为0，无论访问哪种类型的数据，都能在一个周期内完成，目前是最主要的方法；

缺点是，控制逻辑仍然比较复杂，空间仍有一定的浪费。

另外，在字节编址的机器中还存在大端与小端问题，小端模式将数据低地址编址到内存低地址，大端模式将数据高地址编址到内存低地址。



零地址空间个数：

- 三个零地址空间：通用寄存器、主存储器、IO 设备独立编址；
- 两个零地址空间：通用寄存器独立编址，主存储器与 IO 设备统一编址；
- 一个零地址空间：地址最低端是通用寄存器，最高端是 IO 设备，中间为主存储器；
- 隐含编址方式：堆栈计算机、Cache等。

输入输出设备的编址：

- 一台设备一个地址：仅对输入输出设备本身进行编址，需要通过指令中的操作码来识别该输入输出设备接口上的有关寄存器；
- 一台设备两个地址：数据寄存器、状态或控制寄存器，对大多数 IO 设备恰到好处，因为绝大多数设备的接口上只有两个要编址的寄存器；
- 多个编址寄存器共用同一个地址的方法：
  - 依靠地址内部来区分，适用于被编址的寄存器的长度比较短；
  - “下跟法”隐含编址方式，根据访问顺序轮询每一个寄存器，缺点是必须按顺序读写寄存器；
- 一台设备多个地址：适合主存与 IO 设备共同编址的机器，因为寻址空间比较大，但是会增加编程的困难。

并行存储器的编址技术：

- 地址码高位交叉编址：目的是为了扩大存储器容量，地址码的低位是各个存储体的体内地址，高位是各存储体的体号。这种方法要求每个存储体都有各自独立的控制部件，包括地址寄存器、译码器、驱动电路、读写控制电路等。优点是模块化好，方便用户扩展，缺点是速度相对较慢，控制部件比较多。
- 低位交叉编址：主要是提高存储器速度，其低位部分是各个存储体的体号，高位是体内地址。在一个存储周期内，n 个存储体必须分时启动，采用流水线的方式工作，在连续工作的情况下，整个主存的速度可以提高 n 倍。其主要缺点是存在访问冲突问题，使得加速比达不到 n。

模4低位交叉编址

模体	地址编址序列	对应二进制地址码最末二位状态
M_0	0, 4, 8, 12, …, 4i+0, …	00
M_1	1, 5, 9, 13, …, 4i+1, …	01
M_2	2, 6, 10, 14, …, 4i+2, …	10
M_3	3, 7, 11, 15, …, 4i+3, …	11

寻址方式

面向主存储器的寻址方式：

- **直接寻址**：在指令中直接给出参加运算的操作数及运算结果所存放的主存地址。
- **间接寻址**：指令中给出的是操作数地址的地址，必须经过两次（或两次以上）访存操作才能得到操作数。
- **变址寻址（相对寻址、基址寻址）**：指令执行时，用一个硬件加法器，把变址寄存器中给出的基地址加上指令中给出的偏移量，才能得到有效地址。

面向堆栈的寻址方式：堆栈寻址方式的地址是隐含的，在指令中不需要给出操作数的地址 。

定位方式

程序所分配到的主存物理空间和程序本身的逻辑地址空间是不相同的，需要把指令和数据中的逻辑地址（相对地址）转换成主存储器的物理地址（绝对地址）。定位方式主要研究程序的主存物理地址在什么时间确定，采用什么方式来实现。

程序需要定位的主要原因：

- 程序的独立性；
- 程序的模块化设计；
- 数据结构在程序运行过程中，其大小往往是变化的；
- 有些程序本身很大，大于分配给它的主存物理空间；

主要有以下四种定位方式：

- **直接定位方式**：在程序装入主存储器之前，程序中的指令和数据的主存物理就已经确定。
- **静态定位**：在程序装入主存储器的过程中随即进行地址变换，确定指令和数据的主存物理地址，程序每次装入时都可以是不同的地址，但一旦装入就不再改变。
- **动态定位**：必须有硬件支持，确定程序在主存中的起始地址后，将其存入基址寄存器中，程序执行时，将虚拟地址与基址寄存器相加得到在主存中的物理地址。
    - 静态定位是直接修改指令来实现的，例如 `JMP 100` 直接修改位 `JMP 100 + m`，动态定位是用地址加法器和基址寄存器来实现的。

5. 操作码优化设计

操作码的表示方法通常有以下三种：

1. 固定长操作码

所有指令固定为 8 位表示，非常规整，硬件译码也很简单，但是浪费了很多信息量，操作码的总长度增加了。

2. Huffman 编码法

利用Huffman树进行操作码编码（又称最小概率合并法）

- 把所有指令按照操作码在程序中出现的概率大小，自左向右顺序排列。
- 选取两个概率最小的结点合并成一个概率值是二者之和的新结点，并把这个新结点与其它还没有合并的结点一起形成一个新的结点集合。
- 在新结点集合中选取两个概率最小的结点进行合并，如此继续进行下去，直至全部结点合并完毕。
- 最后得到的根结点的概率值为1。
- 每个新结点都有两个分支，分别用带有箭头的线表示，并分别用一位代码“0”和“1”标注。
- 从根结点开始，沿尖头所指方向寻找到达属于该指令概率结点的最短路径，把沿线所经过的代码排列起来就得到了这条指令的操作码编码。

操作码的 **最短平均长度** 可通过如下公式计算:

$$H = -\sum_{i=1}^{n} p_i \cdot log_2 p_i$$

固定长编码相对于最优 Huffman 编码的 **信息冗余量**:

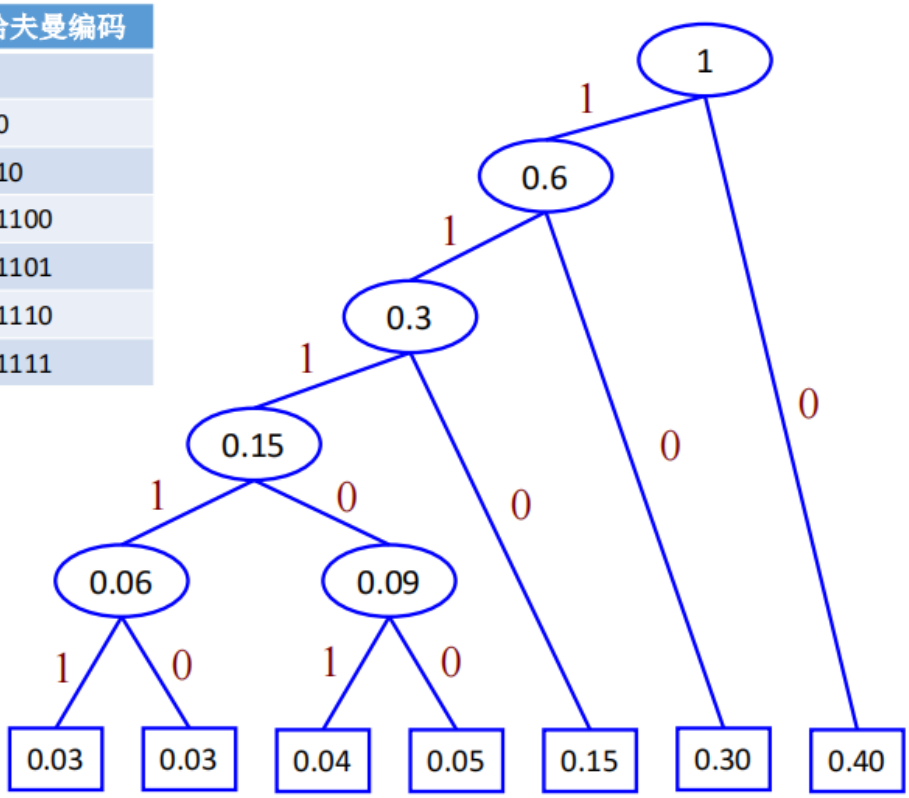
$$R = 1 - \frac{-\sum_{i=1}^{n} p_i \log_2 p_i}{\lceil \log_2 n \rceil}$$

p_i 表示第 i 种操作码在程序中出现的概率。

例：假设一台模型计算机共有7种不同的操作码，如果采用固定长操作码需要3位。已知各种操作码在程序中出现的概率如下表，计算采用Huffman编码法的操作码平均长度，并计算固定长操作码和Huffman操作码的信息冗余量。

指令序号	I_1	I_2	I_3	I_4	I_5	I_6	I_7
出现的概率	0.40	0.30	0.15	0.05	0.04	0.03	0.03

指令	频度p_i	哈夫曼编码
1	0.40	0
2	0.30	10
3	0.15	110
4	0.05	11100
5	0.04	11101
6	0.03	11110
7	0.03	11111



- 采用Huffman编码法的操作码平均长度为：

$$H = \sum_{i=1}^{7} p_i \cdot l_i = 2.20$$

- 操作码的最短平均长度为：

$$H_{opt} = -\sum_{i=1}^{7} p_i \log_2 p_i = 2.17$$

- 采用3位固定长操作码的信息冗余量为：

$$R = 1 - \frac{H_{opt}}{\lceil \log_2 7 \rceil} = 1 - \frac{2.17}{3} \approx 27.7\%$$

- Huffman编码法的信息冗余量仅为：

$$R = 1 - \frac{2.17}{2.20} \approx 1.36\%$$

3. 扩展编码法

Huffman操作码的主要缺点：

- 操作码长度很不规整，硬件译码困难；
- 与地址码共同组成固定长的指令比较困难。

扩展编码法：界于定长二进制编码和完全哈夫曼编码之间的一种编码方式，操作码的长度不是定长的，但是只有有限几种码长。仍然采用高概率指令用短码、低概率指令用长码的哈夫曼编码思想。

指令	频度p_i	哈夫曼编码	OP长度	扩展操作码	OP长度
1	0.40	0	1	00	2
2	0.30	10	2	01	2
3	0.15	110	3	10	2
4	0.05	11100	5	1100	4
5	0.04	11101	5	1101	4
6	0.03	11110	5	1110	4
7	0.03	11111	5	1111	4

- 哈夫曼编码：平均长度=2.2，信息冗余度1.36%
- 扩展操作码：平均长度=2.3，信息冗余度5.65%

操作码 4-8-12 等长扩展编码法

操作码编码	说　明
0000 0001 …… 1110	4 位长度的 操作码共 15 种
1111 0000 1111 0001 …… 1111 1110	8 位长度的 操作码共 15 种
1111 1111 0000 1111 1111 0001 …… 1111 1111 1110	12 位长度的 操作码共 15 种

等长 15/15/15……扩展法

操作码编码	说　明
0000 0001 …… 0111	4 位长度的 操作码共 8 种
1000 0000 1000 0001 …… 1111 0111	8 位长度的 操作码共 64 种
1000 1000 0000 1000 1000 0001 …… 1111 1111 0111	12 位长度的操 作码共 512 种

等长 8/64/512……扩展法

扩展方法主要有两种，一种是全 1 扩展，否则为该种编码；令一种是首位为 0 表示该种编码，首位为 1 表示扩展的下一种编码。在 4-6-10 扩展编码中，5 种可能的扩展方式如下图所示（其中第三种的 6 位操作码全 0 和全 1 都表示扩展）：

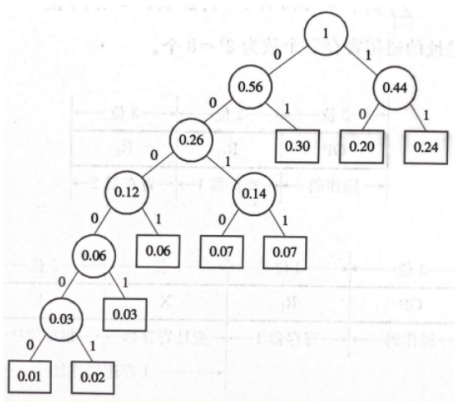
不等长操作码扩展编码法(4-6-10 扩展编码)

编码方法	各种不同长度操作码的指令			指令种类
	4 位操作码	6 位操作码	10 位操作码	
15/3/16	15	3	16	34
8/31/16	8	31	16	55
8/30/32	8	30	32	70
8/16/256	8	16	256	280
4/32/256	4	32	256	292

例题:

某模型机9条指令的使用频度如下表所示。要求有两种指令字长，都按双操作数指令格式编排，采用扩展操作码，并限制只能有两种操作码码长。设该机有若干通用寄存器，主存为16位宽，按字节编址，采用按整数边界存储，任何指令都在一个主存周期中取得，短指令为寄存器-寄存器型，长指令为寄存器-主存型，主存地址应能变址寻址。

指令	频度	指令	频度	指令	频度
ADD（加）	30%	SUB（减）	24%	JOM（按负转移）	6%
STO（存）	7%	JMP（转移）	7%	SHR（右移）	2%
CIL（循环左移）	3%	CLA（清加）	20%	STP（停机）	1%



指令	频度	哈夫曼编码	扩展编码
ADD（加）	30%	01	00
SUB（减）	24%	11	01
CLA（清加）	20%	10	10
JOM（按负转移）	6%	0001	11000
STO（存）	7%	0011	11001
JMP（转移）	7%	0010	11010
SHR（右移）	2%	000001	11011
CIL（循环左移）	3%	00001	11100
STP（停机）	1%	000000	11101

- 哈夫曼编码平均码长：2.61
- 2-5扩展编码平均码长：2.78

操作码（2位）	寄存器1（3位）	寄存器1（3位）

操作码（5位）	寄存器1（3位）	变址寄存器（3位）	相对位移（5位）

6. RISC 处理机

1. RISC 的定义与特点

- **减少指令和寻址方式种类**：指令系统只选择高频指令，减少指令数量，一般不超过100条；减少寻址方式，一般不超过两种；
- **固定指令格式**：精简指令格式限制在两种以内，并使全部指令都在相同的长度；
- **大多数指令在单周期内完成**：让所有指令都在一个周期内完成；
- **采用LOAD/STORE结构**：扩大通用寄存器数量（一般不少于32个），尽量减少访存，所有指令只有存（STORE）/取（LOAD）指令可以访存，其他指令只对寄存器操作；
- **硬布线逻辑**：大多数指令采用硬联逻辑，少数指令采用微程序实现，提高指令执行速度；
- **优化编译**：通过精简指令和优化设计编译程序，简单有效地支持高级语言实现；
- **重视流水线效率**：要提高 RISC 处理机的速度，必须采用流水线，而且要尽量减少断流。

2. RISC 思想的精华

- **减少 CPI 是 RISC 思想的精华；**

  程序的执行时间的计算公式：

$$T_{program} = IC \cdot CPI \cdot \triangle t$$

  - 对于 IC 而言，RISC 的指令条数比 CISC 多 35% 左右；
  - 对于 CPI 而言，CISC 一般是微程序实现的，比较慢，大多数机器的 CPI 大约为 4~6，而 RISC 是硬布线实现的，大多数指令是单周期执行的，其 CPI 大约是 1.1~1.2；
  - 对于 △t 而言，硬布线的时钟周期通常会小一些，目前 RISC 处理机的工作主频一般比 CISC 处理机高。

3. RISC 的关键技术

- **延迟转移技术**：为了使指令流水线不断流，在转移指令之后插入一条不会发生冲突的有效指令，而转移指令好像被延迟执行了一样，这个过程由编译器自动优化。
  - 如果找不到不会引起冲突的指令，必须在条件转移指令后面插入空操作；
  - 如果指令的执行过程分为多个流水段，则要插入多条指令，插入 1 条指令成功的概率比较大，插入 2 条及以上指令成功的概率明显下降。
- **指令取消技术**：由于延时转移技术很容易找不到符合条件的指令，因此许多 RISC 机采用指令取消技术，分为向后转移和向前转移。
  - 向后转移：适用于循环程序（while 和 do while、for），循环体的第一条指令安放在两个位置，分别在循环体的前面和后面。如果转移成功，则执行循环体后面的指令，然后返回到循环体开始；否则取消循环体后面的指令。由于循环程序绝大多数情况下转移可以成功，仅有最后一次出循环时转移不成功，因此这种技术能使流水线保持很高的效率。





- 向前转移：适用于条件判断语句（if then），如果转移不成功，执行转移指令之后的下条指令，否则取消下条指令，相当于插入了一条空指令。转移成功与不成功的概率通常各 50%，因此这种方式还是可能会引起流水线断流。

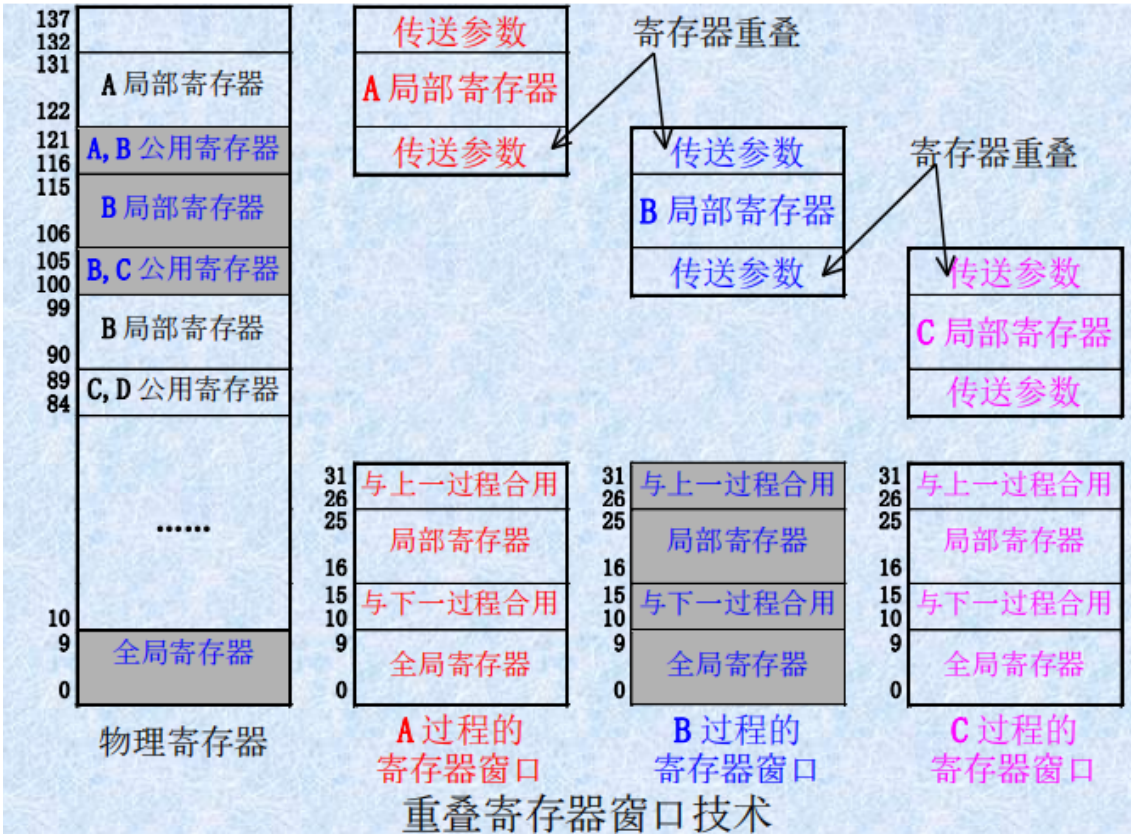


- **重叠寄存器窗口：**由于在 RISC 中，子程序 CALL 和 RETURN 频率高于 CISC，势必会引起频繁访问主存，因此可以设置一个数量较大的寄存器堆，并把它划分成很多个窗口，每个窗口有特定的用途，以此来改善程序调用引起的访存操作。

  在 RISC II 中，共 138 个寄存器，分为 17 个窗口，其中：

  - 有 1 个全局窗口，由 10 个寄存器组成，所有过程都可访问；
  - 有 8 个局部窗口，各由 10 个寄存器组成，能被 8 个过程的局部访问；
  - 有 8 个重叠寄存器窗口，各由 6 个寄存器组成，是相邻两个寄存器共用的。

  每个过程可以访问 1 个全局窗口 + 1 个局部窗口 + 2 个重叠寄存器窗口（一个与上一过程共用，另一个与下一过程共用）。根据调查，RISC II 的访存次数主要是寄存器窗口的溢出引起的，但是很少，只占千分之一左右，对总的性能影响不大。



- **指令流调整技术：**通过调整指令序列和寄存器重命名来消除数据相关或部件相关，以此来提高流水线效率。
- **采用高速缓冲存储器Cache：**设置指令 Cache 和数据 Cache 分别存放指令和数据，保证向指令流水线不间断地输送指令和存取数据，提高流水线效率。
- **优化设计编译系统：**由于使用了大量寄存器，因此要优化寄存器分配；由于要减少流水线断流，因此要调整指令的执行序列，并与硬件配合实现延时转移技术和指令取消技术；由于 CISC 中一条指令在 RISC 中需要一段子程序来实现，所以要设计复杂的子程序库。

第三章 存储系统

1. 存储系统

1. 存储系统的定义

两个或两个以上速度、容量和价格各不相同的存储器用硬件、软件、或软硬件相结合的方法连接起来成为一个存储系统。这个存储系统对应用程序员是透明的，并且，从应用程序员看，它是一个存储器，这个存储器的速度接近 **速度最快** 的那个存储器，存储容量与 **容量最大** 的那个存储器相等，单位容量的价格接近 **最便宜** 的那个存储器。

主要有两种存储系统，一种是由 Cache 和主存构成的 Cache 存储系统，另一种是由主存和磁盘构成的虚拟存储系统。

- **Cache 存储系统：**Cache 一般用 SRAM 实现，存储周期约为几十毫微妙，比较昂贵；主存一般用 DRAM 实现，存储周期为几百毫微秒，比 Cache 慢 5~10 倍，8G 的主存一般配 64M 的 Cache。
- **虚拟存储系统：**虚存需要通过操作系统的存储系统来调度，因此对程序员是不透明的，存储容量比主存大得多。

2. 存储系统的评价标准

1. 存储容量 S

对存储系统进行编址的要求是：对计算机的使用者提供尽可能大的地址空间，且能够随机访问。对于 Cache 存储系统，选择主存进行编址，对 Cache 内部采用相联访问的方式进行管理；对于虚拟存储系统，设计了另一套虚拟地址空间，它既不是主存的地址空间，也不是辅存的地址空间，这个虚拟地址空间比主存要大得多。

需要注意，并非整个磁盘都作为虚拟存储系统，只有在多任务或多用户操作系统中的交换区或交换文件才是用来做虚拟存储系统的。

2. 单位容量的平均价格 C

整个存储系统的平均单位容量价格计算公式：

$$C = \frac{C_1 \cdot S_1 + C_2 \cdot S_2}{S_1 + S_2}$$

当 S_2 >> S_1 时，C ≈ C_2，例如，64M Cache($3.2C/MB) + 8G Mem($0.36C/MB)，则其综合价格为：

$$C = \frac{\$3.2C/M \cdot 64M + \$0.36C/M \cdot 8192M}{64M + 8192M} = \$0.38C/MB$$

3. 访问周期 T

命中率定义：在M1存储器中访问到的概率：

$$H = \frac{N_1}{N_1 + N_2}$$

其中：N1 是对 M1 存储器的访问次数，N2 是对 M2 存储器的访问次数。

访问周期与命中率的关系：

$$T = H \cdot T_1 + (1 - H) \cdot T_2$$

存储系统的访问效率：

$$e = \frac{T_1}{T} = \frac{T_1}{H \cdot T_1 + (1 - H) \cdot T_2} = \frac{1}{H + (1 - H) \cdot \frac{T_2}{T_1}}$$

结论：要提高存储系统的访问效率，应 **提高命中率 H** 或 **降低两个存储器的速度差距。**

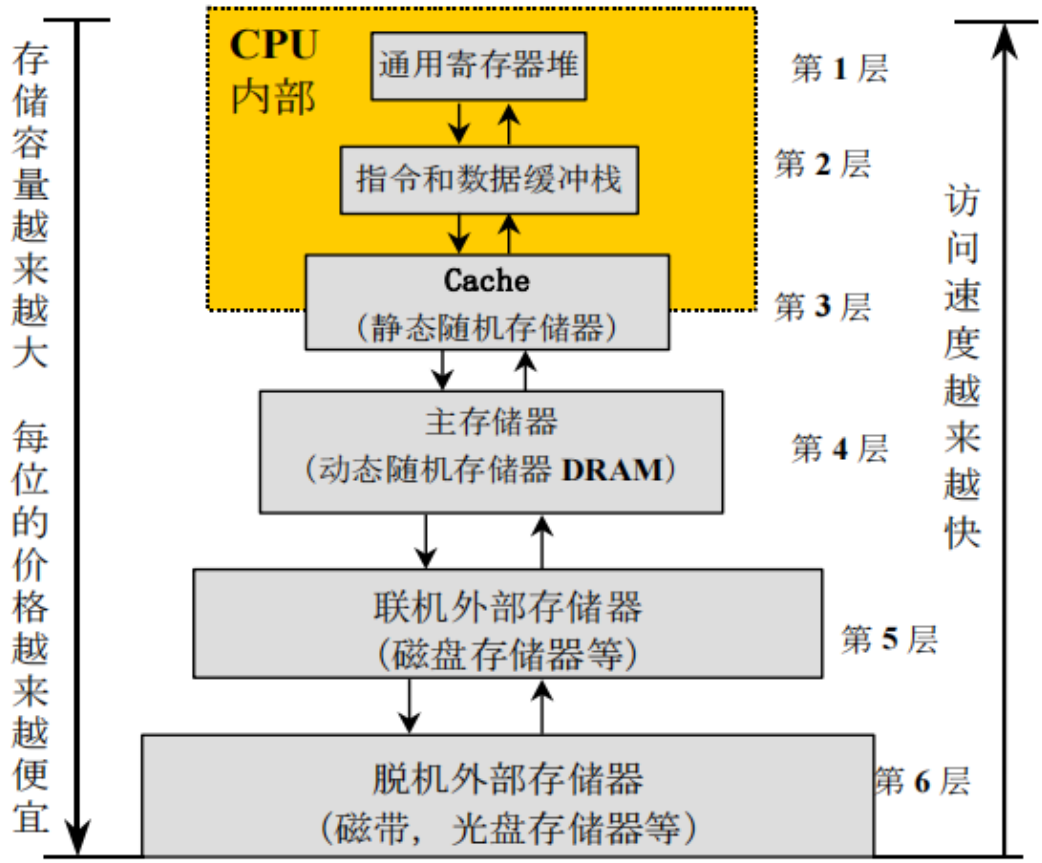
目前 Cache 与主存的速度相差两个数量级，采用多级 Cache 和 CPU 内部的一些缓存寄存器可以使得每两级之间的速度之比为 5 左右，同时采用预取技术提高命中率，可以使存储系统的访问效率较高。

2. 存储系统层次结构

多个层次的存储器：

- 第1层：Register Files（寄存器堆）；
- 第2层：Buffers（先行缓冲站）；
- 第3层：Cache（高速缓冲存储器）；
- 第4层：Main Memory（主存储器）；
- 第5层：On-line Storage（联机存储器）；
- 第6层：Off-line Storage（脱机存储器）。

存储器层次图：



各级存储器的主要主要性能特性：

存储器层次	通用寄存器	缓冲栈	Cache	主存储器	磁盘存储器	脱机存储器
存储周期	<10ns	<10ns	10～60ns	60～300ns	10～30ms	2～20min
存储容量	<512B	<512B	8KB～2MB	32MB～1GB	1GB～1TB	5GB～10TB
价格$C/MB	1200	80	3.2	0.36	0.01	0.0001
访问方式	直接译码	先进先出	相联访问	随机访问	块访问	文件组
材料工艺	ECL	ECL	SRAM	DRAM	磁表面	磁、光等
分配管理	编译器分配	硬件调度	硬件调度	操作系统	系统/用户	系统/用户
带宽(MB/S)	400～8000	400～1200	200～800	80～160	10～100	0.2～0.6

3. 并行存储器

设置多个独立的存储器，让它们并行工作，在一个存储周期内可以访问多个数据，提高存储器的速度。

1. 并行访问存储器

方法：增加存储器的字长，减少存储器的字数，让每个存储周期能访问更多位的数据。

实现：把地址码分成两个部分，一部分作为存储器的地址，另一部分负责控制多路选择器选择数据。

优点：实现简单、容易；

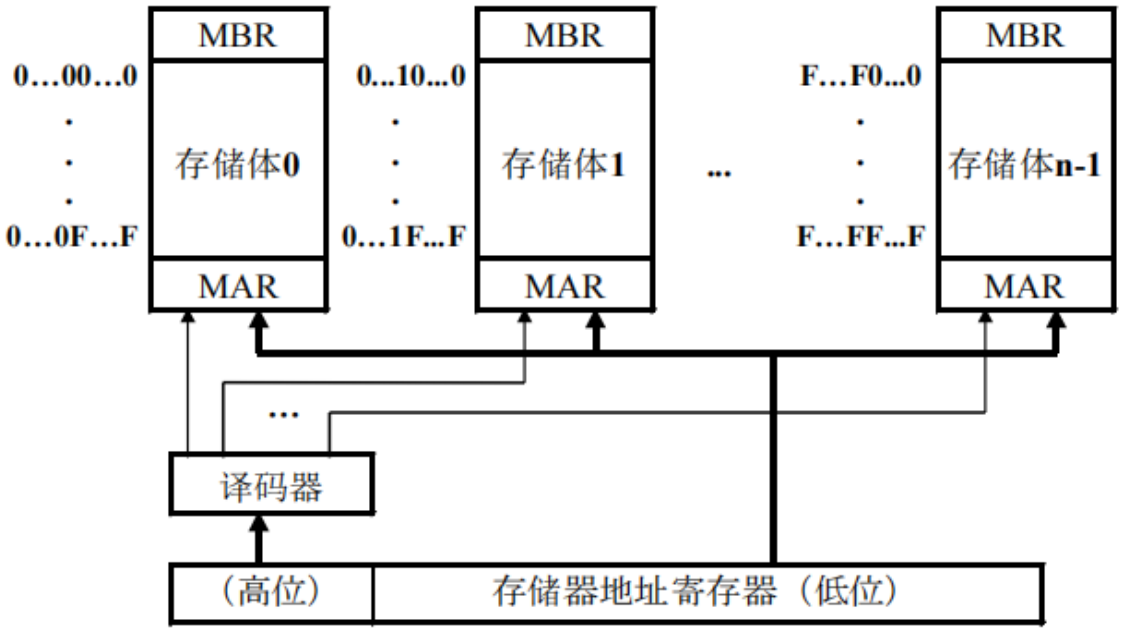
缺点：**①取指令冲突**：当遇到程序转移指令且转移成功时，一个存储周期读出的 n 条指令中，后面的几条指令将无用；**②读数据冲突**：一次读出 n 个操作数，不一定都有用；**③写数据冲突**：需要凑齐n个数据之后才能一起写入存储器；如果只写一个字，必须先把属于同一个存储字的n个数读到数据寄存器中，然后在地址码控制下修改其中一个字，最后把整个存储字写回存储器；**④读写冲突**：当要读出的一个字和写入的一个字处在同一个存储器内时，无法在一个存储周期内完成

本质上来说，引起这些冲突的原因是只有一套控制逻辑，设置多套控制逻辑可以缓解。

2. 交叉访问存储器

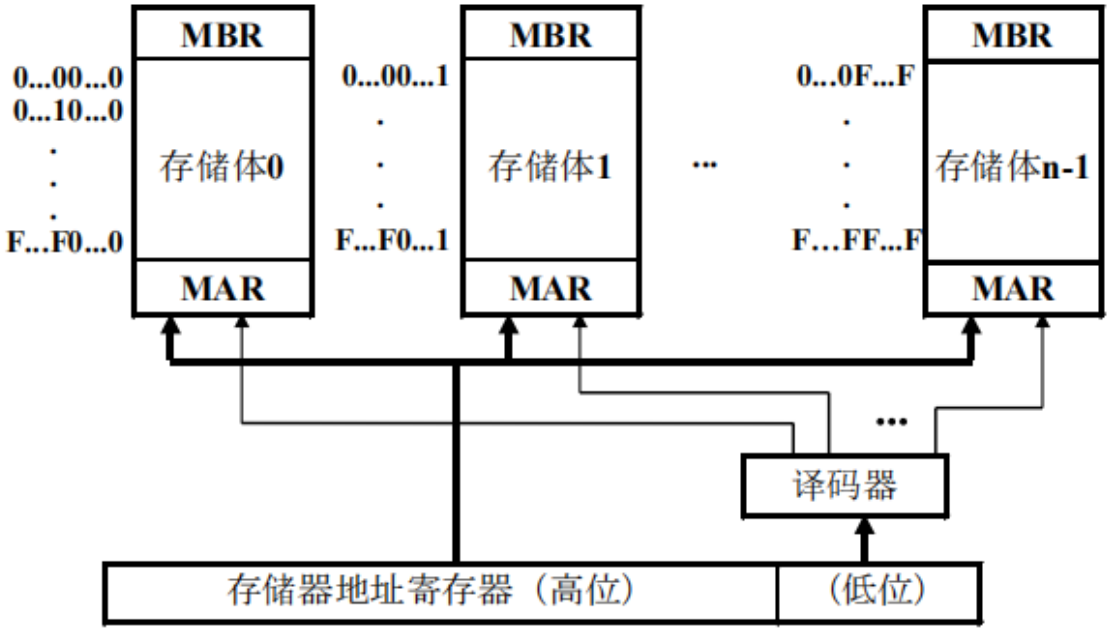
1. 高位交叉访问存储器

主要目的是为了扩大存储器容量，用地址码的高位部分区分存储体号，低位表示体内地址。

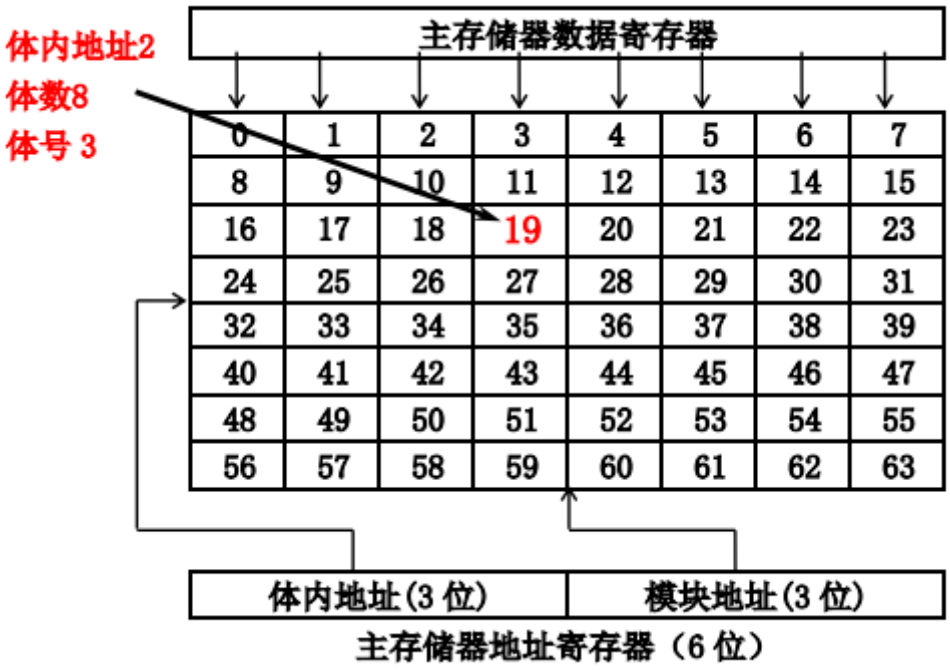


2. 低位交叉访问存储器

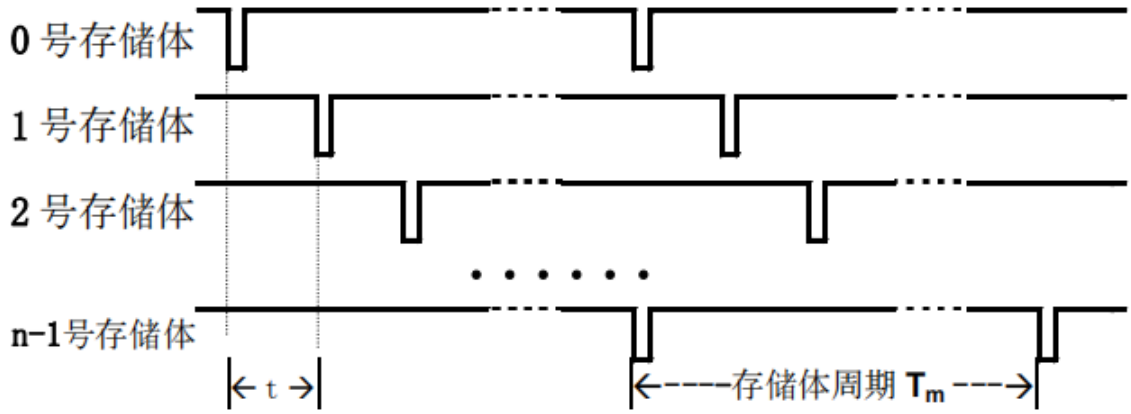
主要目的是为了提高存储器访问速度，用地址码的低位部分区分存储体号，高位表示体内地址。



例如，一个由 8 个存储体构成的，总容量为 64 的主存的地址编址方法如下：



为了达到提高主存速度的目的，低位交叉编址存储器在一个存储周期内，n 个存储体必须采用流水线的方式分时启动，启动的时间图如下：



然而，由于存在访问冲突，主存的加速比通常小于 n，以下是对冲突的分析：

假设有 n 个存储体，就取指令而言，每个存储周期只能取到 k 个有效指令，并且每条指令发生转移的概率为 g：

- 假设 p(k) 是 k 的概率密度函数，即 p(i) 是 k=i 的概率，则 k 的平均值为：

$$N = \sum_{k=1}^{n} k \cdot p(k)$$

  N 是每个存储周期能访问到的平均有效指令个数，也是存储器的加速比。

- 对于 p(k) 而言，有以下关系成立：

$$\begin{aligned} p(1) &= g \\ p(2) &= (1-p(1))g = (1-g)g \\ p(3) &= (1-p(1))(1-p(2))g = (1-g)^2 g \\ &= \cdots\cdots \\ p(n) &= (1-g)^{n-1} \end{aligned}$$

> k=2 表示第 1 个存储体或者取出的不是转移指令，或者转移失败，或者是数据。

将以上关系代入 N 的计算公式中，得出访问 n 个存储体，加速比 N 和指令转移概率 g 的关系为：

$$N = \frac{1-(1-g)^n}{g}$$

下表给出不同的 n 和 g 对于的加速比，g 一般为 0.2 左右，由表可以看出，并行存储体一般不超过 8 为宜，当 > 8 时，加速效果并不明显。

并行存储体个数与程序转移概率的关系

存储体个数	g=0.01	g=0.1	g=0.2	g=0.3	g=0.4
4	3.94	3.44	2.95	2.53	2.18
8	7.73	5.70	4.16	3.14	2.46
16	14.85	8.15	4.86	3.32	2.50
32	27.50	9.66	5.00	3.33	2.50

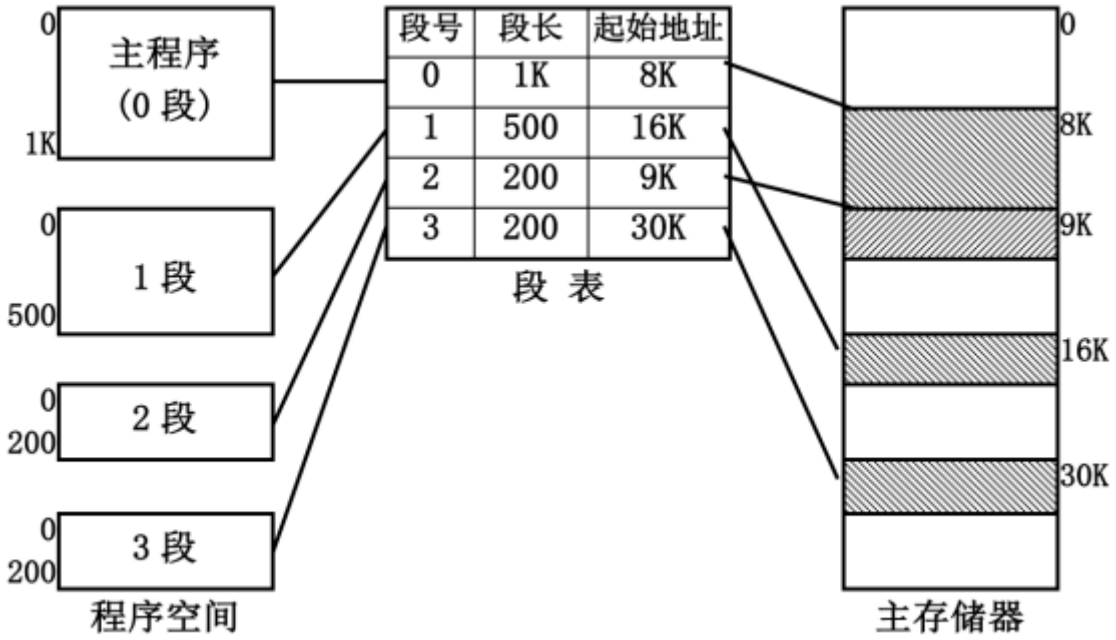
实际对于读操作数和写运算结果，随机性要比取指令大得多，因此低位交叉访问存储器的加速比要比理论值还要低一些。

4. 虚拟存储器

虚拟存储器的工作原理分为段式、页式和段页式三种。

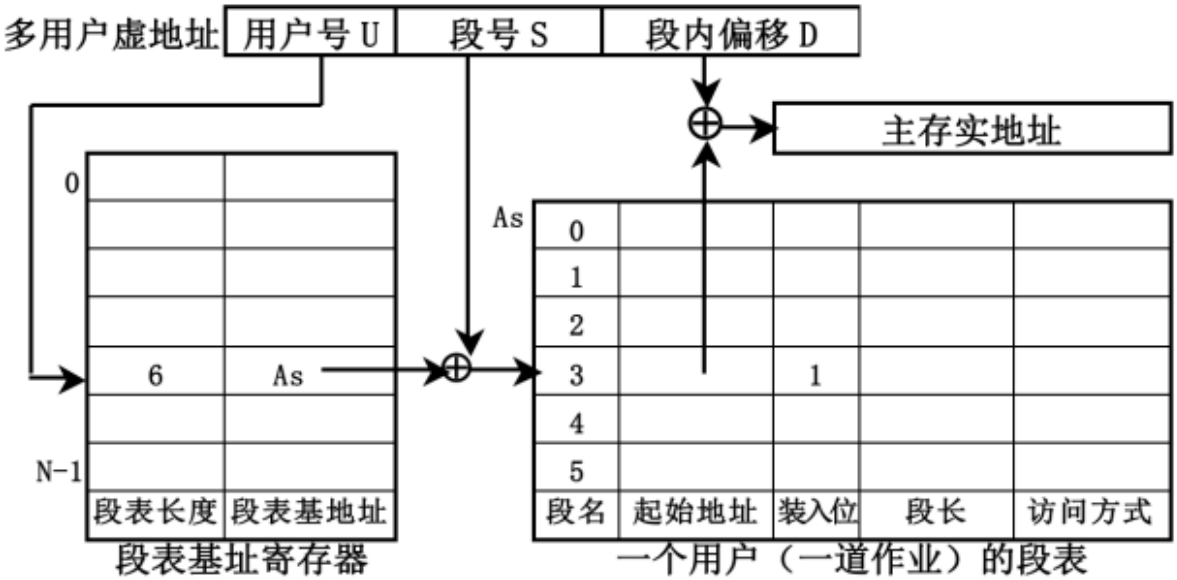
1. 段式虚拟存储器

地址映象方法：每个程序段都从 0 地址开始编址，长度可长可短，可以在程序执行过程中动态改变程序段的长度。每一道程序由一张段表控制，每个程序段在段表中占一行。段号是连续的，因此可以省略。



地址变换方法：一个多用户虚拟地址由三部分组成，用户号 U（或程序号）、段号 S 和段内偏移 D，在 CPU 中通常有一个段表基址寄存器堆，每道程序使用其中一个基址寄存器，因此可以直接通过字段 U 直接找到与该程序定义的基址寄存器。通常，段表是存放在主存中的，从该基址寄存器中就可以直接读出段表的起始地址，把这个地址与虚地址中的字段 S 相加可以得到实段号。

访问这个段表地址，就能得到该程序段的所有信息，如果装入位给出的信息表示要访问的这个程序段已经在主存中，那么只需要把段表中的起始地址与虚地址中的字段 D 相加就可以得到主存实地址。



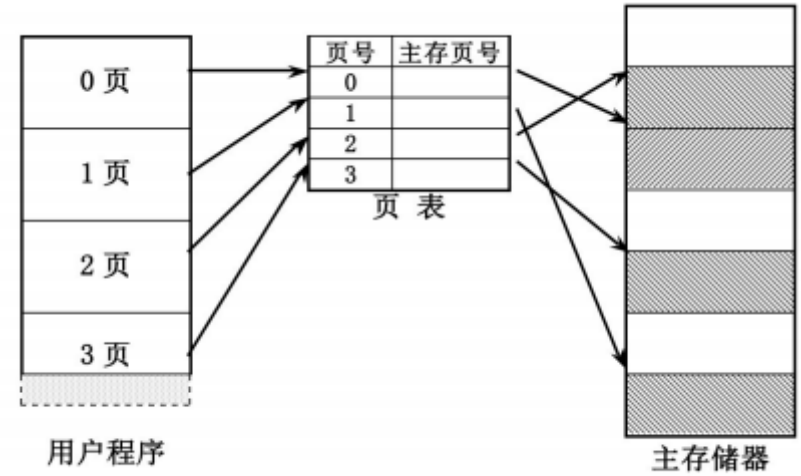
> 段表长度时不确定的，因此段表基址寄存器中有段表长度字段；页表是确定的一页，因此不需要再加页表长度这一项。

段表的优点：①程序模块化好：可由多个程序员并行编写，分别编译和调试，而不会相互影响；②便于程序和数据共享：程序段是按功能划分的，每个段都有完整的功能，不会引起冲突；③程序的动态链接和调度比较容易：可以根据需要一次性把一个程序段或数据段全部装入主存并在装入时实时动态链接；④便于信息保护：由于按照功能划分，因此能更好地区分可读段和可写段。

段表的缺点：①地址变换时间长：从虚地址变换到主存实地址，需要查两次表，做两次加法；②主存利用率低：每个程序段的长度不同，通常会占据一个连续的空间，程序频繁调入调出，使得外部碎片较多，垃圾回收策略的开销也比较大；③对辅存的管理困难：磁盘是按块访问的，如何把不定长的程序段映象到定长的数据块，是一个困难。

2. 页式虚拟存储器

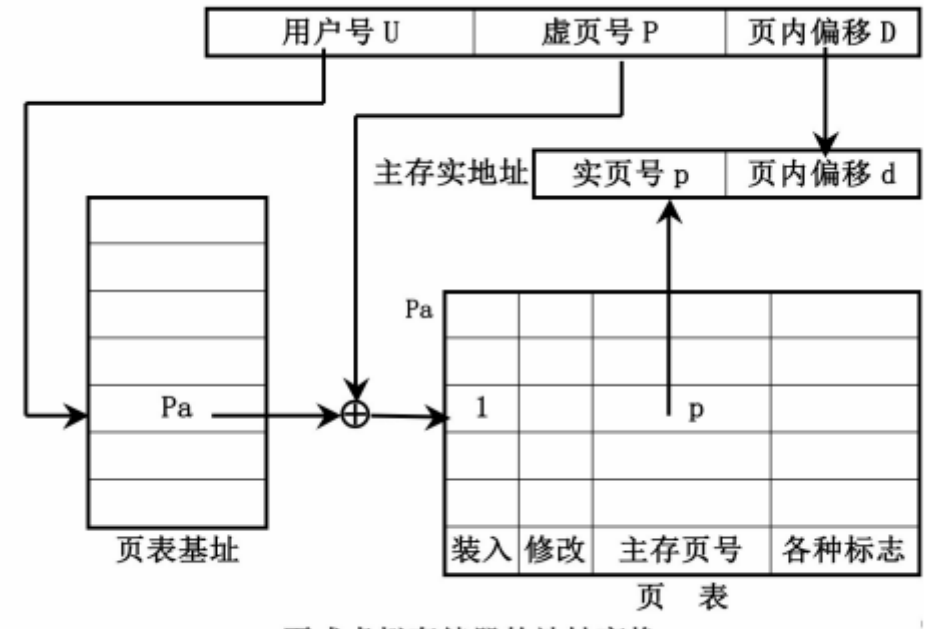
地址映象方法：页式虚拟存储器把虚拟地址空间划分成一个个固定大小的区域，每一区域称为一页，把主存储器的地址空间也按虚拟地址空间同样的大小划分为页。用户的每一页都可以映像到主存的任一页。



页式虚拟存储器的地址映象

地址变换方法：一个多用户虚地址由三部分组成：用户号 U、虚页号 P 和页内偏移 D。在 CPU 内部有一个基址寄存器堆，用来存放页表的基地址，每道程序使用其中的一个基址寄存器，通过虚地址字段 U 可以直接找到与这个用户程序相对应的基址寄存器，从该基址寄存器中读出页表的起始地址。这个用户程序的每一页在页表中都有对应的一行，页表的页号是顺序存放的因此可以省略。将虚页号与基址寄存器的页表起始地址相加可以得到页表项实地址。

然后，访问这个页表项，就能得到被访问页的所有信息。把得到的主存页号 p 与虚地址中的字段 D 拼接起来可以得到主存实地址。



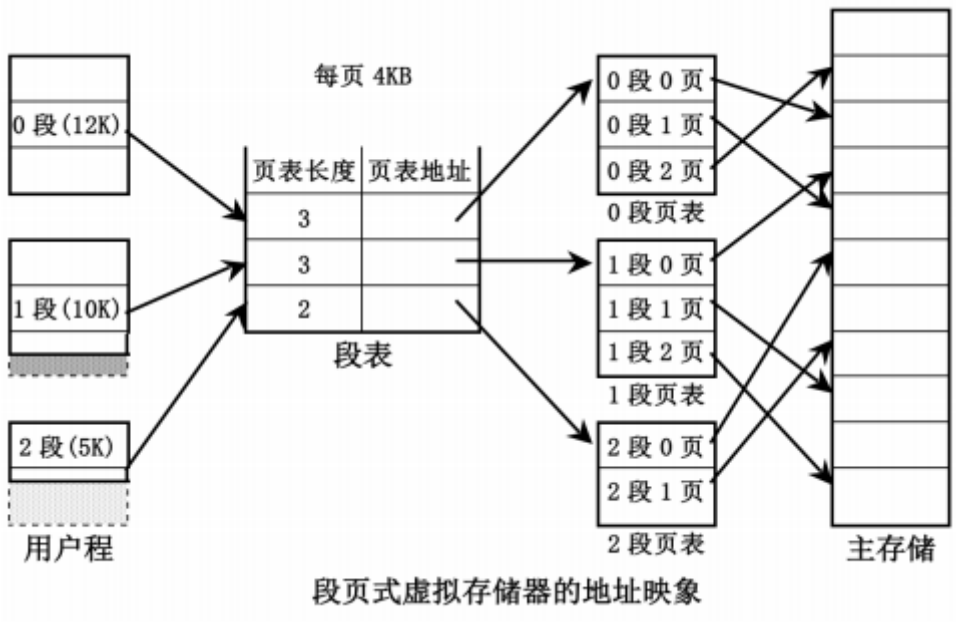
页式虚拟存储器的地址变换

页表的优点：①主存利用率高：每个用户程序只有不到一页的浪费，且避免了外部碎片的产生；②页表实现相对简单：页表基址寄存器中不需要保存页表长度，页表中也不需要保存页长字段，整体空间较小；③地址映象和变换速度快：只需建立虚页号到实页号的对应关系即可，整个过程一次加法；④对辅存管理方便：页大小为辅存物理块的整数倍。

页表的缺点：①程序的模块化性能差：按页划分，失去了每一块的逻辑性；②页表很长：需要占用很大的存储空间，例如，虚地址空间为 4GB，页大小为 1KB，页表项为 4B 的情况下，页表占用空间为 16MB。

3. 段页式虚拟存储器

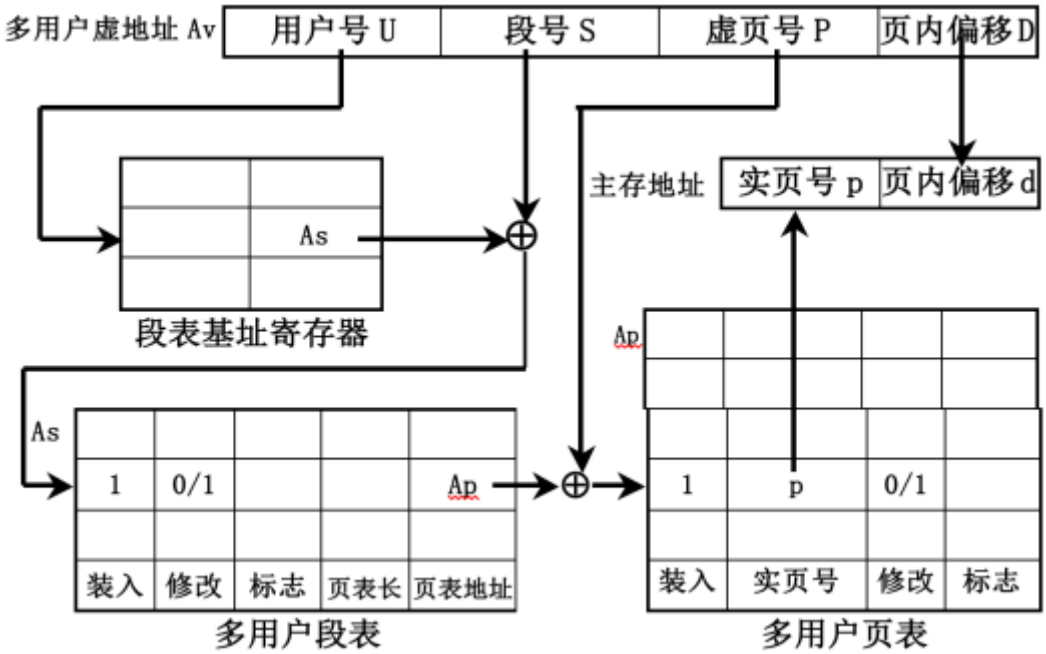
地址映象方法：用户按段写程序，每段分成几个固定大小的页。每个程序段在段表中占一行，在段表中给出页表长度和页表的起始地址，页表中给出每一页在主存储器中的实页号。



段页式虚拟存储器的地址映象

地址变换方法：一个多用户虚地址由四部分组成：用户号 U、段号 S、虚页号 P 和页内偏移 D。进行变换时，首先根据用户号查段表基址寄存器，根据基址寄存器的值得到段表始址，然后查段表。将段表始址与段号 S 相加得到对应的页表始址。将页表始址和虚页号 P 相加得到页表项地址，访问页表项获得实页号 p，最后把实页号 p 和页内偏移 D 拼接得到主存的实地址。

> 这里的页表长度，是指一段中有多少个页表，而不是页表本身的长度。



段页式的优点：①综合了段式和页式的优点；②适合程序员编程，也方便机器处理。

段页式的缺点：①访问速度较慢，需要做 2 次加法；②实现逻辑比较复杂。

5. 加快内部地址变换的方法

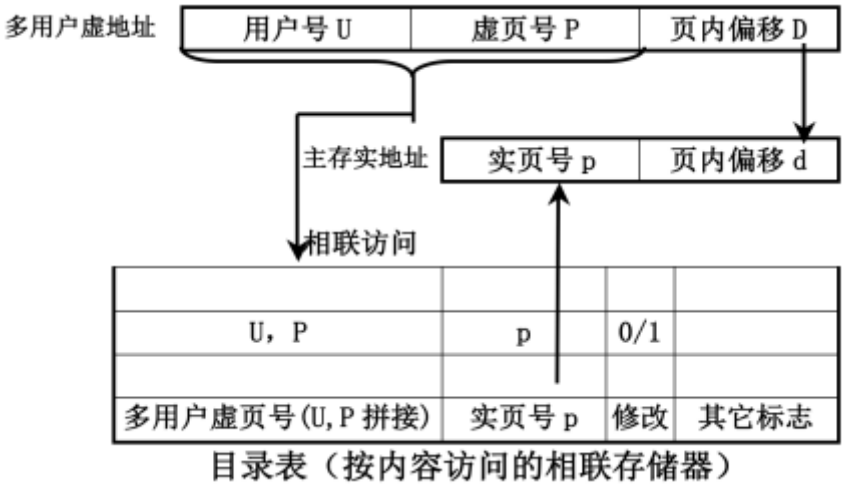
为了保证页表和段表的容量在一个页面以内，应该采用多级页表结构。其中，通常除一级页表必须驻留在主存中外，二级或三级页表只需要驻留一小部分，绝大部分页表可以放在辅存中，使用的时候再装入。

然而，采用多级页表使得访问主存的次数又要增加，因此必须加快查表速度，主要有以下四种方法。

1. 目录表法

基本思想：用一个容量比较小的高速存储器来存放页表，从而加快查表速度。在该存储器中，页表只为装入到主存的页面建立虚页号与实页号之间的对应关系，因此不再需要装入位，并且采用相联访问。

地址变换过程：首先将 U 和 P 拼接起来形成多用户虚页号，将其与相联存储器中的多用户虚页号逐个进行比较，如果有相等的，表示该页面已经装入主存了，可以直接读出实页号字段，将该实页号与页内偏移 D 拼接起来可以直接得到主存实地址。如果没有相等的，则表示要访问的那个页面还没有装入到主存，这时发送失效请求，从辅存中把该页面调入主存。



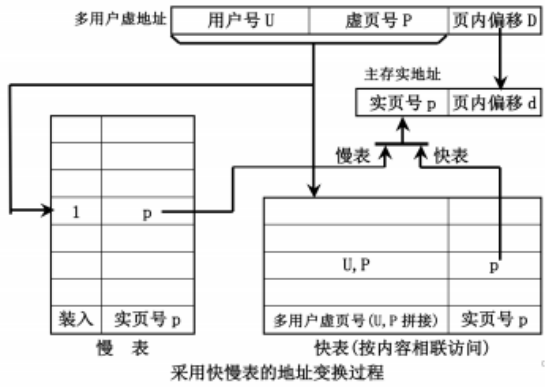
优点：页表不再放在主存中，而是采用高速小容量的相联存储器实现，查表速度快得多。

缺点：随着着主存容量的增加，目录表的容量也必须增加，所以可扩展性较差。当主存容量增加到一定数量，目录表的造价就会很高，查表速度也会降低。

2. 快慢表

基本思想：由于程序访问的局部性，在一段时间内，对页表的访问只是局限在少数几个存储字内。因此，可以大大缩小目录表的存储容量（8-16个存储字），加快访问速度（与CPU中通用寄存器速度相当），此时目录表进化为快表；同时也保留原来的页表，这部分称为慢表。

地址变换过程：用多用户虚页号同时去查快表和慢表（于慢表而言，先访问页表基址寄存器获得页表始址，再根据页表始址和虚页号获得页表项地址），由于快表查表速度快得多，如果快表命中，就立即终止慢表的查询过程，并读出对应的实页号 p 与页内偏移 D 拼接得到主存实地址；反之如果命中失败，则把慢表查到的实页号 p 与页内偏移 D 拼接得到主存实地址，并将该实页号连同多用户虚地址信息送入快表中。如果此时快表已满，则采用相应的替换算法替换掉一个不常用的存储项。



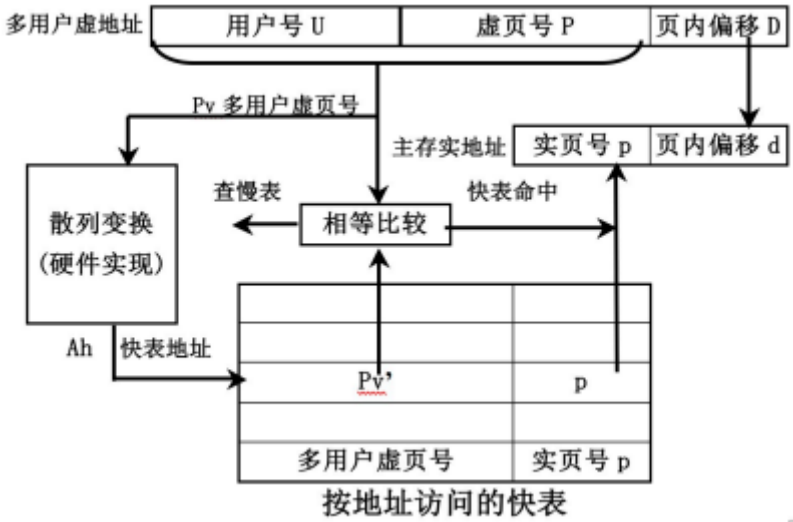
采用快慢表的地址变换过程

- 快表：高速硬件实现，采用相联方式访问。
- 慢表：当快表中查不到时，从主存的慢表中查找；慢表按地址访问；用软件实现。
- 快表与慢表也构成一个两级存储系统。
- 主要存在问题：相联访问实现困难，速度低

3. 散列函数

基本思想：由于快表是按相联访问的，随着项增多访问速度势必会降低，要提高快表命中率，可以舍弃相联访问的办法，采用散列查找方法，速度达到最快。采用一些硬件电路可以实现散列函数，例如将 15 位多用户虚页号变换为 6 位快表索引地址。为了避免散列冲突，需要在命中索引后将快表项中保存的多用户虚页号与给定的多用户虚页号进行相等比较。

地址变换过程：首先将多用户虚页号 Pv（U 和 P 拼接）送入硬件的散列变换部件，经散列变换后得到快表地址 Ah，然后用地址 Ah 访问快表。读出快表项中保存的多用户虚页号 Pv' 并与给定的 Pv 做相等比较，若比较结果相等，则终止查慢表的操作，可以直接拼接主存实地址；若不相等，说明快表 miss，需要继续访问慢表。



虽然有一次相等比较，但是该过程可以和拼接过程同步进行，因此不会增加访问时间。这种结构比之前的快慢表结构相比，命中率高很多，而且查表速度也很快。

6. 主要的页面替换算法

虚拟存储器中的主存页面替换算法一般用软件实现。操作系统为了实现对主存的管理，设置了一个主存页面表，每一项都记录了主存中一个页面的使用情况，主存页面表是面向主存的，放在主存中，整个主存只有一张主存页面表：

实页号	占用位	程序号	段页号	使用位（计数器）	程序优先级	历史位Hb（未使用计数器）	其他
0							
1							
2							
P-1							

1. 随机算法 (RAND)

利用软件或者硬件的随机数产生器产生主存中要被替换的页号，这种算法最简单而且容易实现，但是完全没有利用主存中页面调度的历史信息，也没有反映程序的局部性，所以命中率比较低。

2. 最优替换算法 (OPT)

是一种理论上的最优算法，它选择将来最久没有被访问的页作为被替换页面，这种算法的命中率一定是最高的，仅用来作为评价其它算法好坏的标准。

4. 先进先出算法 (FIFO)

选择最早装入主存的页作为被替换的页 ，容易实现，利用了主存中页面调度情况的历史信息，但是没有反映程序的局部性。

算法的实现：将主存页面表中的 “使用位” 字段设置成一个计数器。每当有一个页面装入主存储器时，让该页面的计数器清零，而其他已经装入主存储器的页面所对应的计数器加1。需要替换时，计数器的值最大的页面就是最先进入主存的页面。

4. 最近最少使用算法 (LRU)

选择近期最少访问的页作为被替换的页 。这种算法不仅利用好了主存中页面调度的历史信息，也能够比较正确地反映程序的局部性，因为目前为止最少使用的页面，很可能也是将来最少访问的页面。

算法的实现：为每个页面设置一个很长的计数器，并且要选择一个固定的时钟为每个计数器定时计数。在选择被替换的页时，要从所有的计数器中找一个计数值最大的计数器。该算法实现起来非常困难，因此通常使用另一种变通的方法，即 LFU 算法。

5. 最近最不常用算法 (LFU)

选择近期最不常用的页面作为被替换页，它把 LRU 算法中要记录数量上的 “多” 与 “少” 简化为判断 “有” 与 ”无“，因此实现起来比较容易。

算法的实现：一般情况下，只需要一个使用位。所有页面的初始使用位均为 0。当页面被访问后，使用位设置为 1。进行替换时，找出使用位为 0 的页面作为被替换页面。但是，不允许所有页面的使用位全部为 1，可以有以下三种策略：

- 当所有页面使用位都为 1 时，把所有页面的使用位清 0，此时，LFU 算法中的 “最近“ 就是从上次清零到这次使用位为全 1 的这段时间；
- 每隔一个固定的时间，把所有页面的使用位清 0，这时的 ”最近“ 就是一段固定的时间；
- 在主存页面表设置一个历史位 Hb（未使用计数器）。每隔一段时间扫描所有页面的使用位，若使用位为 0 则将 Hb 增 1，否则将 Hb 清 0，同时扫描结束后将使用位清 0。因此，Hb越大，说明对应的页面越久没有被访问过，应该成为最先被替换掉的页面。扫描结束后，所有页面的使用位都是 0，相当于又开始了一个 “最近”。

一个程序共有5个页面组成，在程序执行过程中，页面地址流如下：

P1，P2，P1，P5，P4，P1，P3，P4，P2，P4

假设分配给这个程序的主存只有3个页面。

1） 给出用FIFO、LRU和OPT三种页面替换算法对这3个主存页面的调度情况表，并统计页面命中次数。

2） 计算这LRU页面替换算法的页面命中率。

时间 t	1	2	3	4	5	6	7	8	9	10	实际命中次数
页地址流	P1	P2	P1	P5	P4	P1	P3	P4	P2	P4	
先进先出算法(FIFO 算法)	1	1	1	1*	4	4	4*	4*	2	2	
		2	2	2	2*	1	1	1	1*	4	
				5	5	5*	3	3	3	3*	
	调入	调入	命中	调入	替换	替换	替换	命中	替换	替换	2 次
最久没有使用算法(LRU 算法)	1	1	1	1	1*	1	1	1*	2	2	
		2	2	2*	4	4	4*	4	4	4	
				5	5	5*	3	3	3*	3*	
	调入	调入	命中	调入	替换	命中	替换	命中	替换	命中	4 次
最优替换算法(OPT 算法)	1	1	1	1	1	1*	3*	3*	3	3	
		2	2	2	2*	2	2	2	2	2	
				5*	4	4	4	4	4	4	
	调入	调入	命中	调入	替换	命中	替换	命中	命中	命中	5 次

三种页面替换算法对同一个页地址流的调度过程

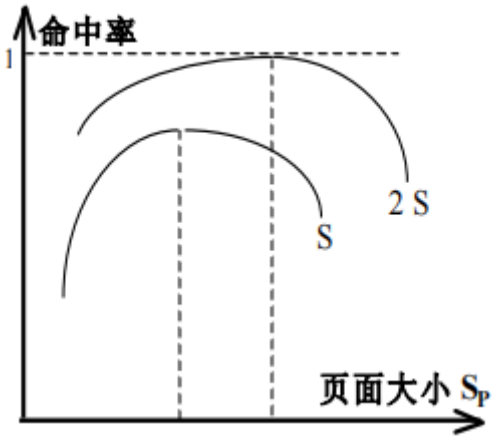
1） FIFO、LRU和OPT的页面命中次数分别为2次、4次和5次

2） LRU页面替换算法的页面命中率为：$H_p=4/10=0.4$

7. 主存命中率的影响因素

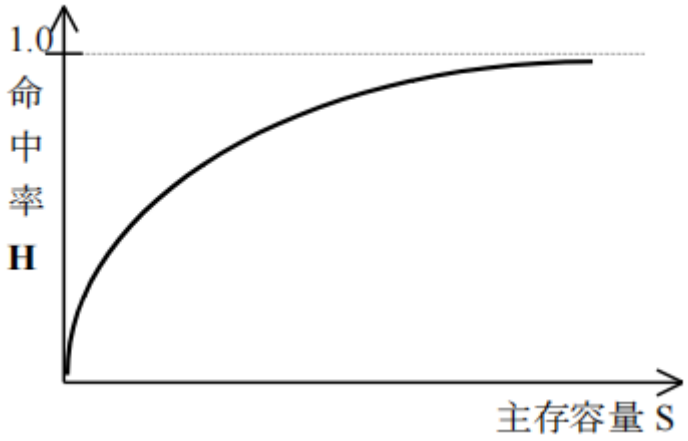
影响主存命中率的主要因素有如下几个：

- 程序在执行过程中的页地址流分布情况：由程序本身决定；
- 所采用的页面替换算法：一般采用 LFU，目前来看是比较理想的；
- **页面大小**：页面大小为某个值时，命中率达到最大。



页面太小可能导致一段局部程序被分为很多页，页面太大可能导致非局部程序频繁缺页，在图中的临界点 S_p 是程序局部性和全局性都兼顾的一个点。此外，页面太小会导致页表和页面表所占空间增大，页面太大会导致内部碎片增多，需要综合考虑以上因素来选择页面大小。

- **主存储器的容量**：主存命中率 H 随着分配给该程序的主存容量 S 的增加而单调上升。在 S 比较小的时候，H 提高得非常快。随着 S 的逐渐增加，H 提高的速度逐渐降低。当 S 增加到某一个值之后，H 几乎不再提高。这说明，在为程序分配空间时，对主存的命中率要求不能过分，选择恰当的主存才能使性价比更高。



- **所采用的页面调度算法**：一般有三种：
  - **分页式**：在程序装入主存之前就对程序进行链接分配，整个程序全部装入主存才开始运行，命中率达到 100%，但是系统并发度低，主存利用率低，小的主存无法运行大程序；
  - **请求分页式**：只在发生页面失效时才把页面调入主存，主存利用率高，但是失效比较频繁，开销较大，特别是在程序刚调入主存的一段时间内命中率很低；
  - **预取式**：根据局部性原理，在程序被挂起后又重新运行之前，把上次活跃的页面一次性调入主存，然后再开始运行程序。主要优点是可以避免程序运行初期频繁缺页，缺点是调入的页面可能用不上，白白浪费了资源。

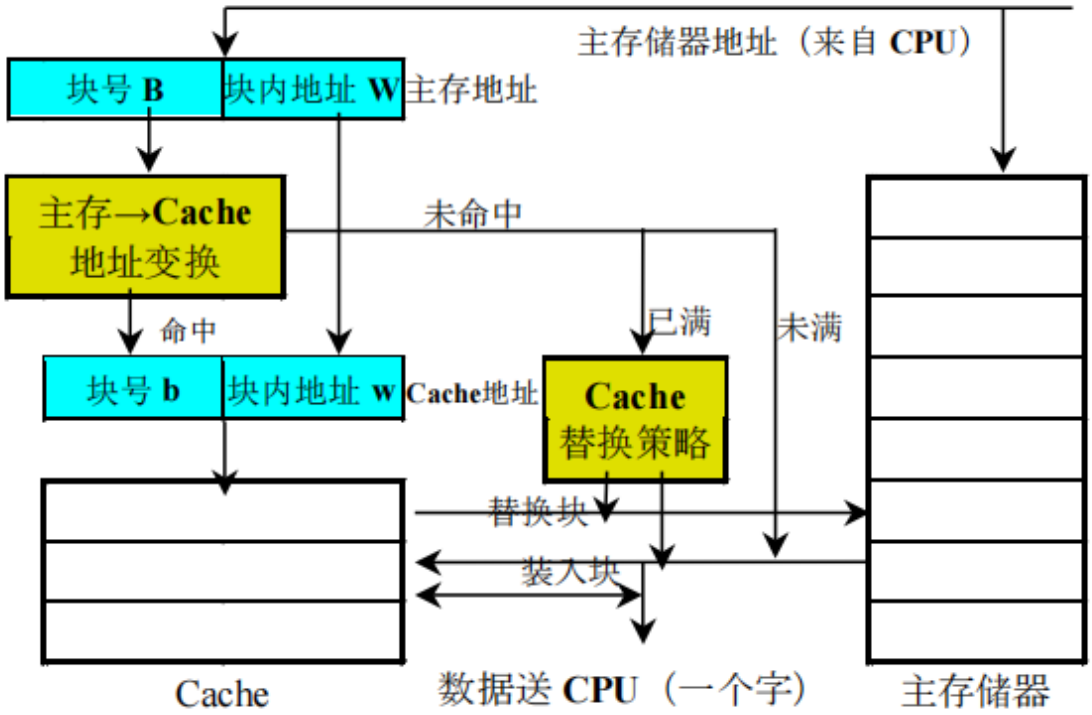
8. Cache 存储系统

一般处理机中有一级 Cache，它与主存储器构成一个两级存储系统，一些高性能处理机都采用两级 Cache，其中第一级在 CPU 内部，第二级在主板上，比第一级慢 5 倍左右；也有三级 Cache 的机器，前两级都在 CPU 内部。

Cache 存储系统与虚拟存储系统的比较

存储系统	Cache 存储系统	虚拟存储系统
要达到的目标	提高速度	扩大容量
实现方法	全部硬件	软件为主，硬件为辅
两级存储器速度比	3～10 倍	10^5 倍
页（块）大小	1～16 字	1KB～16KB
等效存储容量	主存储器	虚拟存储器
透明性	对系统和应用程序员	仅对应用程序员
不命中时处理方式	等待主存储器	任务切换

1. 基本工作原理



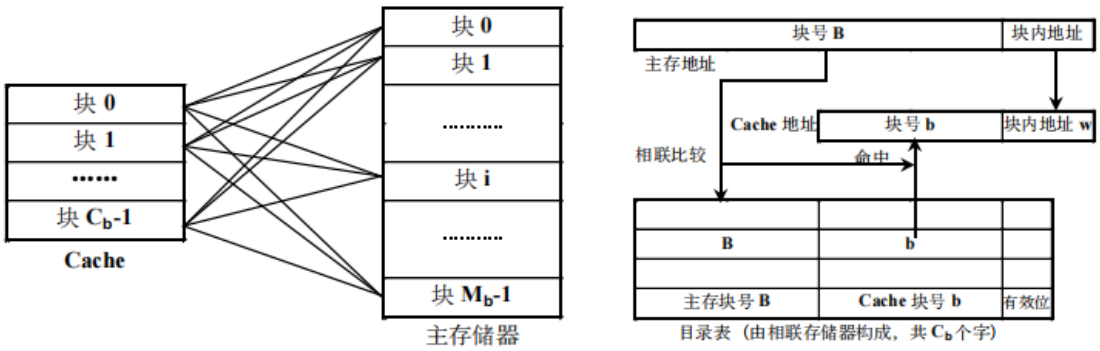
在 Cache 中，**地址映象** 是指主存地址与 Cache 地址之间的对应关系，**地址变换** 是指程序已经装入到 Cache 以后，实际运行过程中如何把主存地址变换为 Cache 地址。

2. 全相联映象及其变换

映象规则：主存的任意一块可以映象到Cache中的任意一块。（映象关系有 $C_b \times M_b$ 种）

地址变换：程序访问 Cache 时，用主存地址中的块号 B 与目录表中的主存块号进行相联比较，若有发现相等的，说明命中，只需读出命中的目录项的 Cache 块号字段，然后将读出的块号 b 和主存块内地址 w 拼接得到 Cache 地址，访问 Cache 得到数据。

如果未命中，此时要用主存地址去访存，把得到的数据送往 CPU，同时送往 Cache 并修改目录表项，另外还要修改主存块号字段，表示当前块已经写到了目录表中。

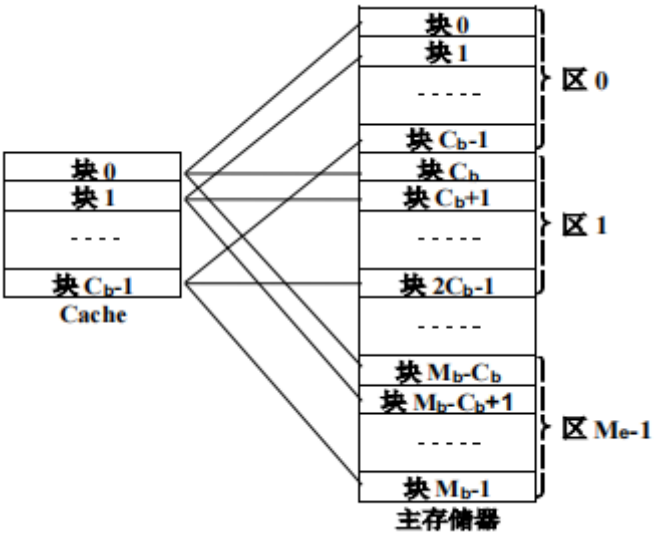


优点：块的冲突率最小，Cache 利用率最高。

缺点：需要一个相联存储器，会降低总体速度，代价很高。

3. 直接映象及其变换

映象规则：主存储器中一块只能映象到 Cache 的一个特定的块中。把主存按 Cache 大小分区，每一分区的块数和 Cache 的块数相等，将它们按顺序映象到 Cache 中，整个 Cache 的地址与主存的低位部分是完全相同的，主存地址比 Cache 地址长出来的部分称为区号 E。

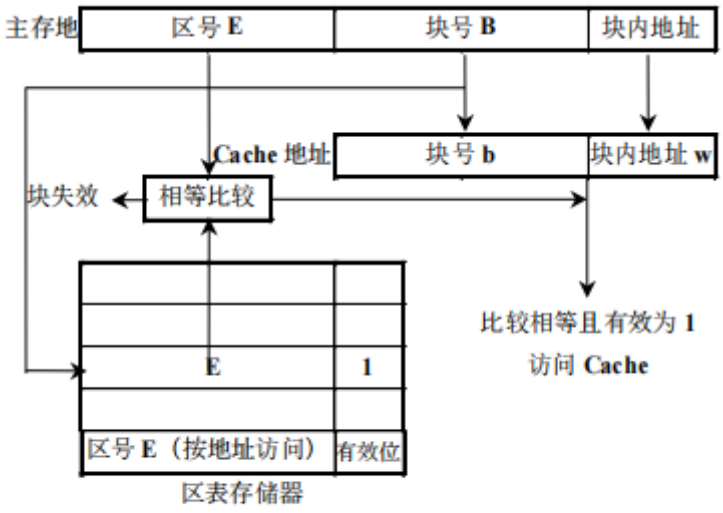


地址变换：为实现变换，需要有一个存放主存区号的小容量存储器，该存储器的容量与 Cache 块数相等，字长为区号 E 的长度，另外再加一个有效位。在变换时，首先用主存地址的块号 B 去访问区号存储器，把读出来的区号与主存地址的区号 E 进行比较，若相等且有效位为 1 则命中，其余情况不命中。

若命中，表示要访问的那一块已经送入 Cache 中了，此时用该 Cache 地址取数据即可；

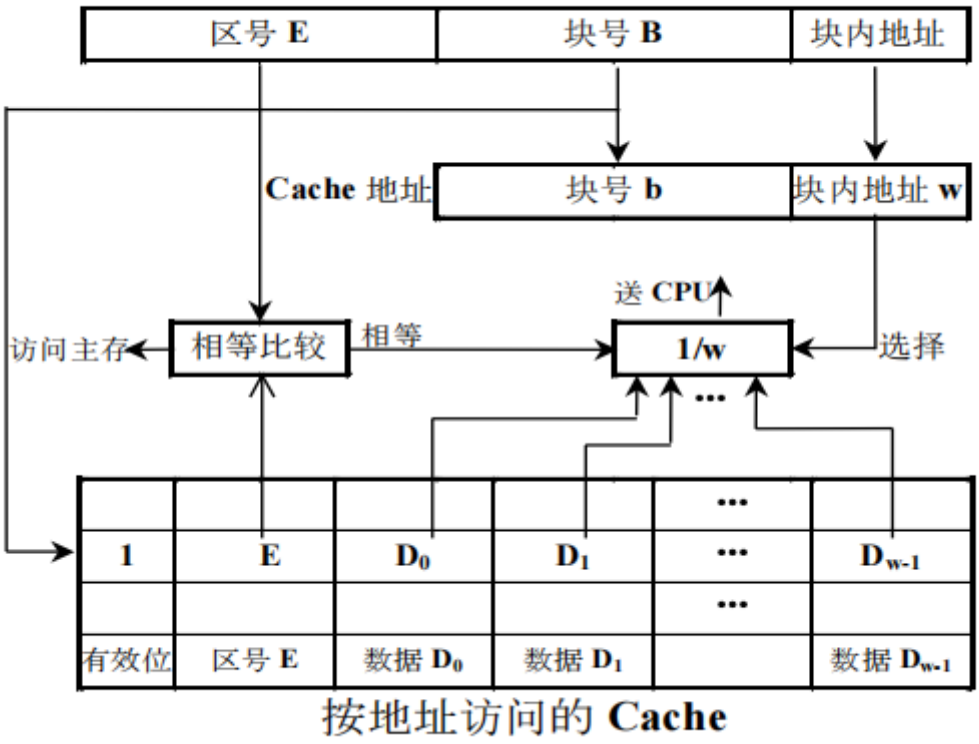
若不命中，先用主存地址访问主存储器，把读出来的字送往 CPU，同时把那一块都从主存中读出来：

- 如果区号比较结果相等但有效位为 0，表示 Cache 中这一块已经作废，这是需要把主存中读出来的新块按照 Cache 块地址装入 Cache，并且把有效位置 1；
- 如果区号比较结果不相等且有效位为 1，表示原来 Cache 中的那一块是有用的，这时需要先把该块写回主存，并把从主存中读出的新块装入 Cache，另外要更新主存区号；
- 如果区号比较结果不相等且有效位为 0，表示表示 Cache 这一块是空的，此时直接把新块装入 Cache 并置有效位为 1，更新主存区号。



直接映象方式的地址变换规则

为了提高访问速度，可以把区号存储器与 Cache 合并为一个存储器，这样用主存块号 B 访问 Cache，把区号和这一块的所有数据都读出来，再执行上述的逻辑，速度会非常快。

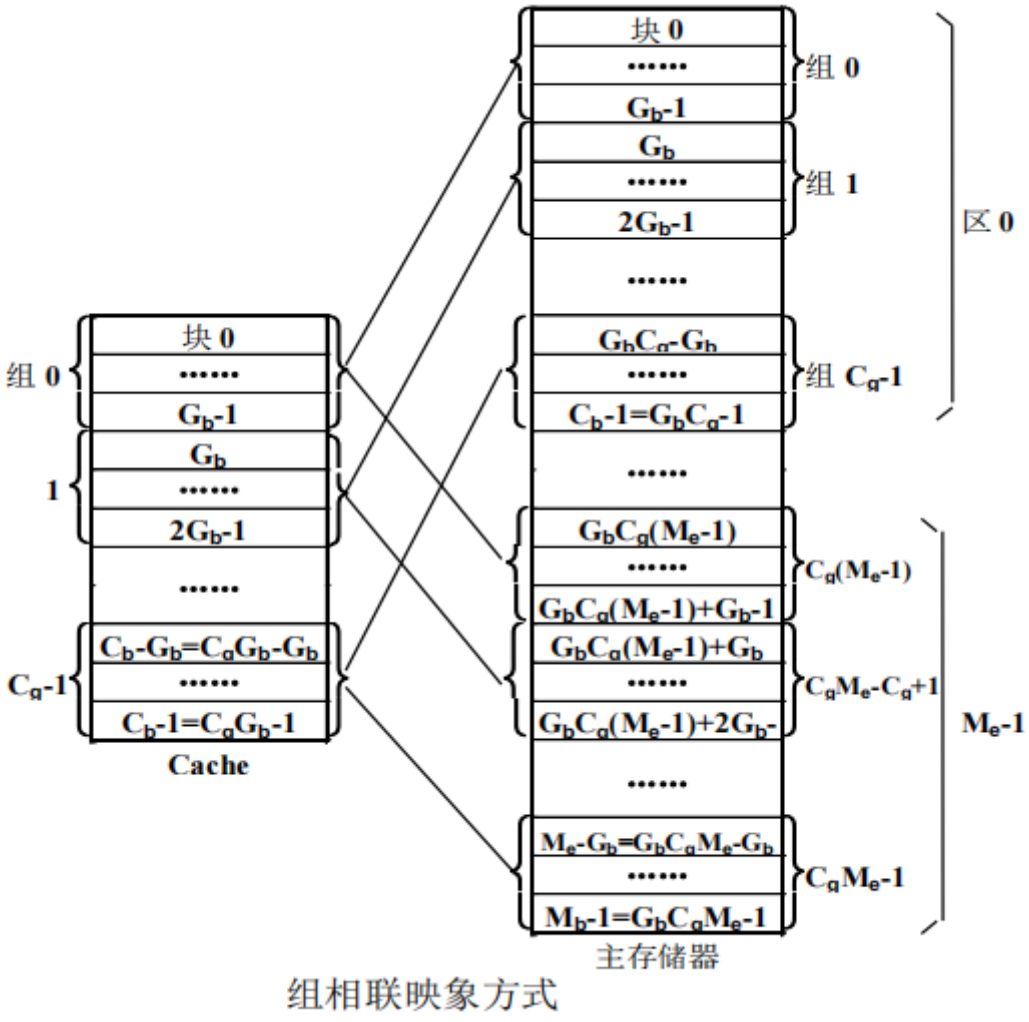


优点：硬件实现简单，不需要采用相联存储器，访问速度比较快，实际上不需要进行地址变换。

缺点：块的冲突率比较高，不够灵活。

4. 组相联映象及其变换

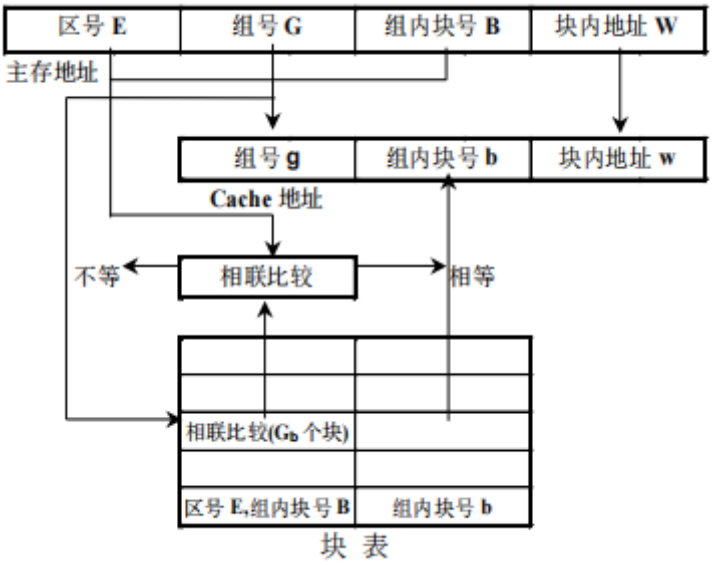
映象规则：主存和 Cache 按同样大小划分为组，主存的每个区内，以组为单位和 Cache 进行直接映象，各组内部采用全相联映象。



组相联映象方式

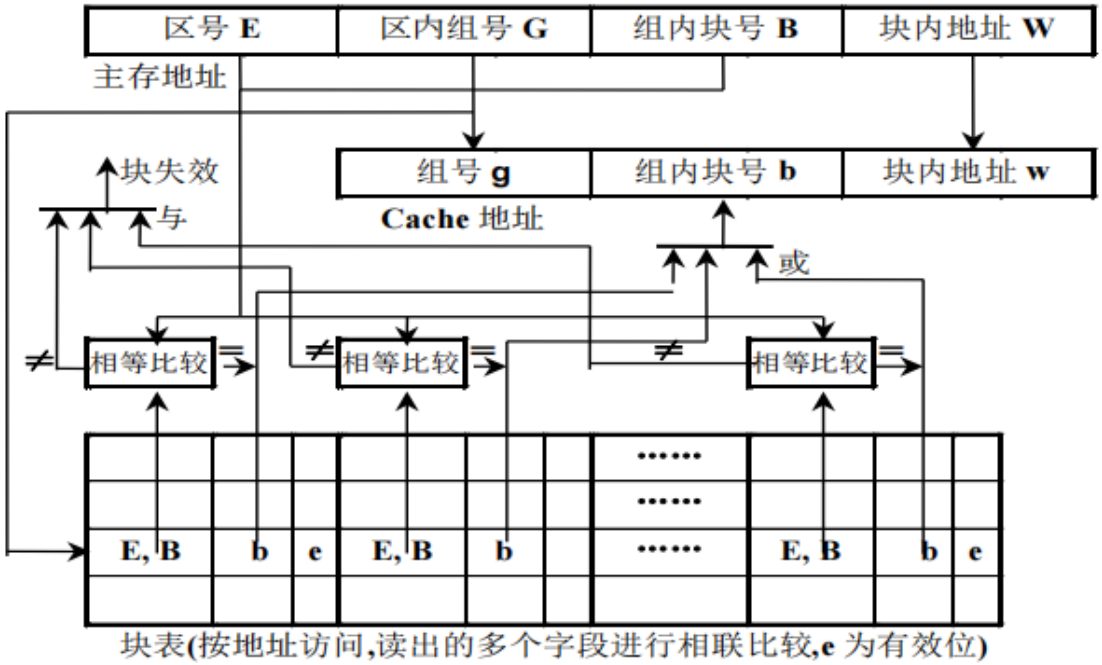
地址变换：为了实现地址变换，需要一个由高速小容量存储器做成的块表存储器，该存储器可以按地址访问和按相联访问，在块内采用相联访问，在块之间采用按地址访问。

首先，由主存地址中的组号 G 按地址访问块表存储器，从块表存储器中读出来的是该组内的所有块，把读出来的一系列区号和主存地址的区号进行相联比较，如果有相等的，表示 Cache 命中，如果全都不相等，表示 Cache 不命中。



块 表

为了提高 Cache 的访问速度，可以把 Cache 的地址变换与访问 Cache 并行操作，并采用流水线工作；也可以把块表存储器中一个相联比较的组按横向展开存放，采用多个相等比较器来并行操作以加快速度。



优点：块的冲突率大大降低，有前两种方法的优点；

缺点：全部采用硬件实现，实现的难度和造价都比较高。

9. Cache 的一致性问题

由于 CPU 写 Cache，没有立即写主存，或由于 IO 处理机写主存而没有更新 Cache，会造成 Cache 不一致。一般有两种 Cache 更新算法：

1. **写直达法（WT）**

   CPU 的数据写入 Cache 时，同时也写入主存。

2. **写回法（WB）**

   CPU 执行写操作时，被写数据只写入 Cache，不写入主存，仅当发送替换时，才把修改过的 Cache 写回主存。

两种方法对比：

	写直达法	写回法
可靠性	高	低
与主存的通信录	少	多
控制的复杂性	简单	复杂
硬件实现的代价	高	低

- 写直达法能够始终保证Cache是主存的副本，如果Cache发生错误，可以从主存得到纠正；
- 写直达法的写次数很多、每次只写一个字；写回法是的写次数很少、每次要写一个块；
- 对于写回法，要为每块设置一个修改位，还要采用复杂的校验电路保证数据一致性，写直达法则不必；
- 写直达法涉及到写操作流水线和数据缓冲站，硬件实现的代价比较昂贵。

写 Cache 的两种方法：

1. **不按写分配法**：在写 Cache 不命中时，只把所要写的字写入主存。
2. **按写分配法**：在写 Cache 不命中时，还把一个块从主存读入 Cache。

目前，在写回法中采用按写分配法，在写直达法中采用不按写分配法。

10. Cache 的预取算法

一般情况下，预取可以大幅度提高 Cache 命中率，算法有如下几种：

1. **不预取：**当出现 Cache 不命中时，才把需要的一个块取到 Cache 中；
2. **恒预取：**无论 Cache 是否命中，都把下一块取到 Cache 中；
3. **不命中预取：**当出现 Cache 不命中，把本块和下一块都取到 Cache 中。

由于预取必然增加 Cache 与主存的通信量，因此考虑预取算法时还要考虑通信量的开销，综合考虑。总的来说，不命中预取可以使命中率降低和通信开销有一个稳定的平衡。

11. Cache 的替换算法

虚拟存储器的页面替换算法主要是软件实现的，Cache 由于速度快，必须全部用硬件实现。主要有以下几种。

1. 轮换法

主要有两种实现方法。

1. 每块一个计数器，被装入或被替换的块，它所属的计数器清 0，同组的其它块所属的计数器都加 1；需要替换时，在同组中选择计数器的值最大的块作为被替换的块。
2. 对于以上算法，实际上就是组内所有块按顺序轮流替换，可以设每组有一个计数器，发生替换时计数器加 1，计数器的值就是要被替换出去的块号。

2. LRU 算法

在块表中为每一块设置一个计数器，计数器的规则是：

- 被装入或被替换的块，它的计数器清 0，同组的其它块所属的计数器都加 1；
- 命中的块，计数器清 0，同组的其它计数器，凡是计数器值小于命中计数器原来值的，都加 1，反之则不变；（发送命中时，命中块之前的那些代码很可能再次使用）
- 需要替换时，选择同组中计数器最大的计数器作为被替换块。

这种算法能正确利用程序局部性的特点，命中率比较高。

块地址流	主存块 1		主存块 2		主存块 3		主存块 4		主存块 5		主存块 4	
	块号	计数器	块号	计数器	块号	计数器	块号	计数器	块号	计数器	块号	计数器
Cache块0	1	00	1	01	1	10	1	11	5	00	5	01
Cache块1		01	2	00	2	01	2	10	2	11	2	11
Cache块2		01		10	3	00	3	01	3	10	3	10
Cache块3		01		10		11	4	00	4	01	4	00
	装入		装入		装入		装入		替换		命中	

3. 比较对法

采用逻辑门来进行替换。

4. 堆栈法

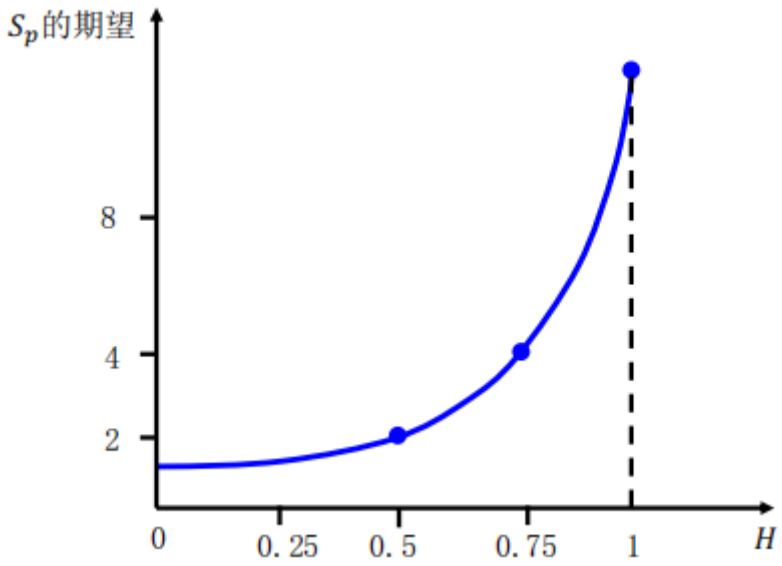
采用堆栈来进行替换。

12. Cache 的加速比

Cache 存储系统的加速比为：

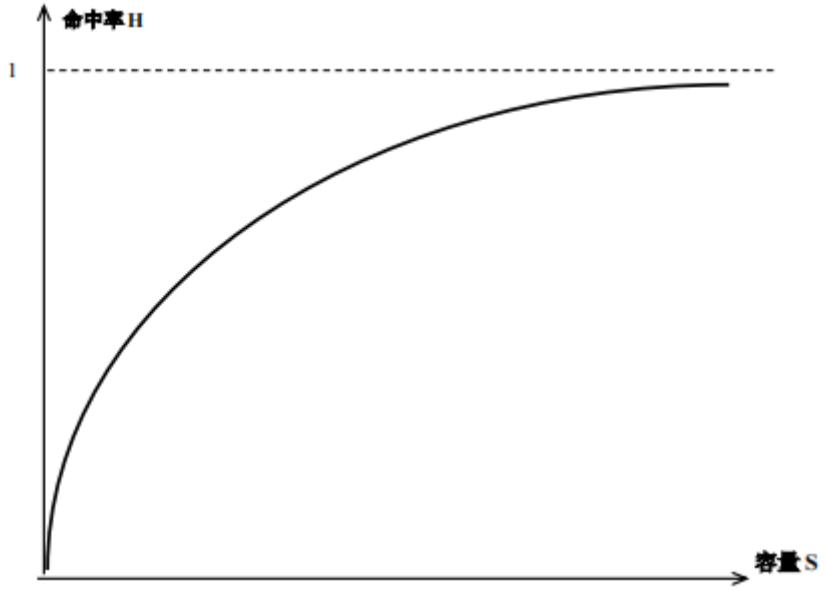
$$S_p = \frac{T_{mem}}{T_{sum}} = \frac{T_{mem}}{H \cdot T_{cache} + (1-H) \cdot T_{mem}} = \frac{1}{(1-H) + H \cdot \frac{T_{cache}}{T_{mem}}}$$

由于主存的访问周期和 Cache 的访问周期受硬件限制基本是确定的，因此提高 Cache 加速比的关键在于 **提高命中率 H。**

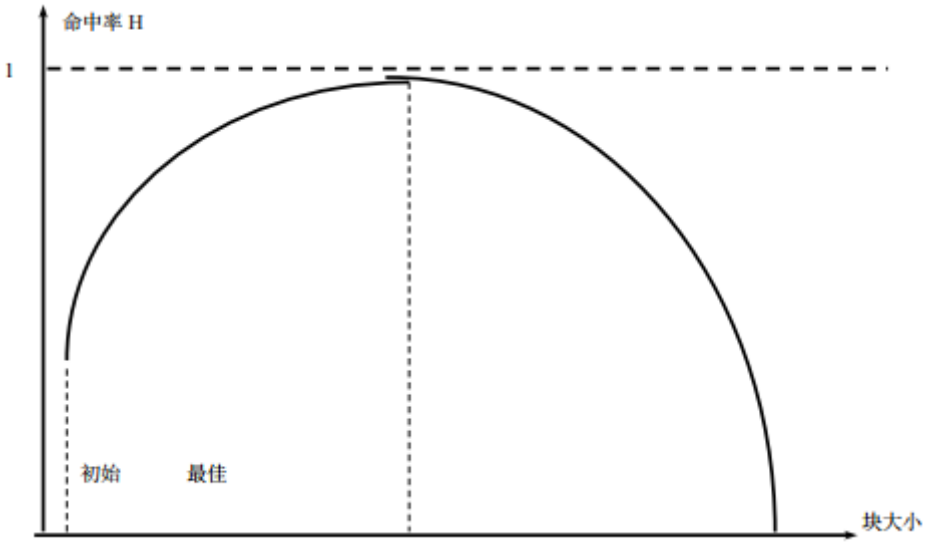


命中率 H 和 3 个因素相关：

- Cache 容量：Cache的命中率随它的容量的增加而提高，Cache容量到达一定程度时，命中率提高很慢。



- Cache 块大小：块很小时，命中率很低；随着块大小增加命中率也增加，有一个极大值；当块非常大时，进入Cache中的数据大量无用；当块大小等于Cache容量时，命中率将趋近零。



- Cache 组数：组数无限降低退化为全相联映象，组数无限增加退化为直接映射，处于中间时可以取得一个很好的平衡。

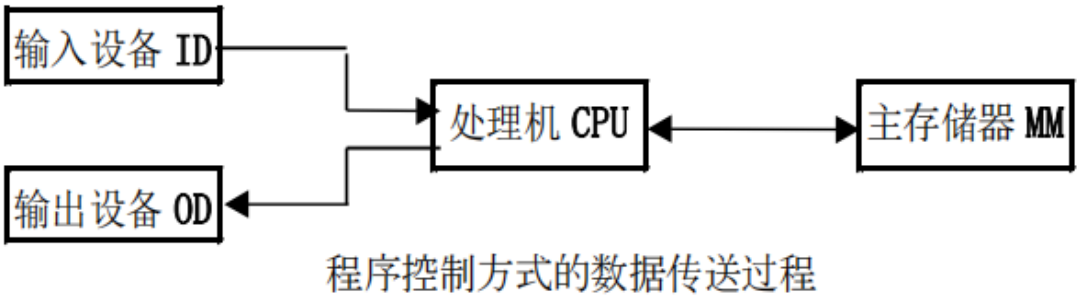
第四章 输入输出方式

1. 基本的输入输出方式

对于工作速度、工作方式和工作性质不同的外设，通常需要采用不同的输入输出方式，目前常用的基本输入输出方式有如下三种。

1. 程序控制输入输出方式（Polling）

由 CPU 控制何时对设备进行输入输出，CPU 通过指令对设备进行测试才能知道设备的工作状态，然后再决定输入输出的时机。数据的输入输出都有经过 CPU，一般用于连接低速外围设备，例如打印机。



程序控制方式的数据传送过程

若一个处理机管理多台外围设备，处理机采用轮询的方法，分时为各台外设服务。优点是灵活性好，可以很方便地改变外设服务的优先级，缺点是不能实现处理机和外设之间的并行工作。

2. 中断输入输出方式

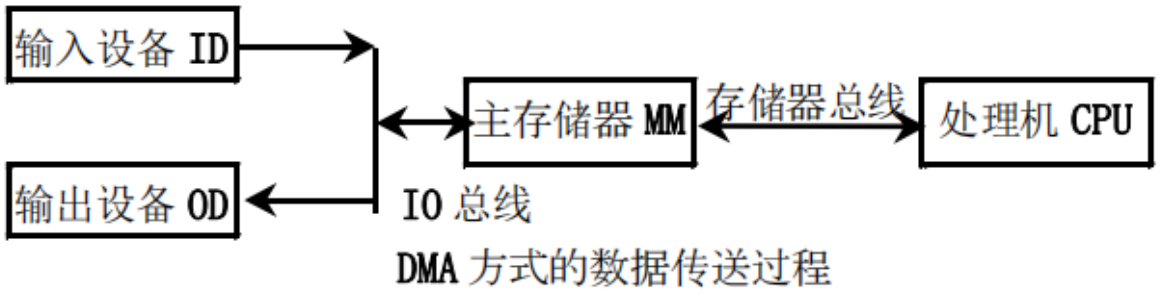
这种方法是为了克服处理机和外设之间不能并行工作的缺点。在这种方式中，外设被动地等待 CPU 来为它服务，每当外设准备好输入数据，就会向 CPU 发送中断请求；CPU 每执行完一条指令都会去测试中断队列，如果发现有外设的中断请求，则暂停当前正在执行的程序，先为外设服务，然后再继续执行原来的程序。

这种方法的特点是：

- CPU 与外设可以并行工作；
- 能够处理突发事件；
- 数据的输入和输出都要经过 CPU；
- 一般用于连接低速外围设备。

3. 直接存储器访问（DMA）方式

主要用来连接高速外围设备。如磁盘存储器，磁带存储器、光盘辅助存储器，行式打印机等。由于速度非常快，必须在外设和主存之间建立直接数据通路，DMA 方式的数据传送过程如下图所示：



DMA 方式的数据传送过程

DMA 方式具有如下特点：

- 主存可以被外设和 CPU 访问，外设访问优先级高；
- 设备访问 IO 直接通过电路访问，速度快；
- DMA 控制器复杂，其数据交换过程全部在硬件控制下完成；
- DMA 开始时需要对控制器进行初始化，结束后需要向 CPU 申请中断做后处理；
- CPU 与外设并行独立工作，外设的工作不影响 CPU 的工作效率。

DMA 输入设备的工作流程为：

- 从设备读一个字节到 DMA 控制器中的数据缓冲寄存器 BD；
- 若一个字已装配满，则将 BD 中的数据送主存数据寄存器；

- 把 DMA 控制器中的主存地址寄存器 BA 中的地址送主存地址寄存器， 并将 BA 中的主存地址增值至下一个字地址；
- 把 DMA 控制器内的数据交换个数计数器减 1；
- 若交换个数为 0，则 DMA 数据传送过程结束，否则回到上面。

DMA输出设备的工作流程如下：

- 从主存读一个字节到 DMA 控制器中的 BD 中；
- 若一个字已装满，把 BD 中的数据逐字符或整个字写到输出介质上；
- 把 DMA 控制器内的数据交换个数计数器 BC 中的内容减 1；
- 若 BC 中的内容为 0，则整个交换过程结束，否则继续交换。

目前使用的 DMA 方式实际上有如下三种：

- **周期窃取方式：** 在每一条指令执行结束时，CPU 测试有没有 DMA 服务申请。若有，CPU 进入一个 DMA 周期，在此周期中借用 CPU 完成 DMA 工作流程。采用周期窃取方式，主存可以不与外设直接相连，而只与 CPU 连接，因为外设与主存的数据交换程序控制输入输出方式和中断输入输出方式一样都是要经过 CPU 的。
- **直接存取方式：** 是一种真正的 DMA 方式，DMA 控制器的数据传送申请不是发向 CPU，而是得到主存响应后，整个 DMA 工作流程全部在 DMA 控制器中用硬件完成。是目前多数计算机系统采用的方式。
- **数据块传送方式：**在设备控制器中设置一个比较大的数据缓冲存储器，设备控制器与主存储器之间的数据交换以数据块为单位，并采用程序中断方式进行。数据块传送的方式实际上不是 DMA 方式，只是它在每次中断输入输出过程中是以数据块为单位获得和发送数据的，这一点与上述两种方式相同，因此也归为 DMA。

2.中断源分类和优先级

1. 中断源种类

IBM 将中断源分为 6 类：

- **重新启动中断：**操作人员重新启动应用程序引起的；
- **机器检验出错中断：**发送软硬件故障时产生，例如电源故障、运算器误操作、主存校验出错等；
- **程序性错误引起的中断：**包括指令或数据格式错误、非法操作码错误、地址越界、运算溢出、除零操作、权限校验失败等错误；
- **访问管理程序中断：**当用户程序要调用管理程序时，执行访管指令引起的中断；
- **外部事件中断：**事件来自机器外部，例如定时器中断、分布式系统其它机器引起的中断；
- **输入输出中断：**用于处理机管理各种外围设备。

可屏蔽中断：可以通过软件把它屏蔽掉；

不可屏蔽中断：不能用软件屏蔽它，一旦申请中断，处理机必定会响应。

2. 中断优先级

安排中断优先顺序主要由下列因素来决定：

- 中断源的急迫性；
- 设备的工作速度；
- 数据恢复的难易程度；

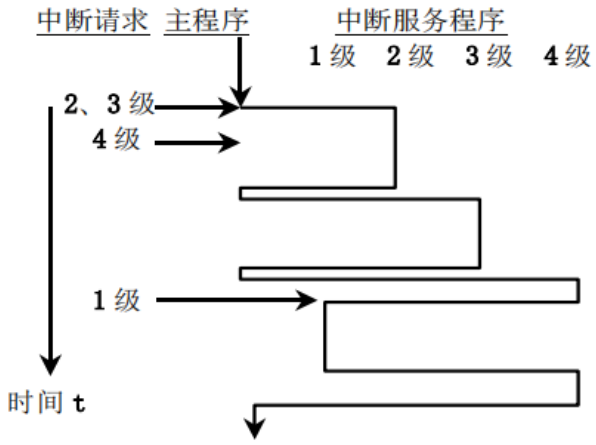
- 要求处理机提供的服务量。

中断优先级与中断服务顺序：

- 要求：响应速度快，灵活性好；
- 做法：由硬件排队器决定中断优先级，通过软件设置中断屏蔽码改变中断服务顺序。

例：某处理机共有4个中断源，中断优先级从高到低分别是：1级、2级、3级和4级。当处理机在执行主程序时，同时有3级和2级两个中断源向处理机发出中断服务请求。当处理机为2级中断源服务时又有4级中断源发出中断服务请求。当处理机为 4 级中断源服务时又有 1 级中断源发出中断服务请求。

解：处理机响应各中断源的中断请求和执行中断服务程序的过程如下：



按照中断优先级响应中断请求的例子

3. 中断处理过程及其软硬件分配

中断处理过程：

中断处理开始 ---> 现行指令结束，关 CPU 中断（CPU 不再响应其它中断） ---> 保存断点（保存 PC） ---> 送中断向量（定位中断服务程序）--->

进入中断服务程序入口 ---> 开 CPU 中断（允许中断嵌套） ---> 执行中断服务程序 ---> 关 CPU 中断（CPU 不再响应其它中断） --->

恢复软硬件现场 ---> 开 CPU 中断 ---> 返回断点 ---> 处理结束。

必须用硬件实现的有：保存断点和进入中断服务程序入口。这两个功能相当于执行一条转子程序指令，因为中断发生在现行程序的什么地方是不确定的，不能由程序员来安排。

必须用软件实现的有：中断服务和返回断点。返回断点可以通过执行一条中断返回指令来实现，中断服务必须用软件实现。

4. 中断响应时间和服务顺序

中断响应时间：

定义：从中断源向处理机发出中断服务请求开始，到处理机开始执行这个中断源的中断服务程序时为止，这一段时间称为中断响应时间。

影响中断响应时间的因素主要有 4 个：（前 2 个属于处理机设计，后 2 个属于中断系统）：

- 最长指令执行时间，因为有些指令的执行时间很长；
- 处理其它更紧急的任务所用时间，比如 DMA 请求等；
- 从第一次关 CPU 中断到第一次开 CPU 中断所经历的时间，这一段是完成中断的前操作部分，主要由硬件完成；
- 多个中断源同时请求中断服务，通过软件找到中断服务程序入口所用时间。

在中断系统的设计中，主要考虑第三部分的时间。

中断服务顺序：可以使用中断请求寄存器、硬件排队器和中断向量法来识别中断源，中断向量法用途更广。

5. 中断屏蔽

设置中断屏蔽，主要是可以改变中断源的服务顺序，实现方法主要有两种：

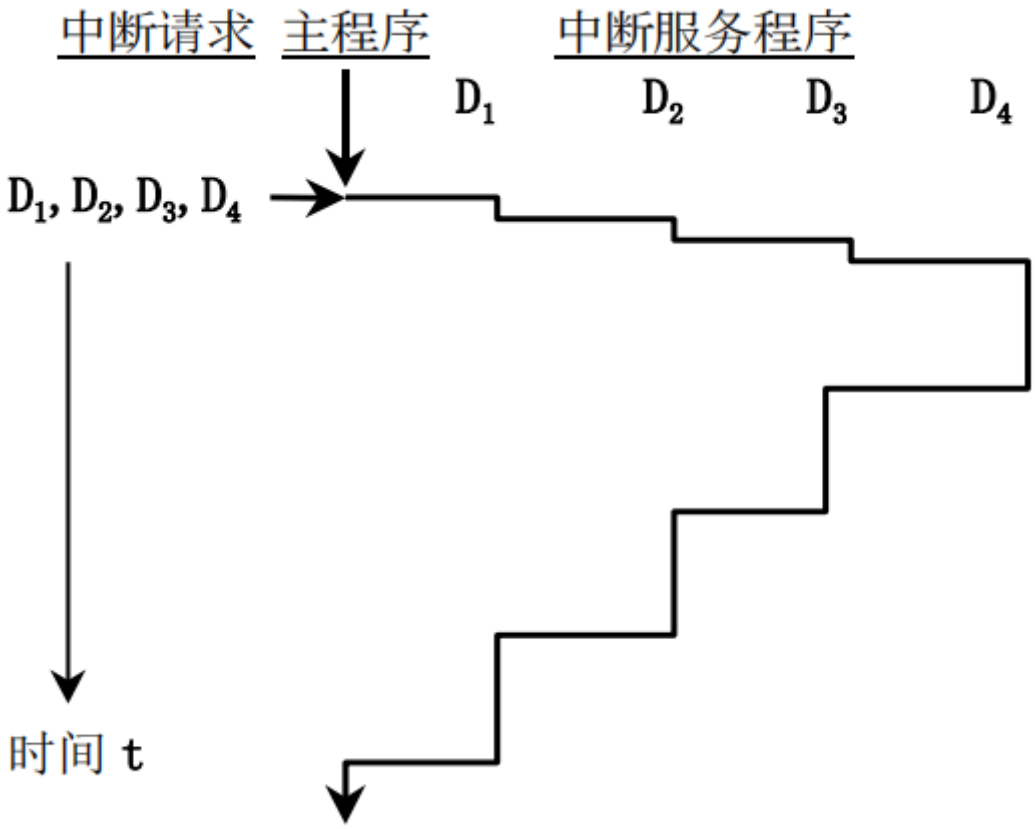
1. 设置中断屏蔽位

例：有四个中断源D_1、D_2、D_3和D_4，它们的中断优先级**从高到低**分别是1级、2级、3级和4级。这些中断源的正常中断屏蔽码和改变后的中断屏蔽码见下表。每个中断源一位，共4位屏蔽码。（1:开放；0：屏蔽）

中断源名称	中断优先级	正常中断屏蔽码 D1 D2 D3 D4	改变后中断屏蔽码 D1 D2 D3 D4
D1	1	0 0 0 0	0 1 1 1
D2	2	1 0 0 0	0 0 1 1
D3	3	1 1 0 0	0 0 0 1
D4	4	1 1 1 0	0 0 0 0

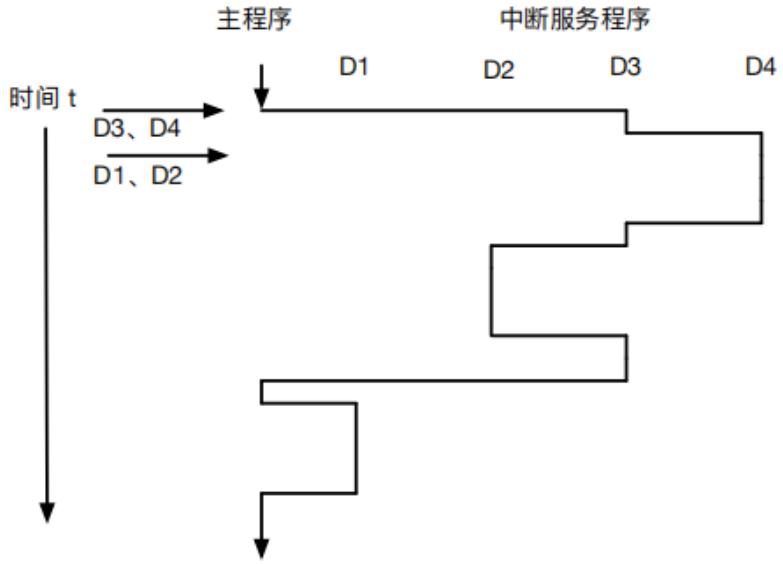
解：

- 如果4个中断源都使用正常的中断屏蔽码，处理机的中断服务顺序将严格按照中断源的中断优先级进行。
- 如果改变中断屏蔽码，当D_1、D_2、D_3和D_4这4个中断源同时请求中断服务时，处理机实际为各个中断源服务的先后次序就会改变。
- 处理机响应的顺序是D_1、D_2、D_3、D_4
- **实际服务的顺序是D_4、D_3、D_2、D_1**



- 若要将实际中断服务先后顺序改为 4-2-3-1，计算新的中断屏蔽码
- 假设从处理机响应中断源的中断服务请求开始，到运行中断服务程序第一次开中断所用的时间为1个时间单位，处理机运行中断服务程序的其他部分所用的时间为3个时间单位。当处理机在执行主程序时，中断源D3和D4同时发出中断服务请求，过两个时间单位后，D1和D2同时发出中断服务请求。采用改变后的中断屏蔽码，画出处理机响应个中断源的中断服务请求和实际运行中断服务程序过程的示意图。

中断源名称	中断优先级	正常中断屏蔽码 D1 D2 D3 D4	改变后中断屏蔽码 D1 D2 D3 D4
D1	1	0 0 0 0	0 1 1 1
D2	2	1 0 0 0	0 0 0 1
D3	3	1 1 0 0	0 1 0 1
D4	4	1 1 1 0	0 0 0 0



2. 改变处理机优先级

中断优先级不仅在处理机响应中断源的中断服务请求时使用，而且为每个中断源的中断服务程序也赋予同样的中断优先级。

例：某处理机共有4个中断源D_1、D_2、D_3和D_4，它们的硬件中断优先级从低到高分别为1级、2级、3级和4级。处理机本身的优先级最低，为0级。在中断源D_1、D_2、D_3、D_4的中断向量中，程序员为它们设置的优先级分别为4级、3级、2级、1级。

解：在处理机状态字中设置3个中断屏蔽位。

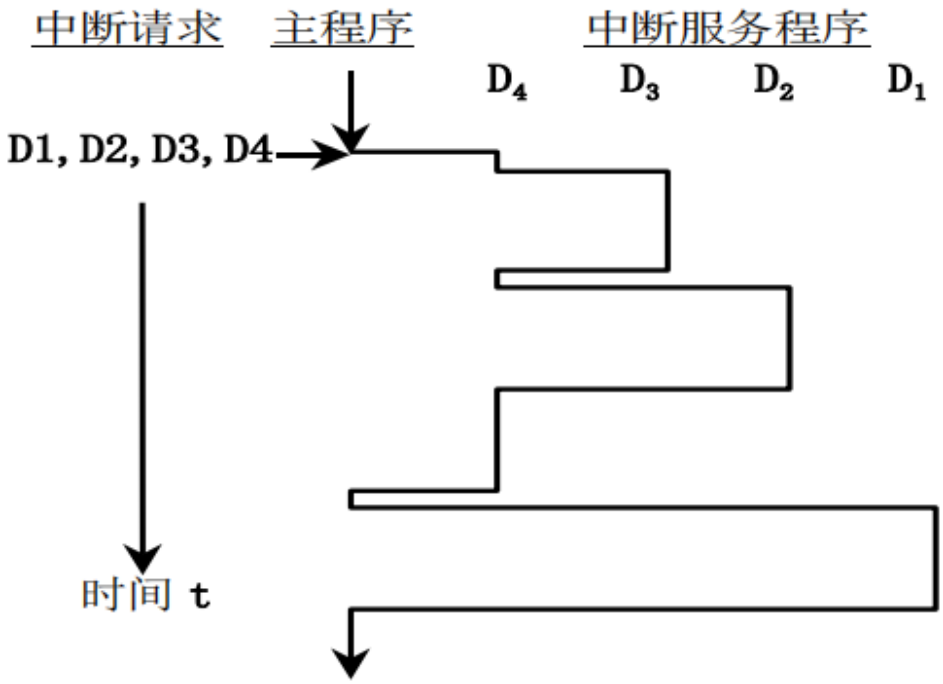
000为处理机本身的优先级，

001～100分别表示4个中断源的中断优先级。

当4个中断源同时请求中断服务时，

解： 处理机实际完成中断服务的过程

是 D_3、D_2、D_4、D_1



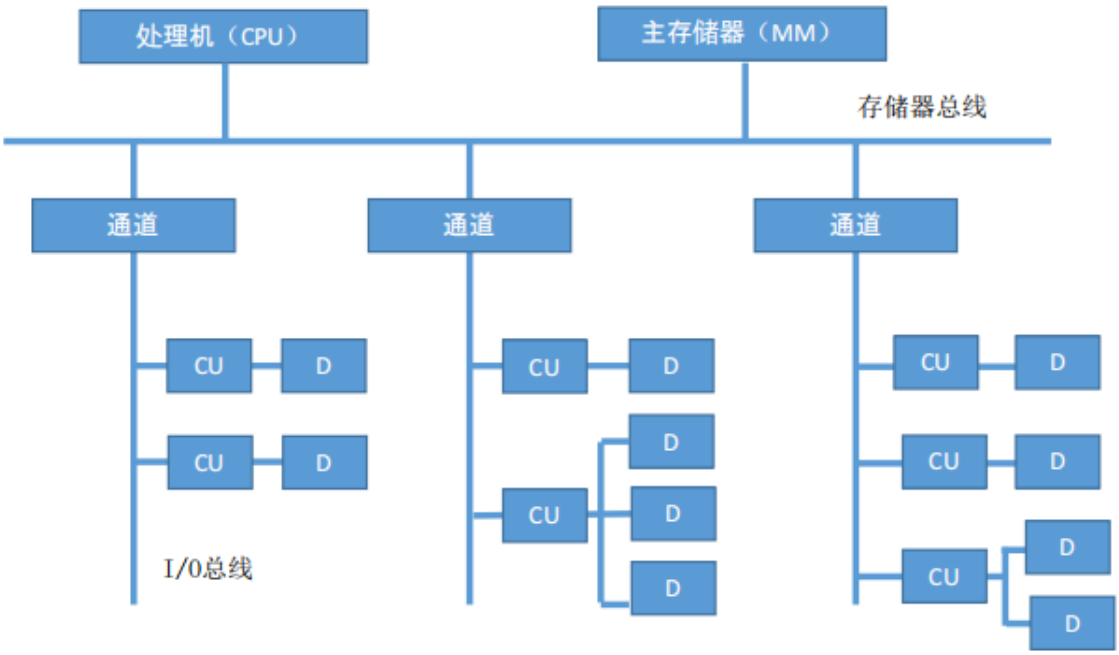
设置中断屏蔽位比改变处理机优先级方便，适合编程，但是使用的位数较多；改变处理机优先级的逻辑比较复杂，只能屏蔽掉比某一个优先级低的中断源，但是所需的位数较少。

6. 通道处理机的作用和功能

在大型处理机中，如果仅采用程序控制、中断和 DMA 来控制 IO 设备，会出现如下两个问题：

1. 所有外设的 IO 工作都要由 CPU 承担，即使是 DMA，其初始化也离不开 CPU 用程序完成，总的来说 CPU 负担很重，不能专注于程序计算；
2. 大型计算机的外设多，但一般不同时工作，若为每一台设备都分配一个接口，开销很大，同时存在如何让 DMA 控制器能被多台设备共享的问题。

为了使 CPU 摆脱繁重的输入输出负担和共享 IO 接口，在大型计算机中采用通道处理机是理想的选择。通道处理机能够分担外设的大部分 IO 操作，对 DMA 接口的初始化和设备故障的检测和处理。一台大型机中可以有多个通道，一个通道可以连接多个设备控制器，一个设备控制器又可以管理一台或多台外设。

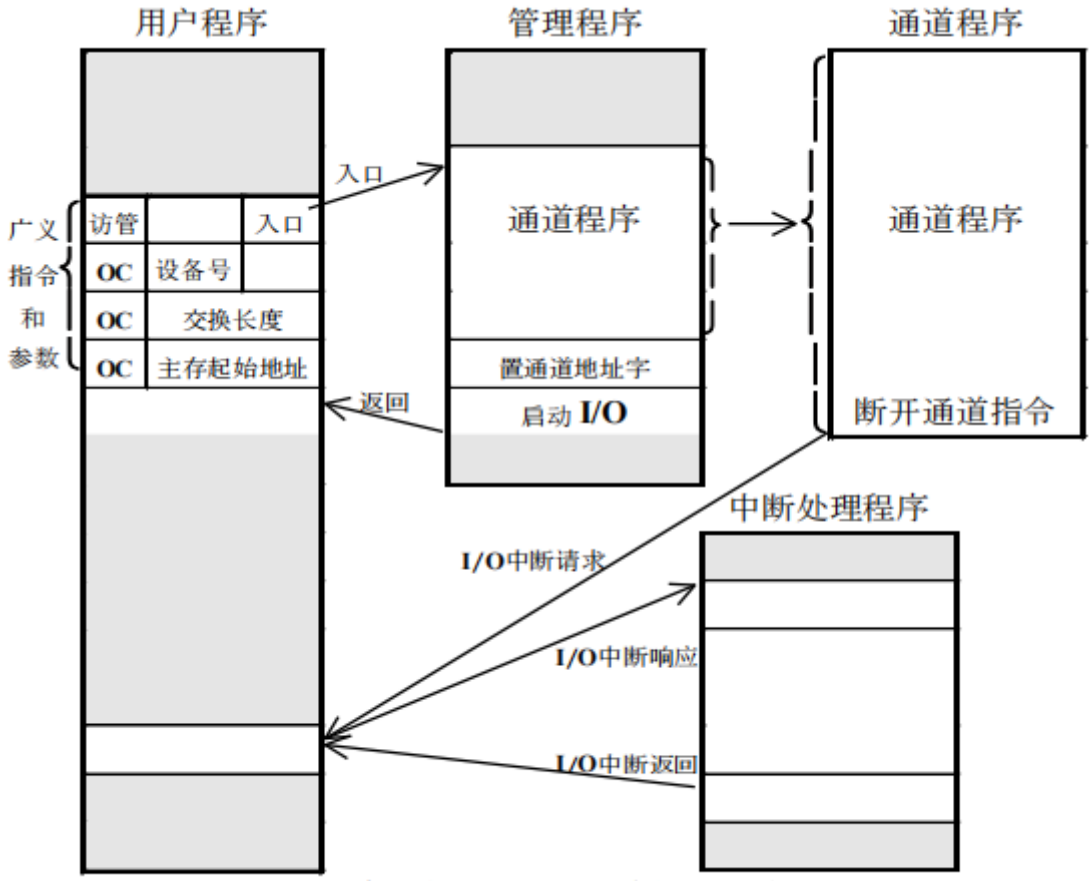


通道的功能主要包括：

1. 接受 CPU 发来的 IO 指令，根据指令选择外设与通道连接；
2. 执行 CPU 为通道组织的通道程序；
3. 给出 IO 操作有关的地址，如磁盘的柱面号、磁头号、扇区号；
4. 给出主存缓冲区的地址，用于和外设进行交互；
5. 控制主存缓冲区与外设交换数据；
6. 指定传送工作结束时的标志和后处理操作；
7. 检查外设工作状态是否正常；
8. 在数据传输过程中完成必要的格式变换。

7. 通道处理机的工作过程

在一般用户程序中，通过调用通道来完成一次数据 IO 的过程如图所示：

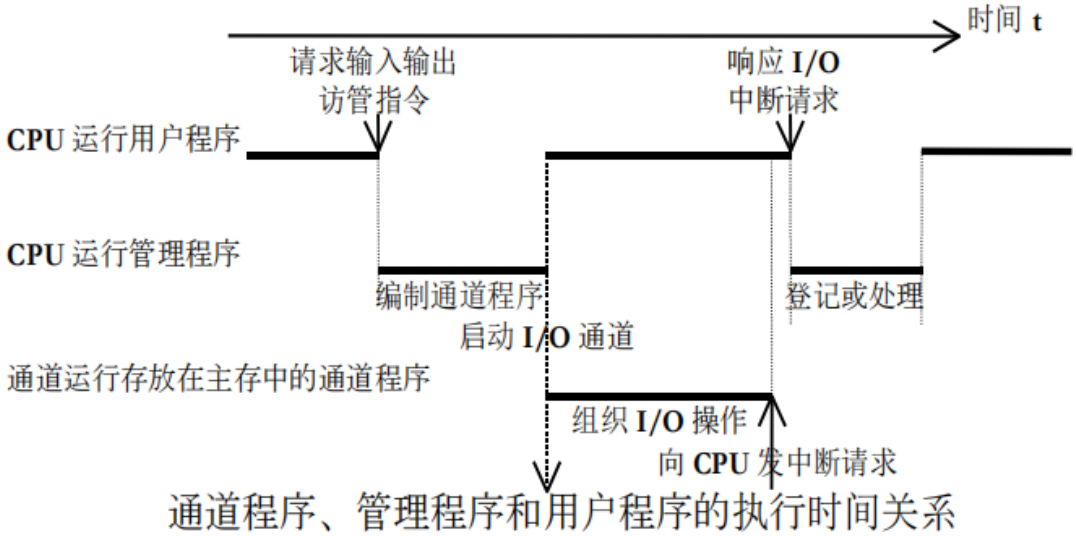


通道完成一次数据传输的主要过程

主要过程分为如下三步进行：

1. 广义指令由一条访管指令和若干参数组成。当用户程序执行到要求进行 IO 操作的访管指令时，产生自愿访管中断请求，CPU 响应并转向管理程序入口。管理程序根据广义指令提供的参数（设备号、主存缓冲区始址等信息）来编制通道程序。通道程序编制好后，放在主存中特定的缓冲区中，用一条启动 IO 设备的指令来启动通道开始工作。

   CPU 执行用户程序，用户程序调用管理程序及通道处理机执行通道程序的时序图如下：



通道程序、管理程序和用户程序的执行时间关系

2. 通道处理机执行 CPU 为它编排的通道程序，完成指定的 IO 操作，此时通道与 CPU 是并行执行的。当通道处理机执行完通道程序的最后一条指令 “断开通道指令” 时，通道的 IO 工作就全部结束了。
3. 通道程序向 CPU 发送中断请求，CPU 响应这个请求，调用相应的中断处理程序进行处理，做必要的登记工作或者故障处理工作。

这样，每完成一次 IO，CPU 只需要两次调用，大大减少了对用户程序的干扰，系统并行性高。

8. 通道种类

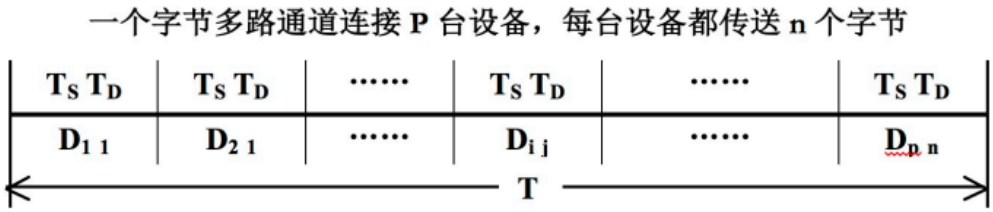
1. 字节多通道

这是一种简单的共享通道，主要为多台低速或中速外设服务，它依赖与 CPU 之间的高速数据通路分时为多台设备服务，采用分时工作的方式。通道总流量等于各外设数据传输速率之和。

1. 字节交叉方式：连接在通道上的各个设备轮流占用一个很短的时间片传输一个字节；
2. 成组方式：允许一个设备一次性占用通道较长时间来传输一组数据。

有多个子通道，每个子通道连接一个公共的控制器，子通道拥有独立寄存器。

字节多通道的数据传送过程：



T_S：设备选择时间　　n：每一个设备传送的的字节个数

T_D：传送一个字节所用的时间　　D_{ij}：连接在通道上的第i台设备传送的第j个数据

P：在一个通道上连接的设备的台数　　T：通道完成全部数据传送工作所需要的时间

$$\text{所需要的总时间：} T_{BYTE} = (T_S + T_D) \cdot P \cdot n$$

字节多通道流量分析：

- **通道流量：**单位时间内能够传送的最大数据量。又称通道吞吐率，通道数据传输率；
- **通道最大流量：**通道在满负荷工作状态下的流量；

假设有 p 台外设，每台外设都传送 n 个字节，通道流量与连接在通道上的设备的数据传输率的关系如下：

$$f = \sum_{i=1}^{p} f_i$$

通道的最大流量是时间片数量与总时间之比，例如 10 个时间片工作了 50s，那么其最大流量就是 0.2/s，表示每秒占用的时间片长度。

$$f_{MAX} = \frac{p \cdot n}{(T_S + T_D) \cdot p \cdot n} = \frac{1}{T_S + T_D}$$

为保证通道不丢失数据，通道的实际流量应不大于通道最大流量：

$$f \le f_{MAX}$$

例题 1：

例：一个字节多路通道连接D_1、D_2、D_3、D_4、D_5共5台设备，这些设备分别每10us、30us、30us、50us和75us发出一次数据传送请求。

1) 计算这个通道的实际流量和工作周期。

2) 如果这个字节多路通道的**最大流量正好等于通道实际流量**，并假设数据传输率高的设备，其优先级也高。5台设备在0时刻同时向通道发出第一次传送数据的请求，并在以后的时间里按照各自的数据传输率连续工作。画出通道分时为各台设备服务的时间图，并计算处理完各设备的第一次请求的时刻。

3) 从时间图中发现什么问题？如何解决？

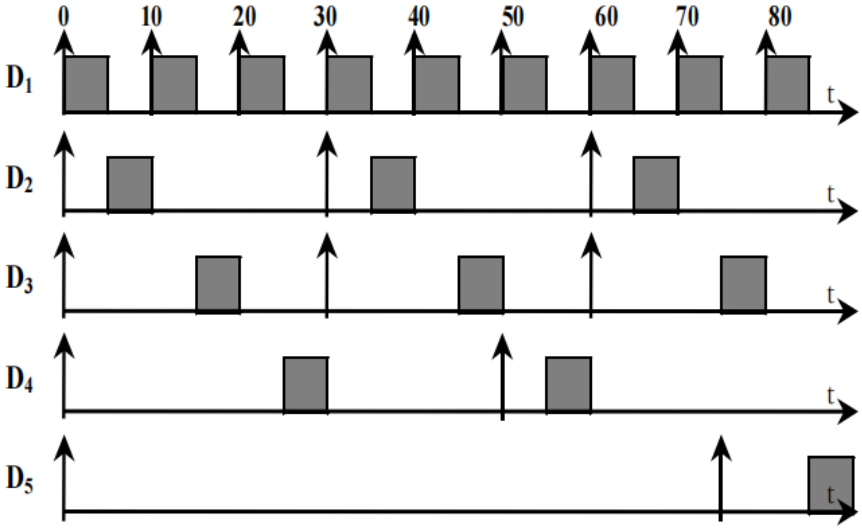
（1）该字节多通道的实际流量为：

$$f = \frac{1}{10} + \frac{1}{30} + \frac{1}{30} + \frac{1}{50} + \frac{1}{75} = 0.2MB/s$$

通道的工作周期为：

$$t = \frac{1}{f} = 5\mu s = T_S + T_D$$

（2）时间图如下：



处理完各设备第一次请求的时间分别为 5μs、10μs、20μs、30μs。

（3）设备 D5 的第一次请求没有得到通道的响应，直到第 85μs 通道才开始响应设备的服务请求，这时，设备已经发出了两个传送数据的服务请求，因此第一次传送的数据有可能丢失。解决方案：

- **增加通道的最大工作流量：**例如，把通道的工作流量增加到 0.25MB/s（工作周期为4μs）；
- **动态改变设备的优先级：**例如，在 30μs 至 70μs 之间临时提高设备 D5 的优先级；
- **增加缓冲存储器。**例如，只要为设备 D5 增加一个数据缓冲寄存器，它的第一次请求可以在第 85μs 处得到响应，第二次请求可以在第 145μs 处得到响应。

例题 2：

2. 一个字节多路通道连接有 5 台设备，它们的数据传输率如下：

设备名称	D_1	D_2	D_3	D_4	D_5
数据传输速率（KB/s）	100	33.3	33.3	20	10
服务优先级	1（最高）	2	3	4	5

(1) 计算这个字节多路通道的实际工作流量

(2) 为了使通道能够正常的工作，请设计通道的最大流量和工作周期

(3) 当这个字节多路通道工作在最大流量时，5 台设备都在 0 时刻同时向通道发出第一次传送数据请求，并在以后的时间里按照自己的数据传输速率连续工作。画出通道分时为各台设备服务的时间关系图，并计算这个字节多路通道处理完各台设备的第一次数据服务请求的时刻。

(1) 字节多通道的实际工作流量为：

$$fBytes = 100 + 33.3 + 33.3 + 20 + 10 = 196.6KB/s$$

(2) 令通道的最大流量等于通道的实际流量 196.6KB/s，则工作周期为：

$$T = \frac{1}{196.6} \approx 5us$$

(3) 各台设备的服务请求间隔时间为：

$$dt1 = \frac{1}{100KB/s} = 10us$$

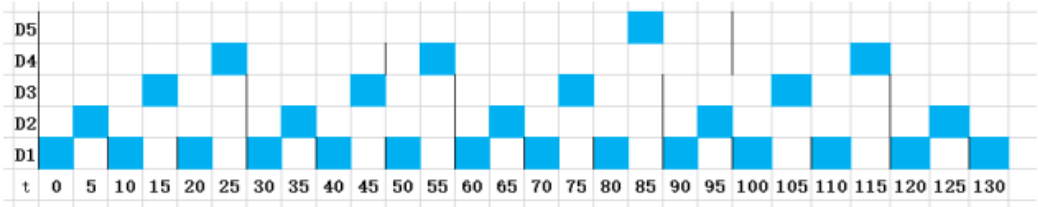
$$dt2 = \frac{1}{33.3KB/s} = 30us$$

$$dt3 = \frac{1}{33.3KB/s} = 30us$$

$$dt4 = \frac{1}{20KB/s} = 50us$$

$$dt5 = \frac{1}{10KB/s} = 100us$$

则出通道分时为各台设备服务的时间关系图如下：



其中，表格有左边框线代表中断请求到来，根据每个时刻中断队列中的元素和优先级来调度时间片分给哪个中断服务程序。

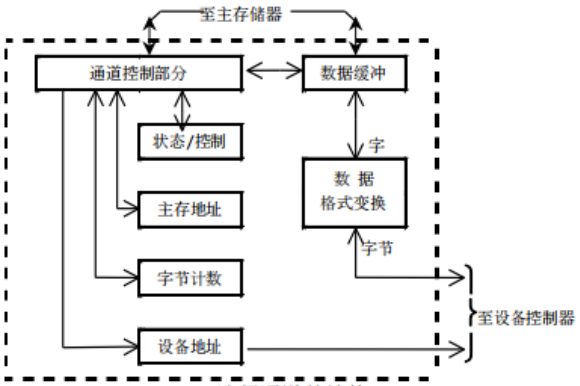
这 5 个设备第一次处理完数据服务请求的时刻为：

$$5us、10us、20us、30us、90us$$

2. 选择通道

2. 选择通道（Selector Channel）

- 为高速外围设备服务
- 一旦选中某一设备，通道就进入“忙”状态，直到该设备的数据传输工作全部完成为止
- 只有一个以成组方式工作的子通道



选择通道的结构

3. 数组多通道

3. 数组多路通道（Block Multiplexor Channel）

- 字节多路通道和选择通道的结合。
- 每次为一台高速设备传送一个数据块，并轮流为多台外围设备服务。
  - 从磁盘存储器读出文件的的过程分为三步：定位、找扇区、读出数据
  - 数组多路通道的实际工作方式是：在为一台高速设备传送数据的同时，有多台高速设备可以在定位或者在找扇区。
- 与选择通道相比，数组多路通道的数据传输率和通道的硬件利用都很高
- 一次输入输出过程中要多次与同一台高速外围设备连接和断开，控制硬件的复杂度也高。

第五章 标量处理机

1. 指令的重叠执行方式

指令的三个阶段:

- **取指令**：按照指令计数器的内容访问主存储器，取出一条指令送到指令寄存器；
- **指令分析**：对指令的操作码进行译码，按照给定的寻址方式和地址字段中的内容形成操作数地址，并用这个地址读取操作数；

- **指令执行**：根据操作码要求，完成规定的功能，将运算结果写到寄存器或主存储器。

指令的二次重叠方式：



在理想情况下，指令的执行时间为：

$$T = (2 + n) \cdot t$$

然而，为了能够实现这种理想的指令重叠执行方式，处理记得结构要做比较大的改变，必须采用先行控制方式。

2. 先行控制技术的基本结构

1. 处理机概念

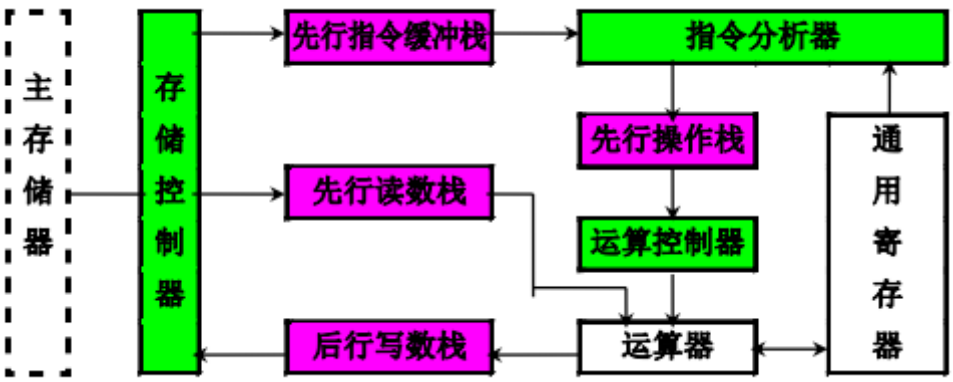
要采用指令的重叠执行方式，必须解决如下 **两个问题**：

1. 取值、分析和执行要能够并行，必须要有独立的功能部件。**解决方案：**把一个集中的指令控制器，分解成多个独立的控制器：存储控制器、指令控制器、运算控制器。
2. 主存访问冲突问题。**解决方案：**①将主存分为指令存储器和数据存储器，独立编址，运算结果写通用寄存器而不访问主存，同时 Cache 也分为 ICache 和 DCache；②主存不分区，但是采用低位交叉存取方式，减小冲突概率；③解决访存冲突的根本方法是 **采用先行控制技术。**

先行控制技术的关键是 **缓冲技术** 和 **预处理技术。**

- 缓冲技术是在工作速度不固定的两个功能部件之间设置缓冲站，用以平滑它们的工作，一般需要四个缓冲栈；
- 预处理技术是把进入运算器的指令都处理成寄存器-寄存器（RR）型指令，与缓冲技术结合，为进入运算器的指令准备好所需要的全部操作数。

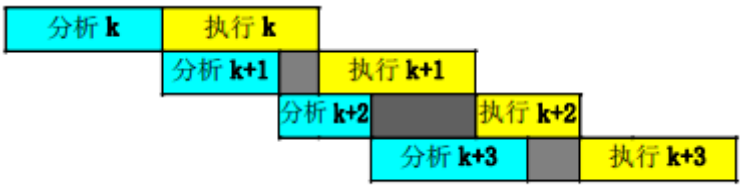
2. 处理机结构



- 三个独立的控制器：存储控制器、指令控制器、运算控制器；
- 四个缓冲栈：先行指令缓冲栈、先行读数缓冲栈、先行操作栈、后行写数栈；

- **先行指令缓冲栈：**用于平滑取指和指令分析之间的速度差异，当指令分析器分析某一条指令的时间较长时，指缓可以预取指令，或者主存忙时，指令分析器可以在指缓中取指令；
- **先行操作栈：**指令分析器把简单的指令直接执行，把复杂的指令处理成 RR 型指令。分析后的指令送往先行操作栈，运算控制器从先行操作栈中取指令并送往运算器进行运算。因此，它是指令分析器和运算器之间的缓冲，使得二者能够完全独立工作；
- **先行读数栈：**其控制逻辑与指令分析器相联，每当指令分析器送来有效地址时，它就开始向主存预取操作数，运算器从先行读数栈中取操作数，可以平滑运算器和主存的速度差异；
- **后行写数栈：**其控制逻辑与运算器相联，每当接收到运算器的结果时，就将其预写到后行写数栈中，由它来访问主存进行写操作，大大提高了指令运算速度。

设置了指令缓冲栈，取指令的时间就可以忽略不计。一条指令的执行可分为分析和执行两阶段。分析指令和执行指令时间一般不相等，然而由于采用了先行控制结构，可以保证指令执行和分析部件各自一直处于忙碌状态：



执行部件时间更长，且是一直忙碌的，理想情况下，连续执行 n 条指令的时间为：

$$T = t_{分析} + t_{执行} \approx t_{执行}$$

3. 数据相关

在采用先行控制的处理机中，存在数据相关和控制相关。

1. 指令相关

第 k + 1 条指令本身的内容取决于第 k 条指令的执行结果。例如，前指令修改了后指令或直接生成了后指令。在先行控制机中，当正在执行第 k 条指令时，可能已经有很多条指令完成了预处理，其结果可能是错误的。

要判断是否发生了相关，需要把每一条指令的结果地址与先行操作栈、指令分析器、指令缓冲栈中的所有指令地址进行比较，如果发现相关，需要在修改主存相关单元的同时，也要修改发生相关的所有指令。

解决指令相关的根本办法是，在程序执行过程中不允许修改指令。

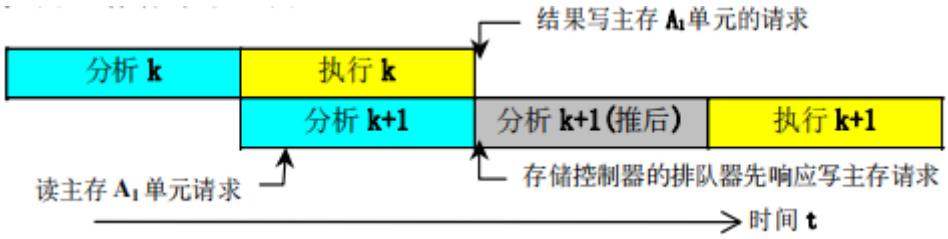
2. 主存操作数相关

当指令的执行结果写到主存，所读取的操作数也取自主存时，就有可能发生主存操作数相关。例如：

```
k:    OP A1, A2, A3   ; A1 = (A2) OP (A3)
k+1:  OP A4, A1, A5   ; A4 = (A1) OP (A5)
```

后指令读的操作数是前指令的执行结果，就可能发生主存操作数相关。在大多数机器中，运算结果一般写到通用寄存器，而不写到主存，因此主存操作数相关的概率比较小。

解决办法一般是推后处理法：



对于刚进入先行操作栈中的指令在读主存操作数时，首先把要访问的地址与后行写数栈中的所有地址比较，若发现相关则把读操作暂缓，等写数完成后再进行读操作。

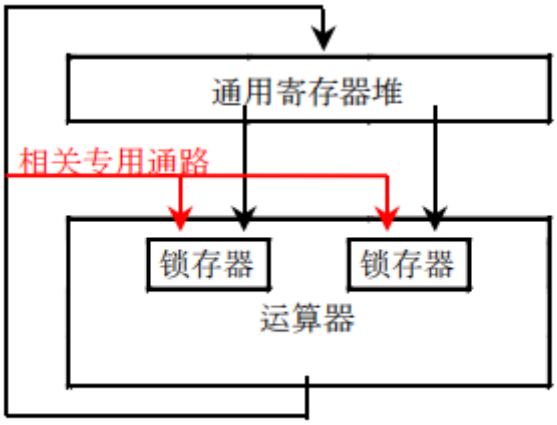
3. 通用寄存器相关

在 RR 和 RS 型指令中都可能发送此种相关：

```
k:    OP R1, A2   ; R1 = (R1) OP (A2)
k+1:  OP R1, R2   ; R1 = (R1) OP (R2)
```

后指令读的寄存器是前指令写的目标，发生的可能性比较大，影响面较广。

解决方法有：①在通用寄存器到运算器之间设置直接通路，在一个节拍内完成读数和写寄存器操作；②分析和执行串行执行，运算速度损失较大；③分析指令推后一个节拍，使写和读刚好岔开一个节拍；④设置专用数据通路，在运算器的输出端到锁存器的输入端建立专用的数据通路，当检测到相关时，把运算结果送入通用寄存器的同时，也送入锁存器的输入端，同时封锁通用寄存器到锁存器的数据流。这种方式增加了硬件代价，但是运算速度高，流水线效率好。



设置专用数据通路解决通用寄存器数据相关

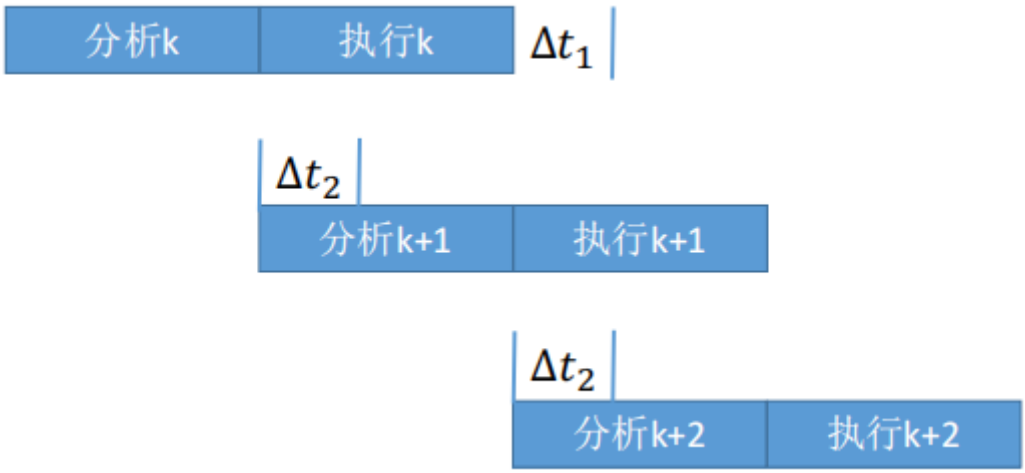
4. 变址相关

由于许多处理机中把通用寄存器当变址寄存器操作，变址寄存器涉及到许多特定的运算顺序，很容易发送相关，因此很有可能发生变址相关。它是变址寄存器相关的专用叫法，发送变址相关的概率比较高，本质上也是通用寄存器相关。

```
k:   OP R1, R2       ; R1 = (R1) OP (R2)
k+1: OP R1, A2(X2)   ; R1 = (R1) OP ((A2)+(X2))
k+2: OP R1, A2(X2)   ; R1 = (R1) OP ((A2)+(X2))
```

k+1 条指令是一次变址相关，发生的原因与通用寄存器数据相关一样；

k+2 条指令是变址相关，这是因为写通用寄存器需要一段稳定时间 Δt_1 才能正常读，分析阶段需要一段很短的译码时间 Δt_2 ，如果满足 $\Delta t_1 > \Delta t_2$，则 k+2 条指令的结果就是错误的：

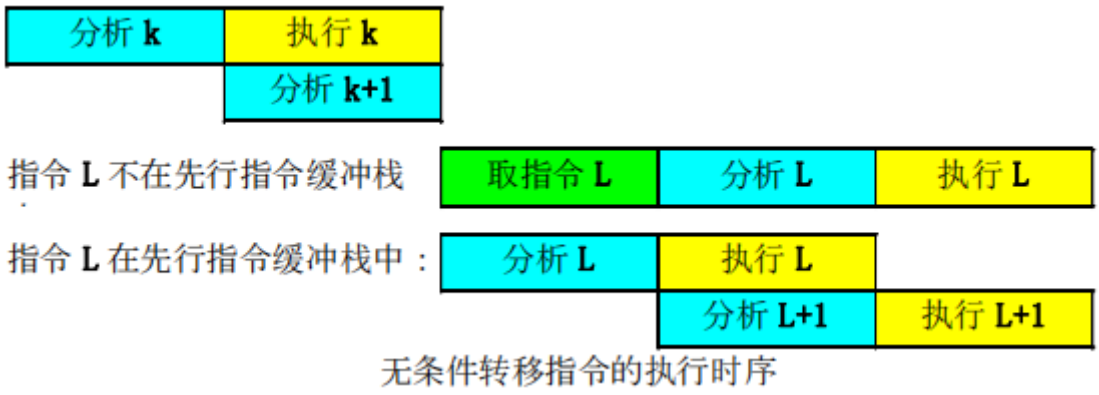


解决变址相关的方法：①推后分析，确保 Δt_2 开始时，Δt_1 已经结束；②设置变址专用通数据路。

4. 控制相关

1. 无条件转移

无条件转移指令可能会引起流水线断流。一般来说该指令在译码阶段就直接完成，如果转移的距离比较远，则指缓中的指令全部作废；如果转移距离比较近，就只作废部分指令。之后，指令分析器接着工作，如果下条指令不在指缓中，则需要先经过一个 “取指令” 周期，反之则继续执行流水线：



无条件转移指令的执行时序

无条件转移指令对程序执行速度的影响很小。

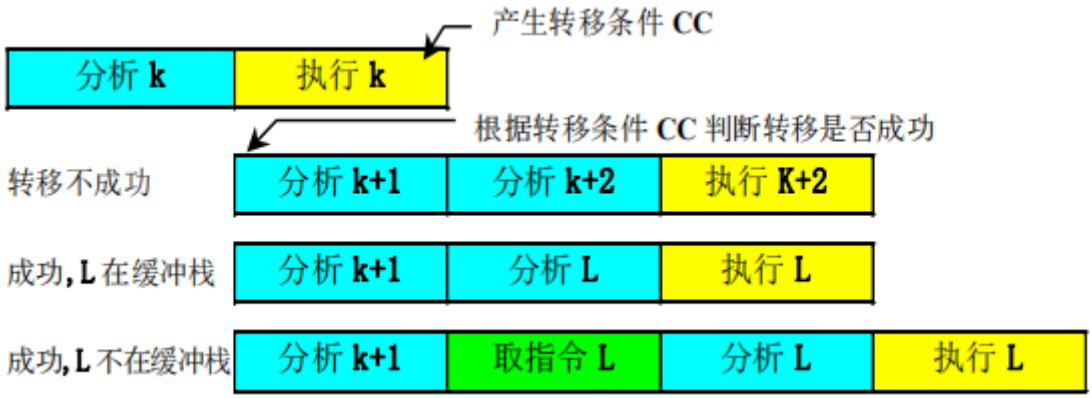
2. 一般条件转移

条件转移指令对程序执行速度影响很大，分为一般条件转移（转移条件来自前指令）和复合条件转移（转移条件来自本指令）。

条件码在 k 指令末尾形成，k+1 推迟一个周期等待条件码。如果转移转移不成功，处理机速度损失很小，因为可以紧接着分析 k+2 和之后的指令，其开销在于，转移指令在分析阶段全部处理，指令分析器要停顿一段时间。；

如果转移成功且距离较近，L 在指缓中，那么只需作废一部分指令，剩余的指令继续流水执行；如果距离较远，L 不在指缓中，那么需要作废全部指令，后续指令经过一个 “取指令” 周期后继续流水线。

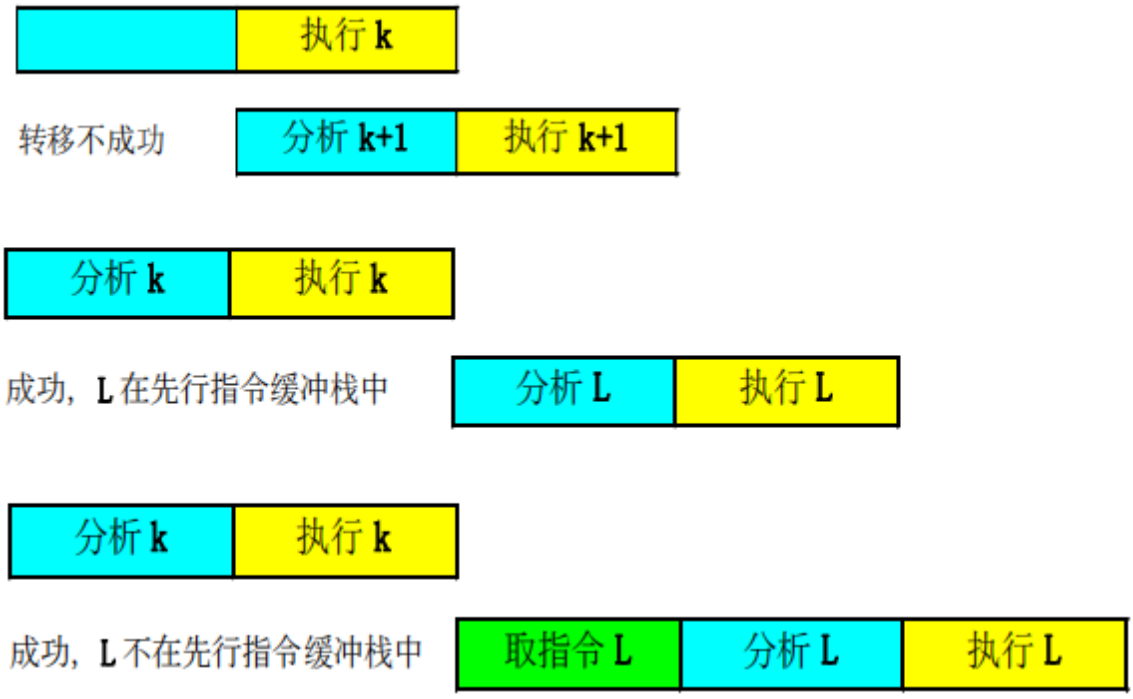
转移成功对机器的影响较大，不仅指令执行过程变成完全串行，而且要作废先行指令缓冲栈中的大量指令。



3. 复合条件转移

转移指令本身就是运算指令，如果转移不成功则不造成任何影响，就象普通的运算型指令一样；如果转移成功：造成的影响比一般条件转移指令还要大得多。

不仅要全部或部分作废先行指令缓冲栈中预取的指令，还要作废先行操作栈中的指令、先行读数栈中的操作数和当前在指令分析器中分析的指令。



一般来说，转移成功的概率很高，不成功的只有一次，通过预测转移成功来进行预处理可以减少流水线的性能损失。

5. 结构相关

结构相关就是多个指令的执行向同一硬件资源提出请求而导致冲突，在线性流水线中，它是在各执行周期不相等的情况下才会产生的。例如，连续两条指令都要进行浮点乘法，但需要耗费很多个时钟周期，后指令译码结束后，浮点乘法运算部件还在忙碌，因此后指令需要等待部件空闲后才可调度。

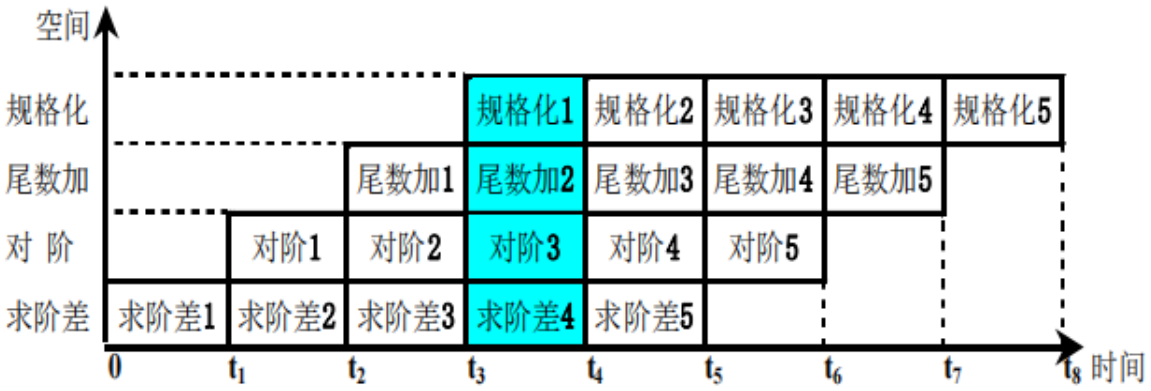
结构相关的解决方案，一是可以在前一个指令执行时，将流水线停顿一个时钟，推迟后面指令的操作，或者添加 nop 指令；二是在流水线处理机中设置相互独立的指令存储器和数据存储器，同时尽量避免同一 寄存器的频繁使用等。

非线性流水线中的结构冲突：由于存在反馈回路，当一个任务在流水线中流过时，在同一功能段中可能会经过多次，因此就可能同一时刻有几个任务争用同一功能段，称为功能部件冲突。

3. 流水线的基本原理（时空图）

流水线方式是把一个重复的过程分解为若干个子过程，每个子过程可以与其他子过程同时进行。处理机各个部分几乎都可以采用流水线方式工作，例如浮点加法一个采用 4 级流水线：求阶差 ---> 对阶 ---> 尾数加 ---> 规格化。

一个浮点加法器流水线时空图：



4. 流水线的分类

1. 线性流水线和非线性流水线

线性流水线：将流水线的各段逐个串接起来，输入数据从一端进入，从另一端输出，数据仅流过各个功能段一次。一条线性流水线通常只完成一种固定的功能。

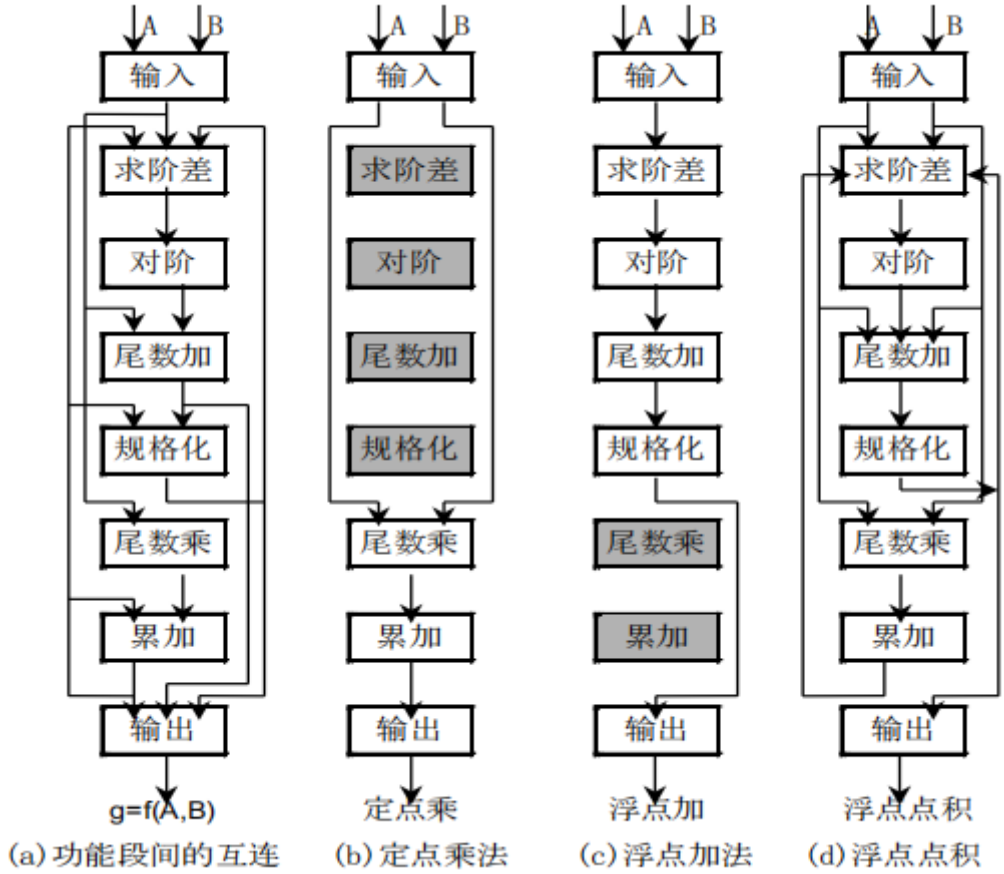
非线性流水线：某些流水段之间有反馈回路或前馈回路。非线性流水线经常用于递归调用，使用 “预约表” 可以方便地描述它的工作情况，在预约表中可以表示使用反馈回路的次数。

一条非线性流水线可以对应很多张预约表，分表表示了不同的工作方式，非线性流水线调度的重要问题就是确定在什么时间可以向流水线输入新的任务，使输入任务与流水线中原有的任务之间不产生冲突。

2. 单功能和多功能流水线

单功能流水线：一条流水线只完成一种固定的功能，例如浮点加法流水线只完成浮点加法运算。

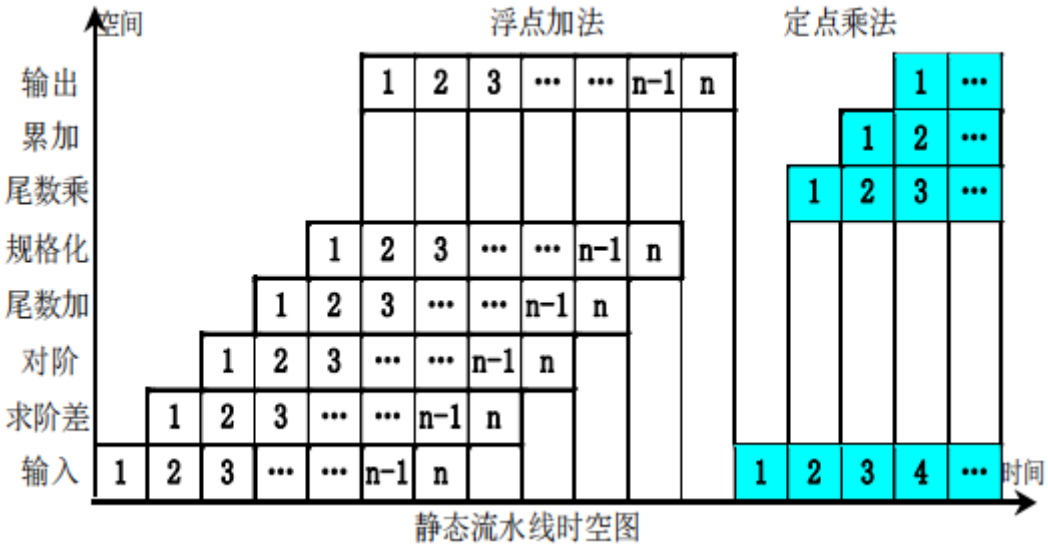
多功能流水线：流水线的各段可以进行不同的连接，多条单功能流水线可以组成多功能流水线，例如求点积的流水线可以连接乘法流水线和加法流水线。



(a) 功能段间的互连　(b) 定点乘法　(c) 浮点加法　(d) 浮点点积

3. 静态和动态流水线

静态流水线：在同一段时间内，多功能流水线中的各个功能段只能按照一种固定的方式连接，实现一种固定的功能，只有流水线排空之后，多功能流水线才能重新连接。



动态流水线：多功能流水线中的各段可以按照不同的方式连接，同时实现不同的功能，允许不同的操作出现在流水线时空图中的同一时间点上。



目前，大多数处理机中仍然采用静态流水线，因为动态流水线的控制和开销要复杂得多。

5. 线性流水线的性能分析

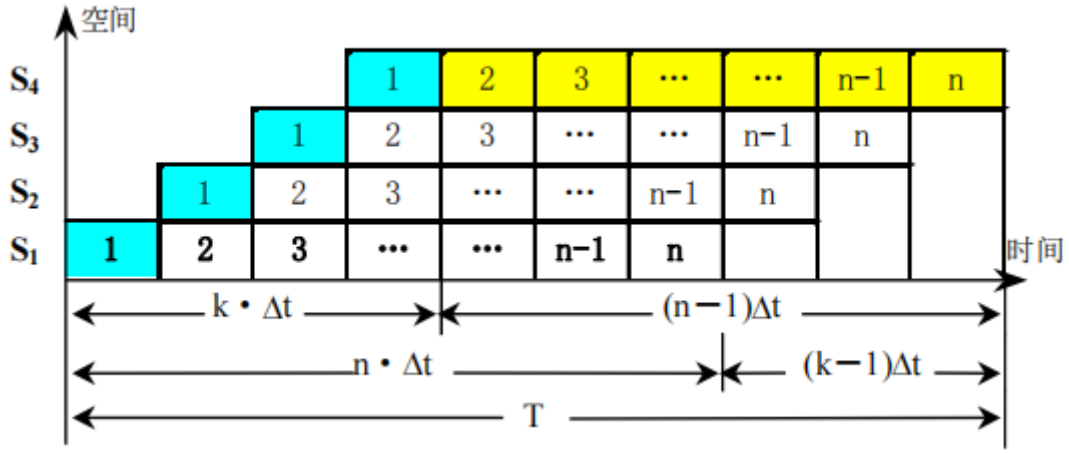
1. 吞吐率

流水线的吞吐率是指在单位时间内流水线所完成的任务数量：

$$TP = \frac{n}{T_k}$$

其中，n 为任务数，T_k 为完成 n 个任务所用的时间。

如下图，一条 k 段流水线能够在 n + k - 1 个时钟周期内完成 n 个任务：

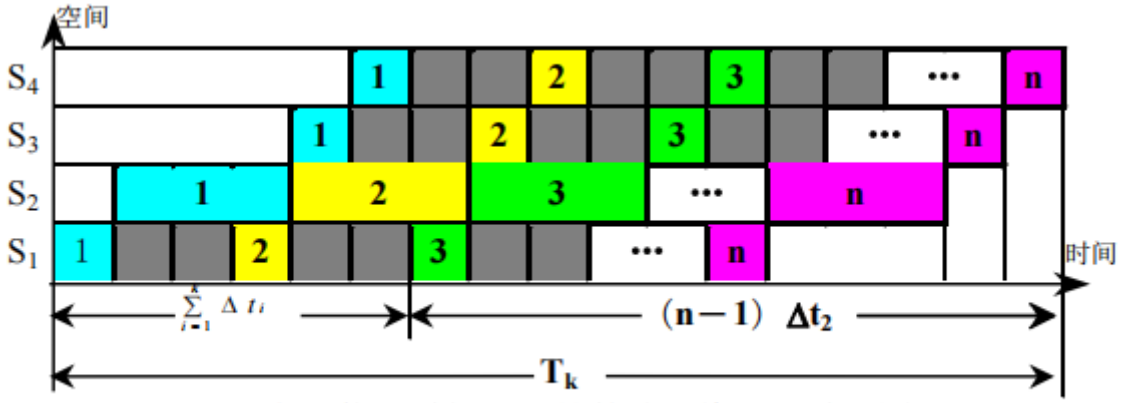


于是，得出一条 k 段线性流水线的吞吐率为：

$$TP = \frac{n}{(n+k-1)\triangle t}$$

$$TP_{MAX} = Lim_{n\to\infty} TP = \frac{1}{\triangle t}$$

由于流水线各执行阶段不完全相等，因此可能会存在瓶颈：



各段执行时间不相等的流水线及其时空图

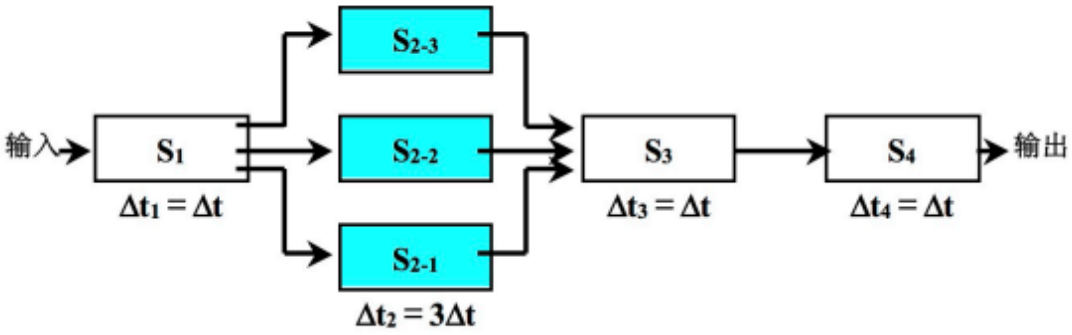
此时的吞吐率计算公式为：

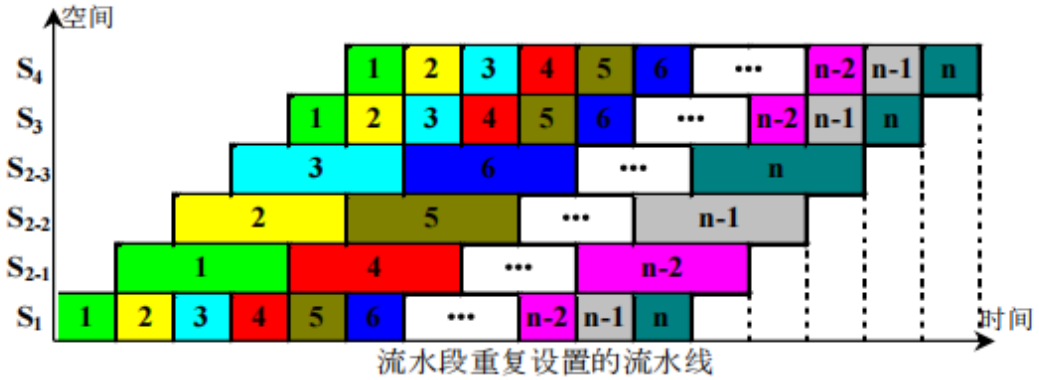
$$TP=\frac{n}{\sum_{i=1}^{k}t_i+(n-1)max(\triangle t_1,\triangle t_2,\ldots,\triangle t_k)}$$

$$TP_{MAX}=Lim_{n\to\infty}TP=\frac{1}{max(\triangle t_1,\triangle t_2,\ldots,\triangle t_k)}$$

可以看出，流水线的最大吞吐率与实际吞吐率主要取决于流水线中执行时间最长的那个功能段，这个功能段就是流水线的瓶颈。解决瓶颈的方法有两种：

1. 将瓶颈细分，增加流水线段数；
2. 重复设置瓶颈段，让多个瓶颈功能段并行工作，流水线的吞吐率仍然可以达到最优；





流水段重复设置的流水线

2. 加速比

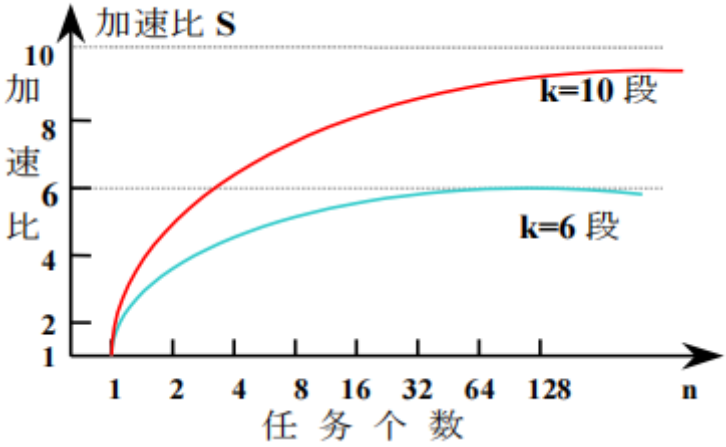
完成一批任务，不使用流水线所用的时间（T_0）与使用流水线所用的时间（T_k）之比称为流水线的加速比（S）：

$$S = \frac{T_0}{T_k} = \frac{n \cdot k \cdot \triangle t}{(n + k - 1) \triangle t} = \frac{n \cdot k}{n + k - 1}$$
$$S_{MAX} = lim_{n \to \infty} S = k$$

当流水线各功能段执行时间不相等时，实际加速比为：

$$S = \frac{n \cdot \sum_{i=1}^{k} \triangle t_i}{\sum_{i=1}^{k} t_i + (n - 1) \cdot max(\triangle t_1, \triangle t_2, \ldots, \triangle t_k)}$$

理想情况下，流水线的最大加速比等于流水线段数。然而，当流水线段数增多时，为了充分发挥其能力，要求连续输入的任务 n 也就越多，下图给出连续任务个数 n 与加速比 S 的关系：



当任务数很小时，加速比较差，任务数越多加速比越大。然而，受软硬件限制，可以连续输入的任务数 n 不会很大，流水线的段数 k 也达不到很高，因此根据实际选择 n 和 k 是很重要的。

3. 效率

流水线的效率是指流水线的设备利用率，定义为时空图中工作的面积与总面积之比：

$$E = \frac{k \cdot n \cdot \triangle t}{k \cdot (k + n - 1) \cdot \triangle t} = \frac{n}{k + n - 1}$$
$$E_{MAX} = lim_{n \to \infty} E = 1$$

总结根据上述关系，得出：

$$TP = \frac{n}{(k + n - 1) \triangle t}$$
$$S = \frac{k \cdot n}{k + n - 1}$$
$$E = \frac{n}{k + n - 1}$$

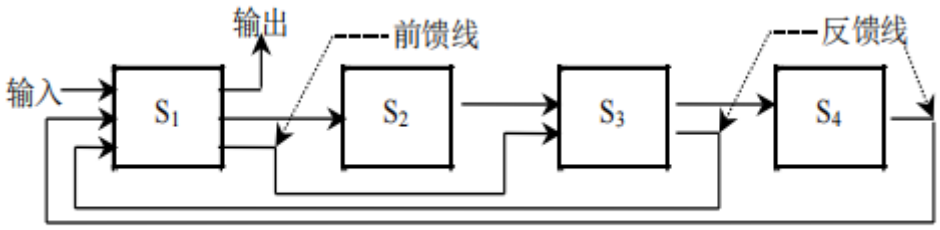
因此：

$$E = TP \cdot \triangle t$$
$$S = k \cdot E$$

6. 非线性流水线的调度

1. 非线性流水线的表示

非线形流水线的连接图和预约表：



功能段 \ 时间	1	2	3	4	5	6	7
S_1	×			×			×
S_2		×			×		
S_3		×				×	
S_4			×				

一张预约表可能与多个流水线连接图相对应，一个流水线连接图也可以对应与多张预约表。

2. 非线性流水线的冲突

非线性流水线中的结构冲突：由于存在反馈回路，当一个任务在流水线中流过时，在同一功能段中可能会经过多次，因此就可能同一时刻有几个任务争用同一功能段，称为功能部件冲突。

非线性流水线调度的任务是要找出一个最小的循环周期，按照这周期向流水线输入新任务，流水线的各个功能段都不会发生冲突，而且流水线的吞 吐率和效率最高。

启动距离为 3 的流水线冲突情况

功能段 \ 时间	1	2	3	4	5	6	7	8	9	10	11	…
S_1	X_1			X_1X_2			$X_1X_2X_3$			$X_2X_3X_4$		…
S_2		X_1			X_1X_2			X_2X_3			X_3X_4	…
S_3		X_1			X_2	X_1		X_3	X_2		X_4	…
S_4			X_1			X_2			X_3			…

启动距离为 2 的流水线冲突情况

功能段 \ 时间	1	2	3	4	5	6	7	8	9	10	11	…
S_1	X_1		X_2	X_1	X_3	X_2	X_1X_4	X_3	X_2X_5	X_4	X_3X_6	…
S_2		X_1		X_2	X_1	X_3	X_2	X_4	X_3	X_5	X_4	…
S_3		X_1		X_2		X_1X_3		X_2X_4		X_3X_5		…
S_4			X_1		X_2		X_3		X_4		X_5	…

3. 无冲突调度方法

启动距离：向一条非线性流水线的输入端连续输入两个任务之间的时间间隔；

禁止启动距离：引起非线性流水线功能段冲突的启动距离；

禁止向量：把所有禁止启动距离组合在一起形成一个数列；

要正确调度一条非线性流水线，首先要找出流水线的禁止启动距离，将它们组合形成禁止向量。

由预约表计算禁止向量的方法：把预约表的每一行任意两个 X 之间的距离计算出来，去掉重复值，得到的就是禁止向量。例如，下图的禁止向量是（3, 4, 6）：

时间 / 功能段	1	2	3	4	5	6	7
S_1	×			×			×
S_2		×			×		
S_3		×				×	
S_4			×				

冲突向量：由禁止向量计算而来，用一个 m 位数的二进制数表示，m 是禁止向量中的最大值。冲突向量用 C 表示：

$$C = (C_m C_{m-1} \dots C_2 C_1)$$

如果 i 在禁止向量中，则 $C_i = 1$，否则为 0。对于上图，禁止向量为 （3, 4, 6），对应的冲突向量为（101100）。

状态图：由冲突向量计算而来，用于找出各种可行的启动循环。构造方法如下：

1. 将冲突向量逻辑右移，若移出去的是 1，则表示该启动距离会导致冲突，不做任何处理；
2. 若移出去 0，则不会导致冲突，把移位器中的值与 **初始冲突向量 S** 按位或运算，得到一个新的冲突向量，假设该冲突向量 C_j 由冲突向量 C_i 右移 k 次得来，则从 C_i 向 C_j 引一条权为 k 的弧（注意是与冲突向量 S 进行或操作，从 C_i 向 C_j 引弧）；
3. 上述操作执行 m 次，对于新构造的冲突向量，也采用同样的办法；
4. 图中每一个状态都向初始冲突向量 S 引一条弧，权值为 m+1。

启动循环：使非线性流水线不发生冲突的启动距离循环数列；

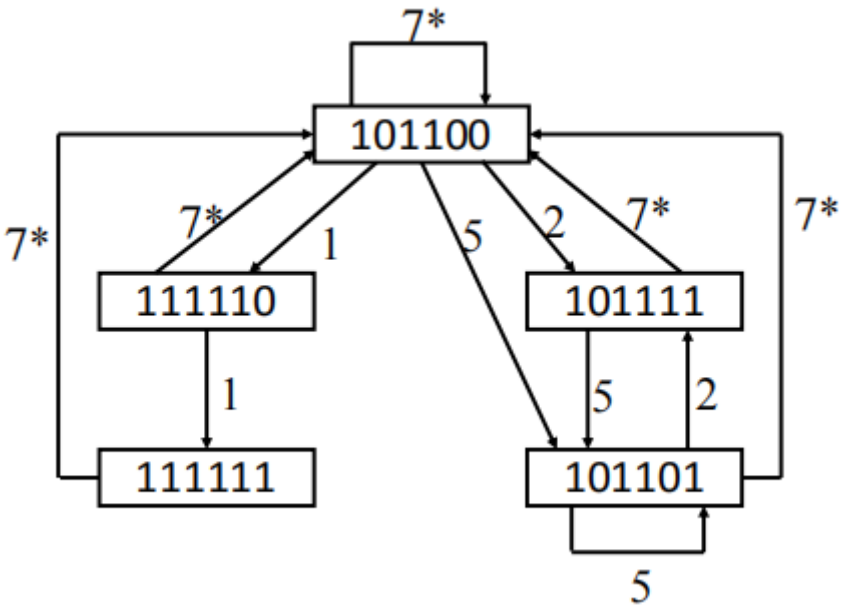
简单循环：在一个周期内状态不会重复的启动循环，例如下图中 (2, 2, 7) 不是简单循环，(2, 7) 是简单循环。

平均启动距离：启动循环的所有距离相加再除以启动距离个数；

最小启动循环：平均启动距离最小的循环；

恒定循环：为了简化流水线控制逻辑，采用相等间隔的调度方法，循环只有一个数。

当初始冲突向量 S 确定后，状态图就是唯一的，从预约表可以得到状态图，但从状态图无法得到预约表。在状态图中，只要能构成循环的序列都可以是启动循环。例如上图对应的状态图如下：



简单循环和平均启动距离是：

简单循环	平均启动距离
(1,7)	4
(1,1,7)	3
(2,7)	4.5
(2,5)	3.5
(2,5,7)	4.7
(5,7)	6
(5)	5
(5,2,7)	4.7
(7)	7

最小启动循环是 (1, 1, 7)，最小恒定启动循环 (5)。以最小启动循环画出上图的预约表为：

	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
S1	X1	X2	X3	X1	X2	X3	X1	X2	X3	X4	X5	X6	X4	X5	X6	X4
S2		X1	X2	X3	X1	X2	X3				X4	X5	X6	X4	X5	X6
S3		X1	X2	X3		X1	X2	X3			X4	X5	X6		X4	X5
S4			X1	X2	X3							X4	X5	X6		

7. 全局相关

由条件转移或程序中断引起的相关称为全局相关。在流水线中，全局相关对流水线的影响比局部相关大得多，而且条件转移指令的占比也相当大。无论是一般条件转移指令还是复合条件转移指令，在其进入流水线到形成转移码之前，后续指令都不能进入流水线，这会对流水线性能造成很大影响。

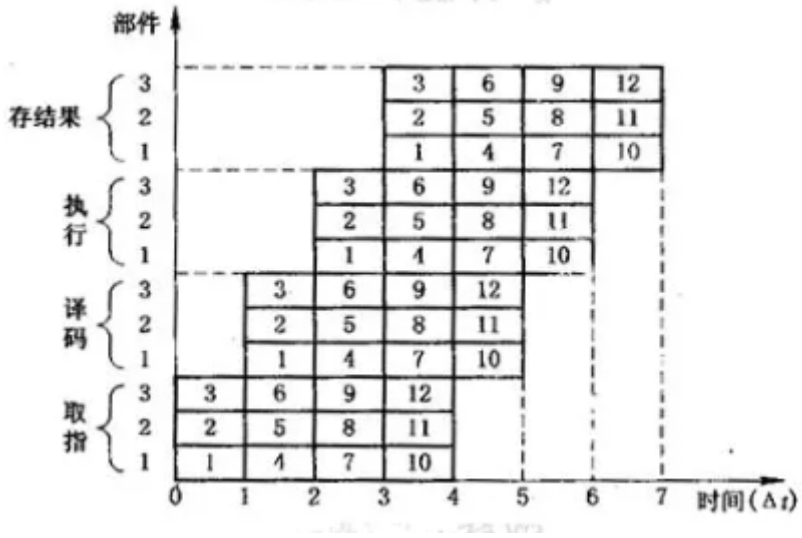
为了减少全局相关带来的影响，可以采取如下几种措施：

1. **延迟转移技术和指令取消技术：**由软件优化，把一条或几条不会冲突的指令调度到转移指令的后面，使流水线不断流，但是这种方法缺点也很明显，成功调度多条不冲突指令的概率很低，而且一般只适用于单流水线标量机，且流水线的级数不能太多；
2. **动态转移预测技术：**根据近期转移是否成功的记录来预测下一次转移的方向，近期成功的比例越大，下一次就预测转移成功，反之预测转移失败；
3. **静态转移预测技术：**在处理机设计完成后，转移预测方向就确定了，或者预测为成功，或者预测为不成功，在程序执行的过程中，转移预测方向不能改变；
4. **提前形成条件码：**在绝大多数情况下，只需在运算部件的入口处设置一个比较器，通过比较两个操作数的符号或者阶码就能提前形成结果的正负号、是否为 0、是否溢出等条件，从而迅速给后指令使用，这样流水线就不会断流。

8. 高级处理机

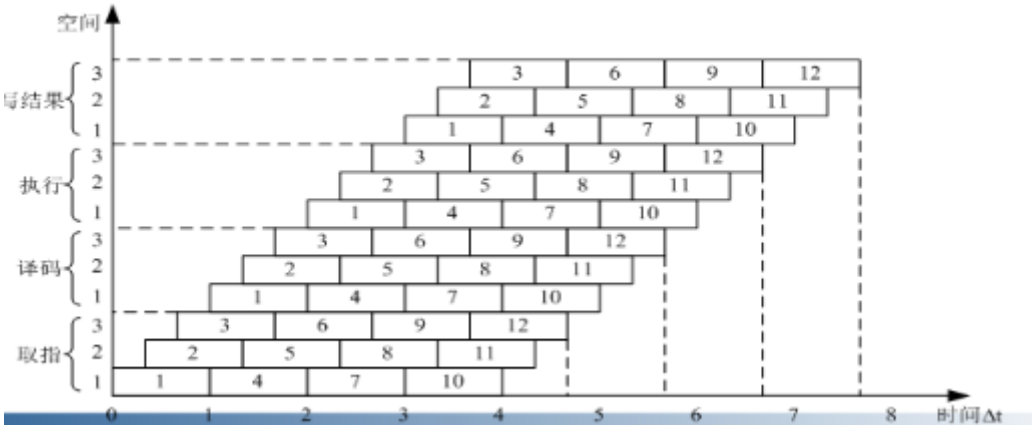
1. 超标量处理机

超标量处理器的基本要求是在一个时钟周期内能够 **同时** 发射两条或两条以上的指令，需要有多条流水线，理论上最佳情况是每个时钟周期发射 3 条指令。



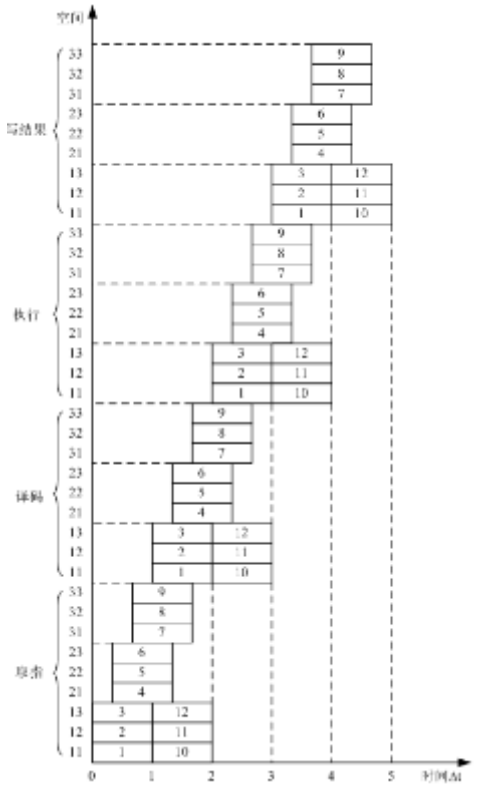
2. 超流水线处理机

超标量处理器的基本要求是在一个时钟周期内能够 **分时** 发射两条或两条以上的指令，需要有多条流水线，理论上最佳情况是每个时钟周期发射 3 条指令。



3. 超标量超流水线处理机

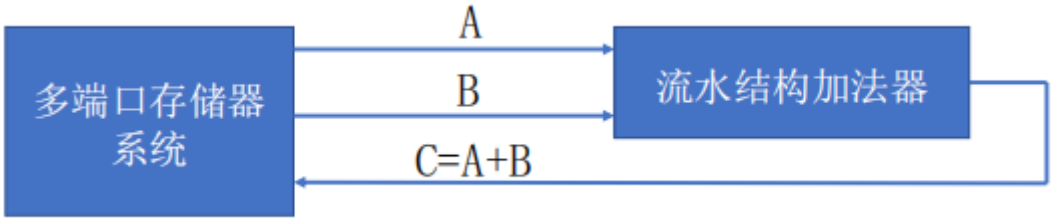
是超标量处理机和超流水线处理机的结合。



第六章 向量处理机

1. 向量的基本结构

向量处理剂的基本思想是采用流水线的工作方式，把两个向量的对应分量进行运算，最终产生一个结果向量。一种采用向量机实现向量加法的模型如下：

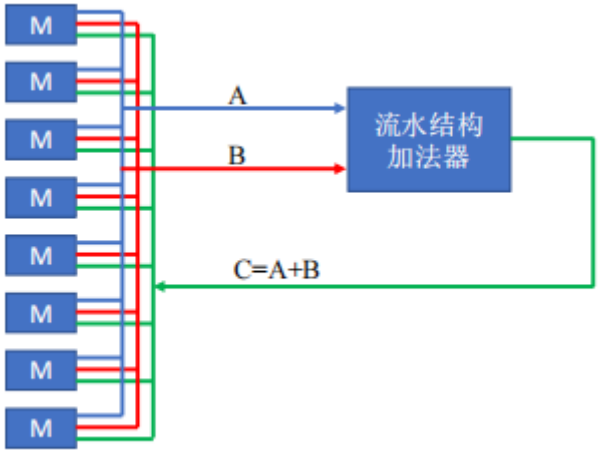


其困难之处在于使存储器系统能够提供给运算器连续不断的数据流，以及接受来自运算器的连续不断的运算结果，对此，向量机在系统结构方面必须设法维持连续稳定的数据流。例如上图要求存储系统能在一个时钟周期内读出两个操作数和写回一个运算结果，其存储器带宽应至少三倍与一般的存储系统。目前的向量机主要采用两种办法：

1. 利用几个独立的存储器模块来支持对相互独立的数据的并发访问，从而提升存储器带宽，即 **存储器-存储器** 结构；
2. 构造一个具有所要求带宽的高速中间存储器，并实现该高速中间存储器与主存储器之间的快速数据交换，即 **寄存器-寄存器** 结构；

1. 存储器-存储器结构

此种结构的向量机采用多个存储体交叉和并行访问方式来提高存储器速度，如下是一个 8体-3端口 存储器：



如下是计算向量 C = A + B 的过程中，数据在 8 个主存模块中的最佳排布方式：

模块0	A[0]		B[6]			C[4]	… …
模块1	A[1]		B[7]			C[5]	… …
模块2	A[2]	B[0]				C[6]	… …
模块3	A[3]	B[1]				C[7]	… …
模块4	A[4]	B[2]			C[0]		… …
模块5	A[5]	B[3]			C[1]		… …
模块6	A[6]	B[4]			C[2]		… …
模块7	A[7]	B[5]			C[3]		… …

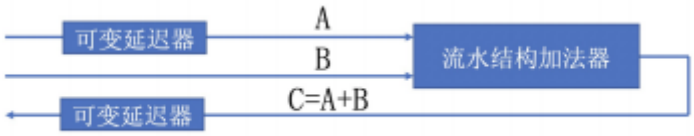
其对应的分量操作流水线和计算流水线如下所示：

	0	1	2	3	4	5	6	7	8	9	10	11	12
流水线段4						0	1	2	3	4	5	6	7
流水线段3					0	1	2	3	4	5	6	7	
流水线段2				0	1	2	3	4	5	6	7		
流水线段1			0	1	2	3	4	5	6	7			
存储器7						RB5	RB5	RA7	RA7	W3	W3		
存储器6					RB4	RB4	RA6	RA6	W2	W2			
存储器5				RB3	RB3	RA5	RA5	W1	W1				
存储器4			RB2	RB2	RA4	RA4	W0	W0					
存储器3		RB1	RB1	RA3	RA3								
存储器2	RB0	RB0	RA2	RA2									W6
存储器1		RA1	RA1				RB7	RB7				W5	W5
存储器0	RA0	RA0				RB6	RB6				W4	W4	

时间（时钟周期）

每两个时钟周期，都能同时取出 A 的分量和 B 对应的分量，通过 4 个周期的计算流水线后存入主存中，主存不会发生访问冲突，效率很高，缺点是数据的排版比较复杂；

为了解决这个缺点，也可以使用在运算流水线的输入端和输出端增加缓冲器以便消除争用存储器现象：



可变延迟器在读 A 分量时延迟两个周期，写 C 时延迟四个周期，此时数据只需规则地从模块 0 开始排布，其流水线如下：

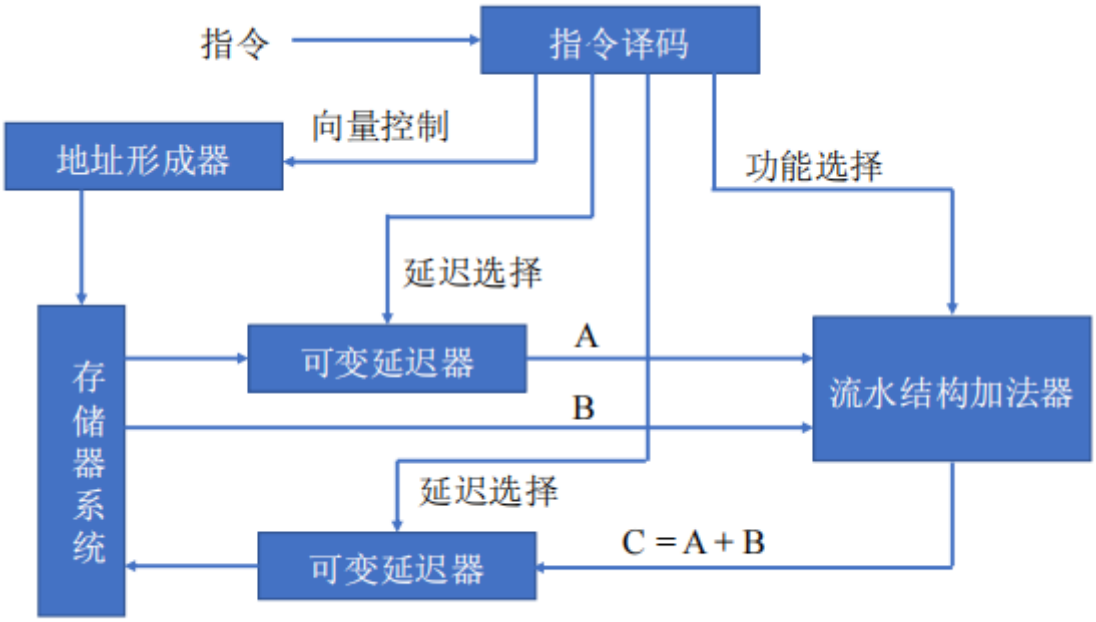
	0	1	2	3	4	5	6	7	8	9	10	11	12
流水线段4								0	1	2	3	4	5
流水线段3							0	1	2	3	4	5	6
流水线段2						0	1	2	3	4	5	6	7
流水线段1					0	1	2	3	4	5	6	7	
存储器7								RA7	RA7	RA7	RA7		
存储器6							RA6	RA6	RB6	RB6			
存储器5						RA5	RA5	RB5	RB5				
存储器4					RA4	RA4	RB4	RB4					...
存储器3				RA3	RA3	RB3	RB3					...	...
存储器2			RA2	RA2	RB2	RB2					...	...	...
存储器1		RA1	RA1	RB1	RB1					RA9	RA9	RB9	RB9
存储器0	RA0	RA0	RB0	RB0					RA8	RA8	RB8	RB8	W0

时间（时钟周期）

每 4 个周期，就取出 A 和 B 的各 1 个分量，经过 4 个周期的计算流水线后生成结果，因此第 1 个计算结果在第 7 个时钟周期结束时产生。为了避免读写冲突，将写操作推迟 4 个周期，因此第 1 个计算结果在第 12 个时钟周期写入主存，之后每个时钟周期都写一个计算结果。

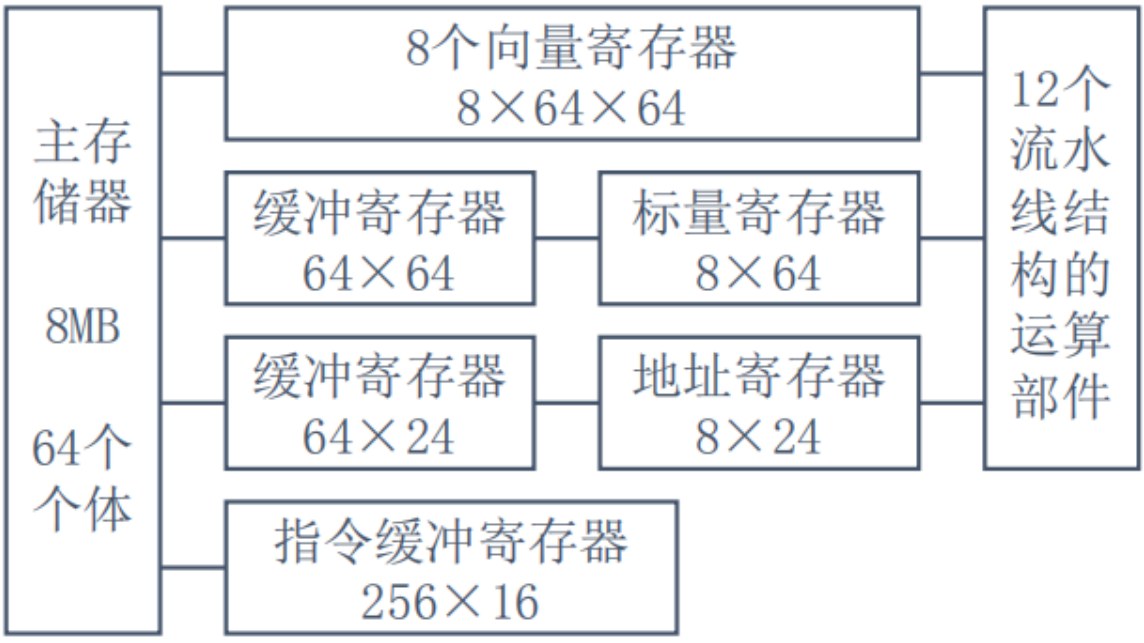
可变延迟的成本较高，性能相较第一种办法有所下降，但是数据的排布比较方便，也是值得选择的一种方案。

整个结构的表示图如下：



2. 寄存器-寄存器结构

核心思想是在运算器与主存之间增加层次结构的存储系统，带宽最高的这一级靠近仅运算器的一侧，称为向量寄存器，运算部件需要的操作数从向量寄存器中读取，运算的中间结果也写到向量寄存器中。其结构如下：



其中标量寄存器处理标量运算，向量寄存器处理向量运算。

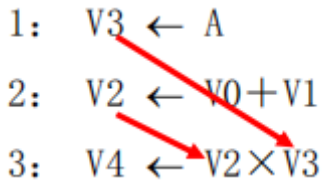
2. 向量链接技术

性能瓶颈：对于寄存器-寄存器结构，当发出向量指令时，所使用的功能部件和操作数寄存器就要预定若干个时钟周期，直到计算完成，此时使用同一组功能部件或同一寄存器的后续指令是不能发出的。通常来说可能产生下图中的 b、c、d 三种冲突：



基本思想：链接是把一个流水部件的计算结果，直接送入另一个流水线的操作数寄存器的过程。中间结果不必送回存储器，而是直接作为后指令的操作数。

例如，有如下 3 条向量指令：



第 1、2 条指令没有数据相关和功能部件冲突，可以并行执行；第 3 条指令与前两条指令均存在写读数据相关，可以链接执行。

向量链接技术的主要要求：

- 无向量寄存器使用冲突，无向量功能部件使用冲突；
- 只有在前一条向量指令的 **第一个结果元素** 送入结果向量寄存器的那一个时钟周期才可以进行链接，若错过该时刻就不能进行链接；
- 所使用的有关向量功能部件的延迟时间相等；
- 所有可链接执行的向量指令的向量长度相等。

计算执行时间的例题：

> 假设向量长度均为 64，在 CRAY-1 机上所用浮点功能部件的执行时间分别为：相加 6 拍，相乘 7 拍，求倒数近似值 14 拍；在存储器读数 6 拍，打入寄存器及启动功能部件各 1 拍。问下列各指令组内的哪些指令可以链接？哪些指令不可链接？不能链接的原因是什么？分别计算出各指令组全部完成所需的拍数。
>
> (1) V0←存储器
> V1←V2+V3
> V4←V5×V6
>
> (2) V2←V0×V1
> V3←存储器
> V4←V0+V3
>
> (3) V0←存储器
> V2←V0×V1
> V3←V2+V0
> V5←V3+V4
>
> (4) V0←存储器
> V1←1/V0
> V3←V1×V2
> V5←V3+V4

访存所需时钟周期为：1 + 6 + 1 + 63 = 71 拍；

加法所需时钟周期为：1 + 6 + 1 + 63 = 71 拍；

乘法所需时钟周期为：1 + 7 + 1 + 63 = 72 拍；

（1）三条指令可以并行，指令所需时钟周期分别是：71 拍，71 拍，72 拍；三条指令并行共需 72 拍；

（2）第一条可与第二条并行，与第三条串行，因为第三条指令与第一、第二条指令分别存在结构冲突和数据冲突，共需 72 + 71 = 143 拍；

（3）第一条可与第二条链接，与第三条和第四条串行执行，因为第三条与第二条存在数据冲突，第四条与第三条存在加法部件冲突，第 1、2 条指令需要 1 + 6 + 1 + 1 + 7 + 1 + 63 = 80 拍，三条指令串行共需 80 + 71 + 71 = 222 拍；

（4）四条指令可以链接，第一个分量需要 1 + 6 + 1 + 1 + 14 + 1 + 1 + 7 + 1 + 1 + 6 + 1 = 41 拍，总共需要 41 + 63 = 104 拍。

3. 向量循环开采技术

当向量长度大于向量寄存器的长度时，必须把长向量分成长度固定的段，每次分段处理，这种技术称为向量循环开采技术。

4. 向量机的性能评价

1. 向量指令的处理时间 T_{vp}

执行一条长度为 n 的向量指令，所需时间 T_{vp} 可表示为：

$$T_{vp} = T_s + T_{vf} + (n-1) \cdot T_c$$

- T_s：向量流水线的建立时间，即流水线部件的启动时间；
- T_{vf}：流水线的流过时间，即第一组分量流过流水线并产生结果所花的时间；
- T_c：流水线 “瓶颈” 段的执行时间，典型的结构中每个部件花费 1 个时钟周期。

为简化起见，记 $\triangle t$ 为时钟周期长度，T_{start} 为向量指令的启动时钟周期数（从第一条指令开始执行，到还差一个时钟周期就产生第一个运算结果所需的时钟周期数），则上式表示为：

$$T_{vp} = (T_{start} + n) \cdot \triangle t$$

将几条能 **并行** 执行的向量指令称为一个 **编队** ，编队内部一定不存在冲突，向量指令序列的总的执行时间为 m 个编队的执行时间的和：

$$T_{all} = \sum_{i=1}^{m} T_{vp}^{i} = \sum_{i=1}^{m} (T_{start}^{(i)} + n) \cdot \triangle t = (T_{start} + mn) \cdot \triangle t$$

- T_{vp}^{i} ： 第 i 个编队的执行时间；
- m：编队的个数；
- T_{start} ： 每个编队的启动时间之和。

如果向量长度大于寄存器长度 MVL，则需要分段开采，其开销由执行标量代码的开销 T_{loop} 和每个编队的向量启动开销 T_{start} 组成。根据上述公式可以看出，循坏开采只会影响 T_{start} 的值，mn 项不变，所以总时间为：

$$T_{all} = (\lceil \frac{n}{MVL} \rceil \cdot (T_{start} + T_{loop}) + mn) \cdot \triangle t$$

例题：

在一台向量处理机上实现 A = B × s 操作，其中 A 和 B 是长度为 200 的向量，s 是一个标量。向量寄存器长度为 64，T_{loop} = 15，各功能部件的启动时间如下，求总的执行时间。

- 取数和存数部件为 **12** 个时钟周期；
- 乘法部件为 **7** 个时钟周期；
- 加法部件为 **6** 个时钟周期；

```
LV   V1, Rb
MUL  V2, V1, Fs
SV   Ra, V2
```

Solution.

各编队之间存在写读数据相关，因此划分为 3 个编队，m = 3；

$$\begin{aligned} T_{all} &= \lceil \frac{n}{MVL} \rceil \cdot (T_{start} + T_{loop}) + mn \\ &= \lceil \frac{200}{64} \rceil \cdot (12 + 7 + 12 + 15) + 200 \cdot 3 \\ &= 784 \end{aligned}$$

2. 最大性能 R_∞

R_∞ 表示向量长度为无穷大时流水线的最大性能，也称峰值性能，时钟频率的单位为 MHz，结果单位为 MFLOPS：

$$\begin{aligned} R_\infty &= lim_{n \to \infty} \frac{浮点运算次数 \times 时钟频率}{所有指令所花费的时钟周期数} \\ &= lim_{n \to \infty} \frac{浮点运算次数 \times 时钟频率}{\lceil \frac{n}{MVL} \rceil \cdot (T_{start} + T_{loop}) + mn} \end{aligned}$$

3. 半性能向量长度 $n_{1/2}$

$n_{1/2}$ 是达到一半 R_∞ 所需的向量长度，是评价向量流水线建立时间对性能影响的参数。若 $n' = n_{1/2}$，说明整个处理机只有一半时间在做有效操作。通常希望 $n_{1/2}$ 的值较小。一般测试得出。

4. 向量长度临界值 n_v

n_v 表示，对于某一计算任务而言，向量方式的处理速度优于标量串行方式处理速度时所需要的最小向量长度。假设对于某一任务而言，其标量计算时间为：

$$T_{scalar} = (10 + 12 + 12 + 7 + 6 + 12) \times n = 59n$$

其向量方式所需的计算时间为：

$$T_{vector} = 64 + 3n$$

令 $T_{scalar} = T_{vector}$，则有：

$$n_v = n = \lceil \frac{64}{56} \rceil = 2$$

第七章 互联网络

1. 互连网络的作用

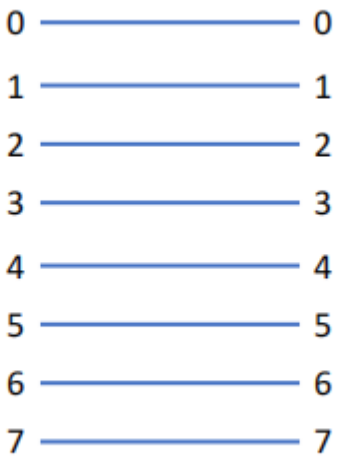
互连网络是一种由开关元件按照一定的拓扑结构和控制方式构成的网络，用来实现计算机系统中节点之间的相互连接。

2. 互连函数

互连函数表示相互连接的输出端号和输入端号之间的一一对应关系，即在互连函数 f 的作用下，输入端 x 连接到输出端 f(x)。下面介绍几种常见的互连函数：

- **恒等置换**：相同编号的输入端与输出端一一对应互连所实现的置换：

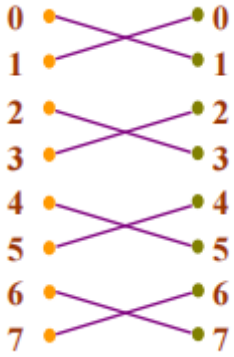
$$I(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-1}x_{n-2}\dots x_1x_0$$



$N = 8$的恒等置换

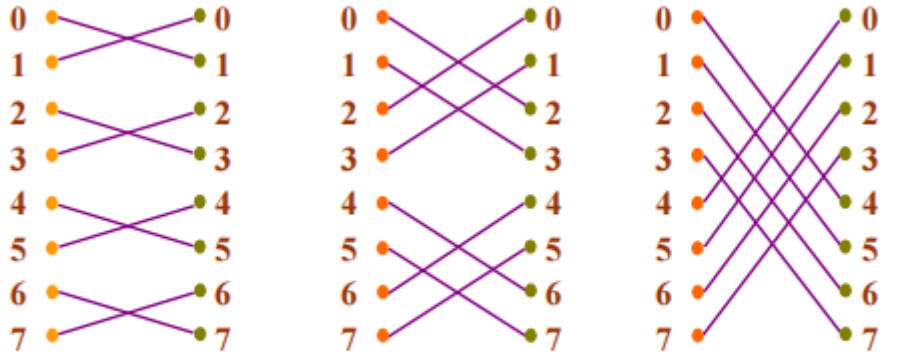
- **交换置换**：实现二进制地址编码中第 0 位互反的输入端与输出端之间的连接：

$$E(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-1}x_{n-2}\dots x_1\overline{x_0}$$



- **方体置换**：实现二进制地址编码中第 k 位互反的输入端与输出端之间的连接：

$$C_k(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-1}\dots\overline{x_k}\dots x_0$$



(a) Cube_0 交换函数　(b) Cube_1 交换函数　(c) Cube_2 交换函数

- **均匀洗牌置换**：将输入端二进制循环左移一位得到对应的输出：

$$\sigma(x_{n-1}x_{n-2}\dots x_1x_0) = x_{n-2}x_{n-3}\dots x_0x_{n-1}$$

  - **子洗牌**：把输入端的二进制编号的低 k 位循环左移一位得出输出：

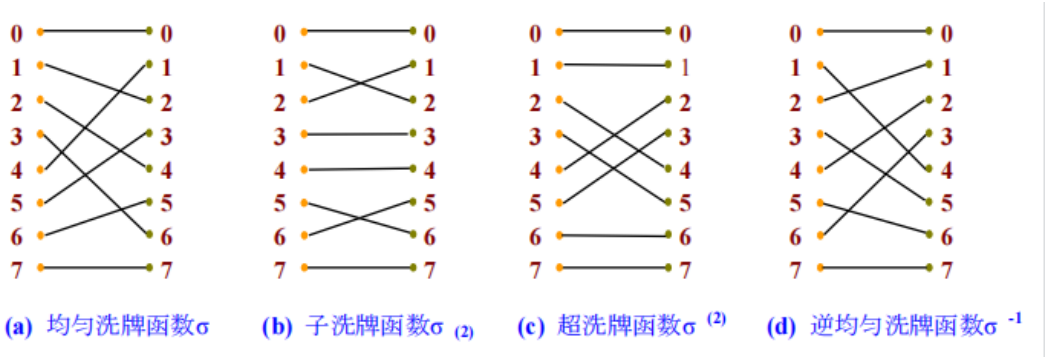
$$\sigma_k(x_{n-1}\dots x_kx_{k-1}\dots x_0) = x_{n-1}\dots x_k\dots x_0x_{k-1}$$

- **超洗牌**：把输入端的二进制编号的高 k 位循环左移一位得出输出：

$$\sigma^k(x_{n-1}x_{n-2}\dots x_{n-k}\dots x_0) = x_{n-2}\dots x_{n-k}x_{n-1}\dots x_0$$

- **逆混洗**：将输入端二进制 循环右移 一位得到对应的输出：

$$\sigma^{-1}(x_{n-1}x_{n-2}\dots x_1x_0) = x_0x_{n-1}\dots x_2x_1$$



- **蝶式置换**：把输入端的二进制编号的最高位与最低位互换位置，得到输出端的编号：

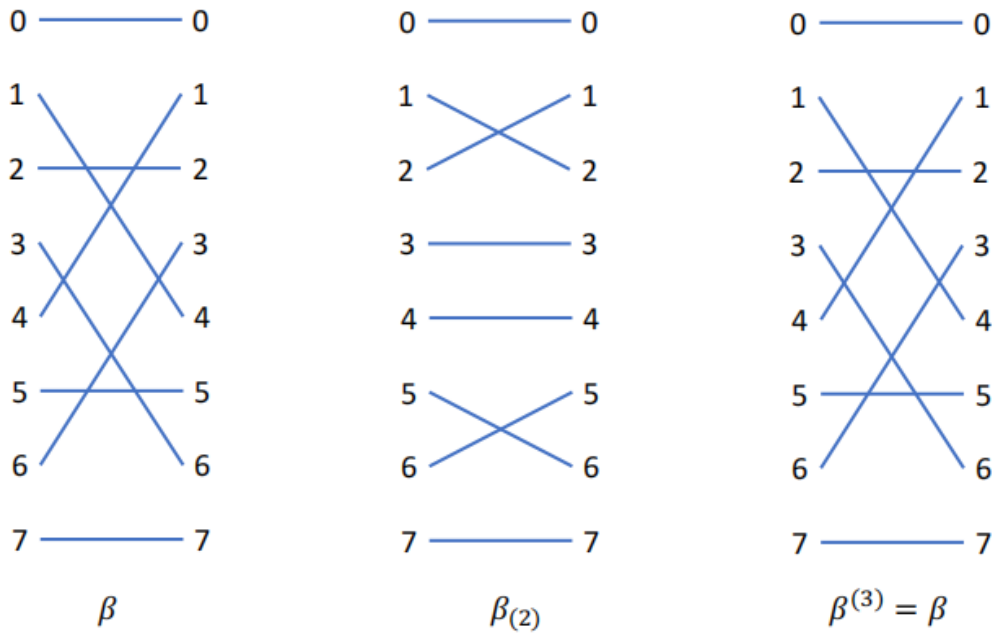
$$\beta(x_{n-1}x_{n-2}\dots x_1x_0) = x_0x_{n-2}\dots x_1x_{n-1}$$

- **子蝶式**：把输入端的二进制编号的低 k 位中的最高位与最低位互换：

$$\beta_k(x_{n-1}\dots x_kx_{k-1}\dots x_0) = x_{n-1}\dots x_kx_0\dots x_{k-1}$$

- **超蝶式**：把输入端的二进制编号的高 k 位中的最高位与最低位互换：

$$\beta^k(x_{n-1}\dots x_{n-k}\dots x_0) = x_{n-k}\dots x_{n-1}\dots x_0$$



- **位序颠倒置换**：将输入端二进制编号的位序颠倒过来求得相应输出端的编号：

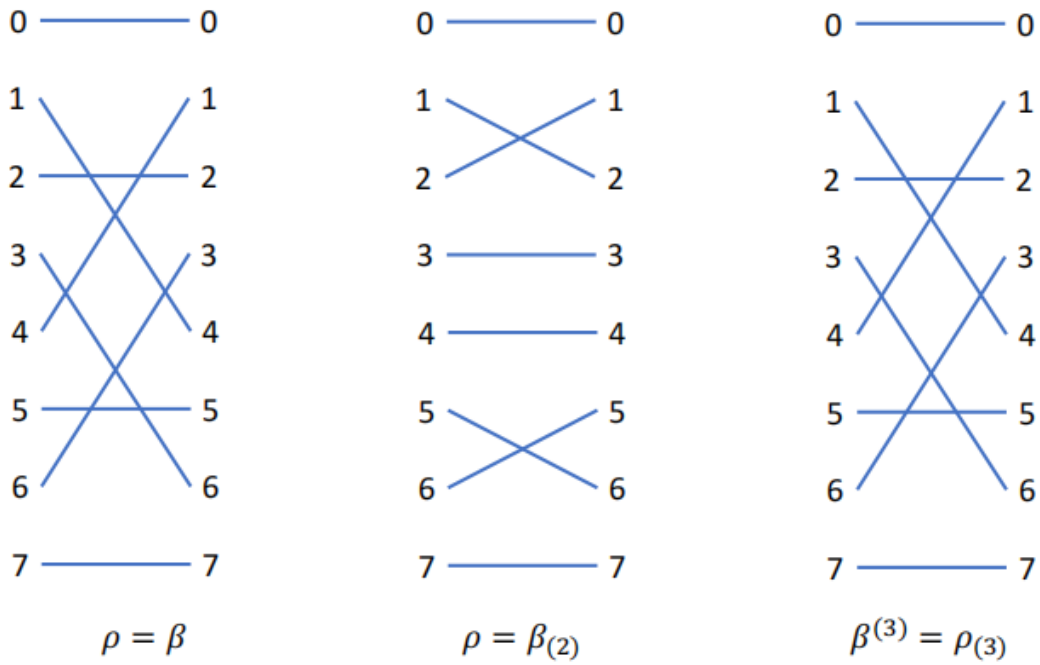
$$\rho(x_{n-1}x_{n-2}\dots x_1x_0) = x_0x_1\dots x_{n-2}x_{n-1}$$

- **子位序颠倒**：把输入端的二进制编号的低 k 位中各位的次序颠倒过来：

$$\rho_k(x_{n-1}\dots x_kx_{k-1}\dots x_0) = x_{n-1}\dots x_kx_0x_1\dots x_{k-1}$$

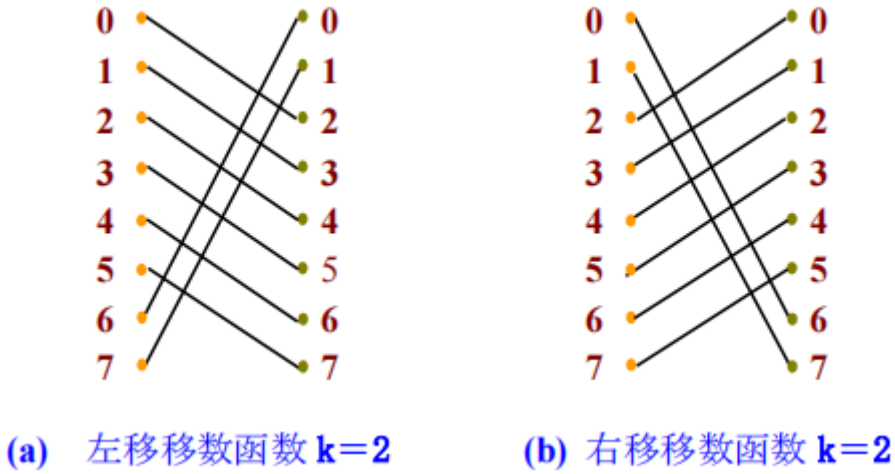
- **超位序颠倒**：把输入端的二进制编号的高 k 位中各位的次序颠倒过来：

$$\rho^k(x_{n-1}\dots x_k x_{k-1}\dots x_0) = x_{n-k}x_{n-k+1}\dots x_{n-2}x_{n-1}x_{n-k-1}\dots x_1 x_0$$



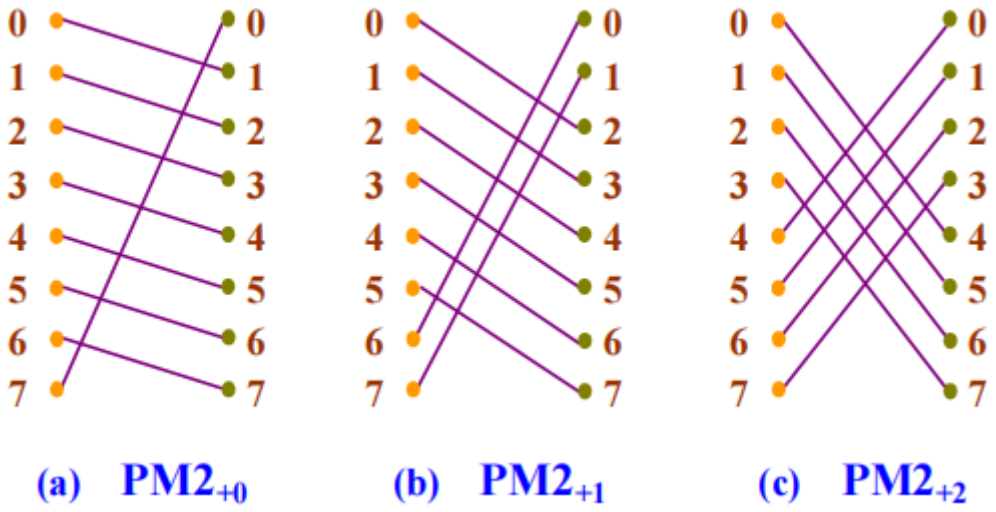
- **移数置换**：将各输入端都循环错开一定的位置后连到输出端：

$$a(X) = (X + k)\ mod\ N$$



- **PM2I 函数**：P 和 M 分别表示加和减，2I 表示 2^i：

$$PM2_{+i}(X) = X + 2^i\ mod\, N$$
$$PM2_{-i}(X) = X - 2^i\ mod\, N$$



3. 静态互连网络的基本类型

网络规模：网络中的结点数；

结点度：与结点相连的平均边数；

距离：两结点之间相连的最少边数；

网络直径：任意两结点之间的距离最大值；

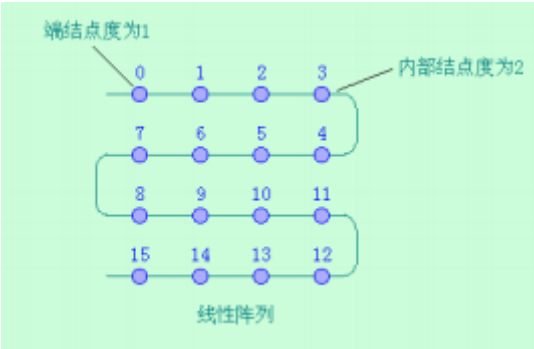
等分宽度：网络被切分成相等的两半时，沿切口的最小边数；

结点间线长：两个结点之间连线的长度；

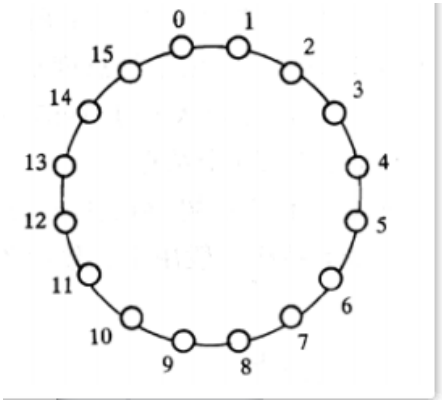
对称性：完全对称的网络称为对称网络。

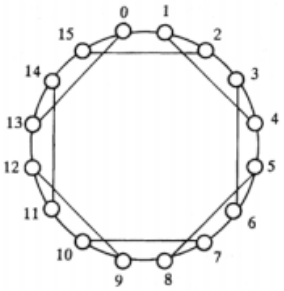
1. 线性阵列

一种一维的线性网络，其中 N 个节点用 N-1 个链路连成一行。

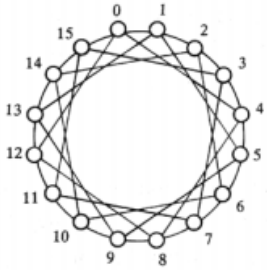


2. 环和带弦环





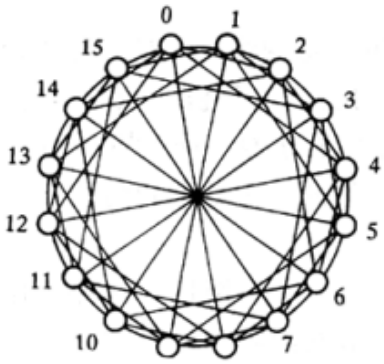
节点度3，直径5



节点度4，直径3

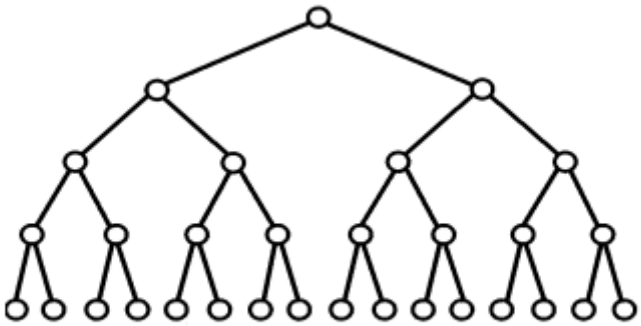
3. 循环移数网络

通过在环上每个节点到所有与其距离为 2 的整数幂的节点之间都增加一条附加链而构成：



- 循环移数网络的连接特性与结点度较低的任何带弦环相比有了改进
- 对$N = 16$的情况，循环移数网络的结点度为7，直径为2
- 复杂性比全连接网络要低得多

4. 树形和星形

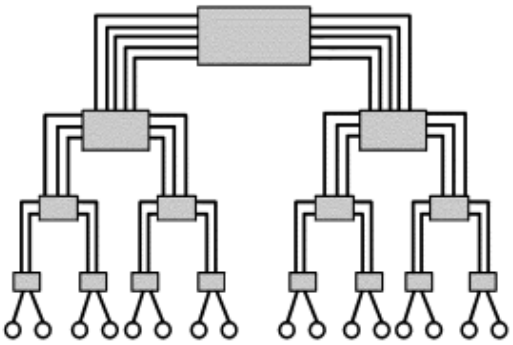


(a) 二叉树

5. 胖树

胖树的通道宽度从叶结点往根结点上行方向逐渐增宽

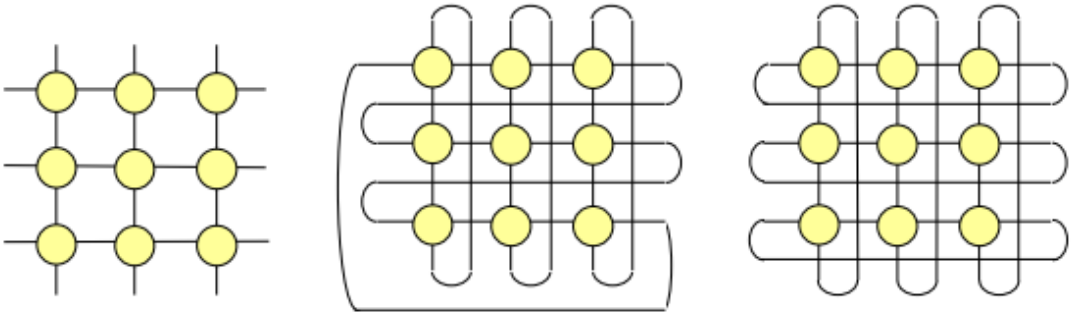
传统二叉树的主要问题之一就是通向根结点的瓶颈问题，这是因为根部的通信最忙，而胖树的提出使问题得到了缓解



(c) 二叉胖树

6. 网格形和环形

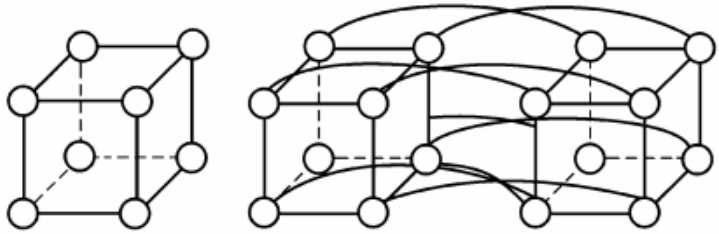
Illiac 网络把二维网格形网络的列首尾相连，行循环相连，环形网的行列都首尾相连：



（a）网格形　　（b）Illiac网　　（c）环网形

7. 超立方

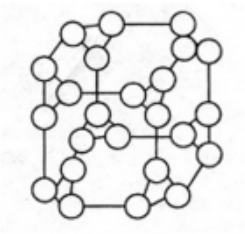
- 一种二元n-立方体结构，由$N=2^n$个节点组成，它们分布在n维上，每维有两个节点
- 为实现一个n-立方体，只要把两个$n-1$立方体中相对应的节点用链路连接起来即可。共需要2^{n-1}条链路
- n-立方体中节点的度都是n，直径也是n，等分宽度为$b=N/2$



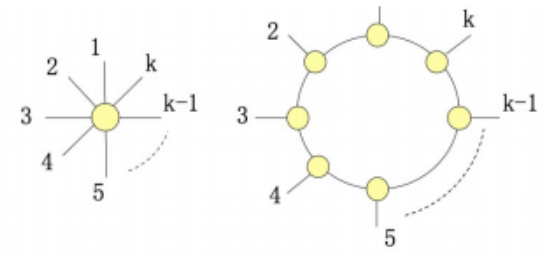
(a) 3–立方体　　(b) 由 2 个 3– 立方体组成的 4– 立方体

8. 带环立方体

- 带环k-立方体（简称k-CCC），是k-立方体的变形，通过用k个节点构成的环取代k-立方体中的每个节点而形成的。
- 网络规模为$N=k\times 2^k$；网络直径为$D=2k-1+\lfloor k/2\rfloor$，比$k$-立方体的直径大一倍；等分宽度为$b=\frac{N}{2k}$



(c)把3-立方体的每个节点换成一个由3个节点构成的环，形成带环3-立方体



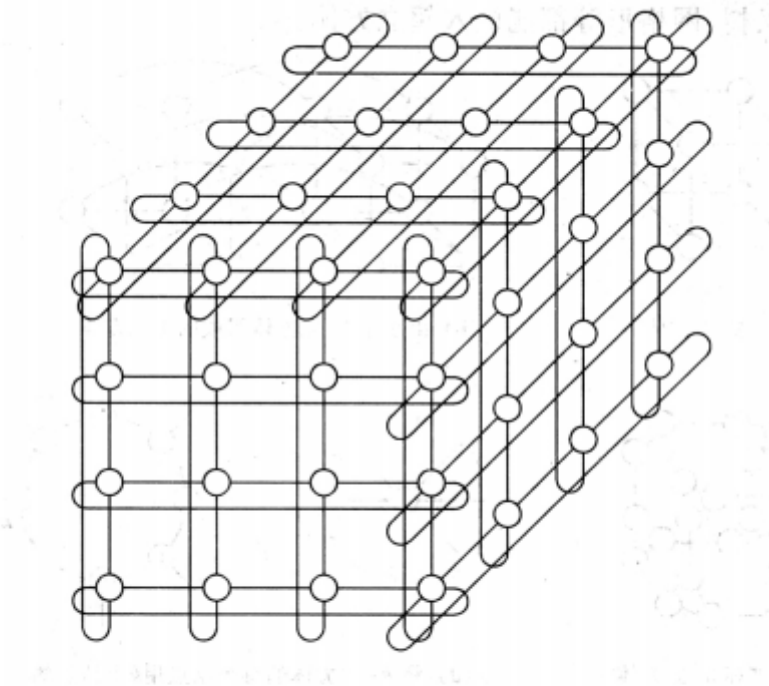
(b) 将k-立方体的每个节点用由k个节点的环来代替，组成带环k-立方体组成

9. k 元 n 立方网络

- 环形、网格、环网形、二元n-立方体（超立方体）和Omega网络都是k元n-立方体网络系列的拓扑同构体。
- 在k元n-立方体网络中，参数n是立方体的维数， k是基数，即每一维上的节点个数。

$$N = k^n, \left(k = \sqrt[n]{N}, n = \log_k N\right)$$

  - k元n-立方体的节点可以用基数为k的n位地址$A = a_1 a_2 \ldots an$来表示。
  - 其中a_i表示该节点在第i维上的位置
  - 通常把低维k元n-立方体称为**环网**，而把高维k元n-立方体称为**超立方体**。



4元3-立方体网络

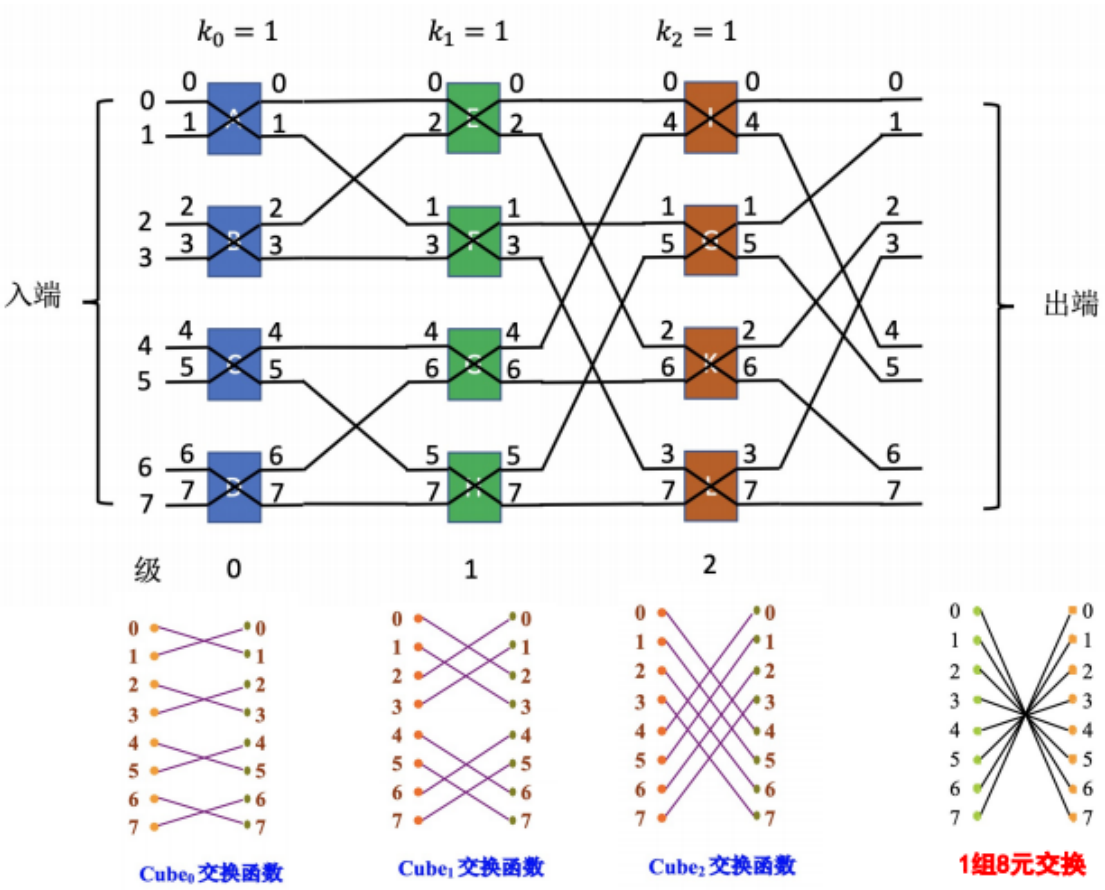
4. 动态互连网络

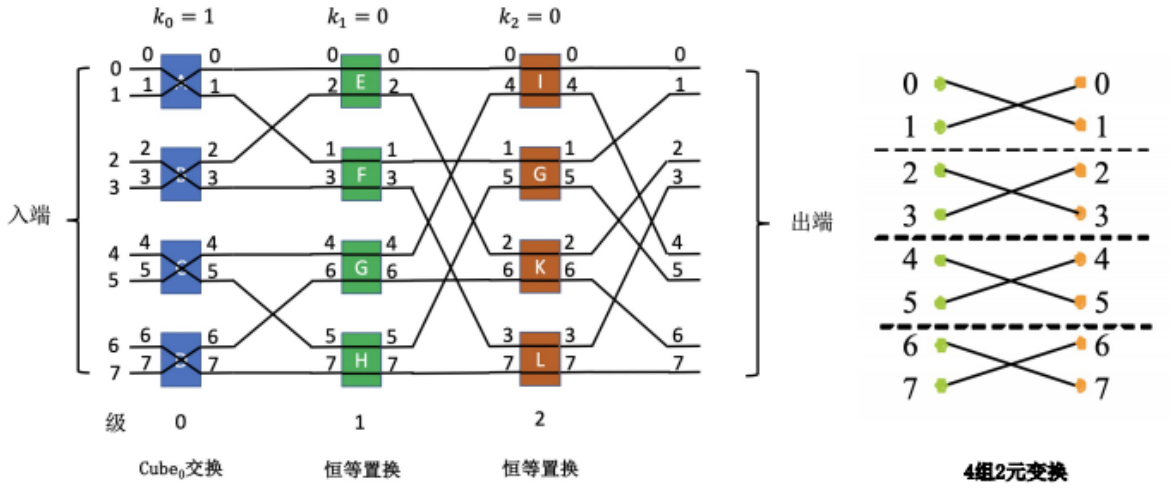
1. 多级立方体网络

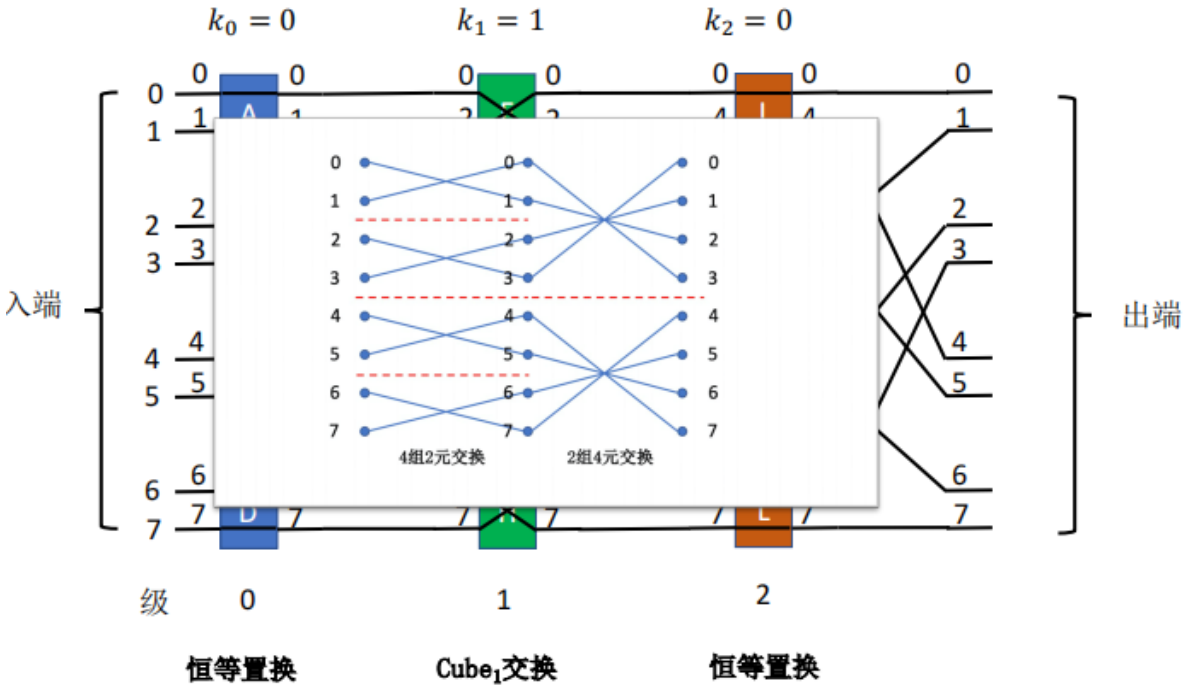
➢多级立方体网络包括**STARAN网络**和**间接二进制n方体网络**等。

- 两者仅在**控制方式**上不同：STRAN网络采用级控制（称交换网络）和部分级控制（其中可实现移数功能的称移数网络），而间接二进制n方体网络采用单元控制，具有更大的灵活性。
- 共同点：都采用二功能（直送和交换）的2×2开关；当第i级（$0 \le i \le n-1$）交换开关处于交换状态时，实现的是$Cube_i$互连函数。

➢一个N输入的多级立方体网络有$\log_2 N$级，每级用$N/2$个2×2开关模块，共需要$\log_2 N \times N/2$个开关。



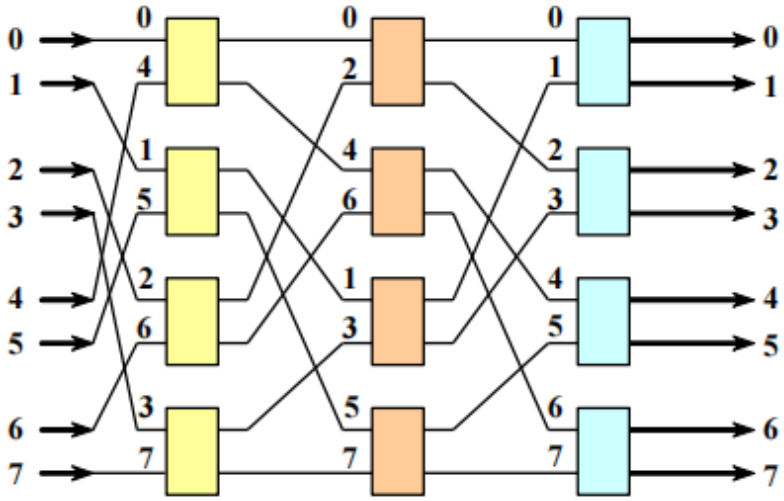

$k_0 = 1$
$k_1 = 0$
$k_2 = 0$
入端
出端
级 0 1 2
Cube$_0$交换
恒等置换
恒等置换
4组2元交换


$k_0 = 0$
$k_1 = 1$
$k_2 = 0$
入端
出端
4组2元交换
2组4元交换
级 0 1 2
恒等置换
Cube$_1$交换
恒等置换

级控制信号 $k_2k_1k_0$	连接的输出端号序列（入端号序列：01234567）	实现的分组交换	实现的互连函数
000	0 1 2 3 4 5 6 7	恒等	I
001	1 0 3 2 5 4 7 6	4组2元交换	$Cube_0$
010	2 3 0 1 6 7 4 5	4组2元交换＋ 2组4元交换	$Cube_1$
011	3 2 1 0 7 6 5 4	2组4元交换	$Cube_0$＋$Cube_1$
100	4 5 6 7 0 1 2 3	2组4元交换＋ 1组8元交换	$Cube_2$
101	5 4 7 6 1 0 3 2	4组2元交换＋ 2组4元交换＋ 1组8元交换	$Cube_0$＋$Cube_2$
110	6 7 4 5 2 3 0 1	4组2元交换＋ 1组8元交换	$Cube_1$＋$Cube_2$
111	7 6 5 4 3 2 1 0	1组8元交换	$Cube_0$＋$Cube_1$＋ $Cube_2$

2. Omega 网络

- 一个8×8的Omega网络
  - 每级由4个4功能的2×2开关构成
  - 级间互连采用均匀洗牌连接方式

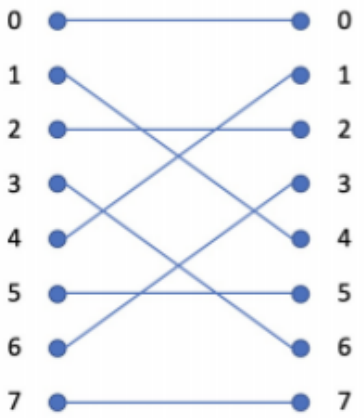


在 Omega 网络中，目的地址编码从高位开始的第 i 位为 0 时，第 i 级的 2×2 开关的输入端与上输出端连接，否则与下输出端连接。发生冲突时，可以采用多级 Omega 网络。

➢ 一个N输入的Omega网络

- 有$\log_2 N$级，每级用N/2个2×2开关模块，共需要$N\log_2 N/2$个开关。
- 每个开关模块均采用单元控制方式。
- 不同的开关状态组合可实现各种置换、广播或从输入到输出的其他连接。

5. 给出如下所示的 N=8 的蝶式变换



1) 写出互连函数关系式。

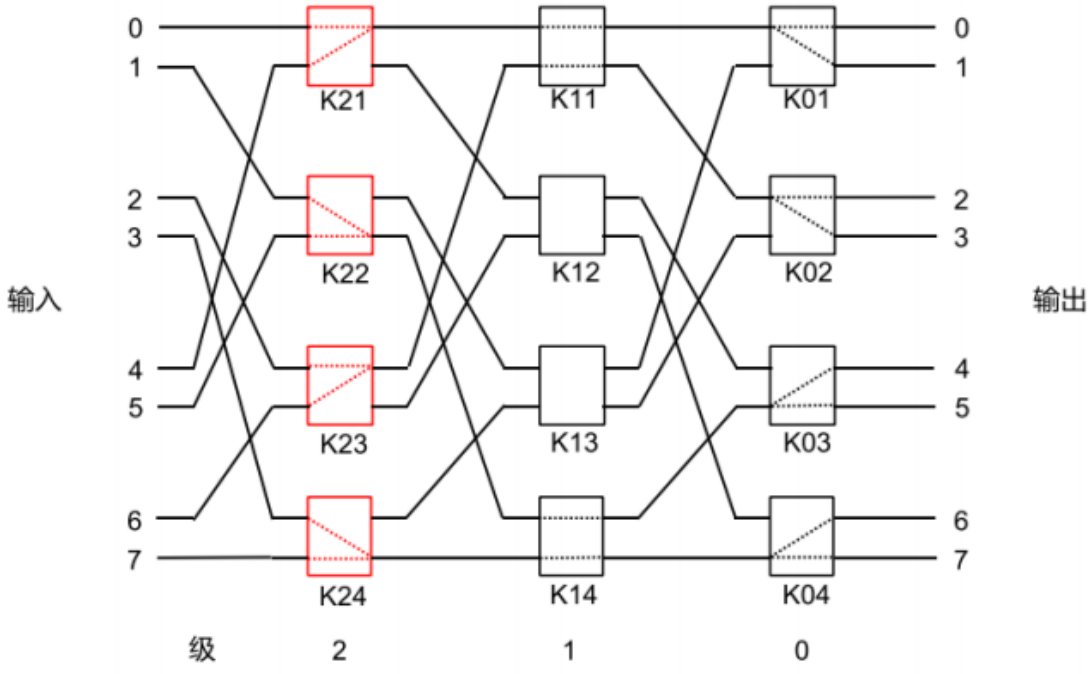
2) 如采用 Omega 网络，需几次通过才能完成此变换。

3) 列出 Omega 网络实现此变换的控制状态图。

1）互联函数关系式：

$$f(P2P1P0) = P0P1P2$$

2）如果采用一次连接：

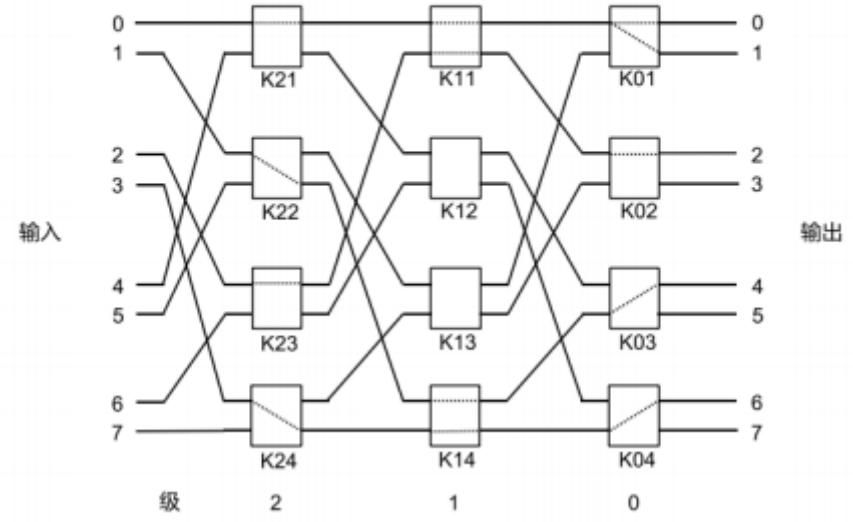


在第 2 级开关会发生以上冲突，因此至少需要两层实现：

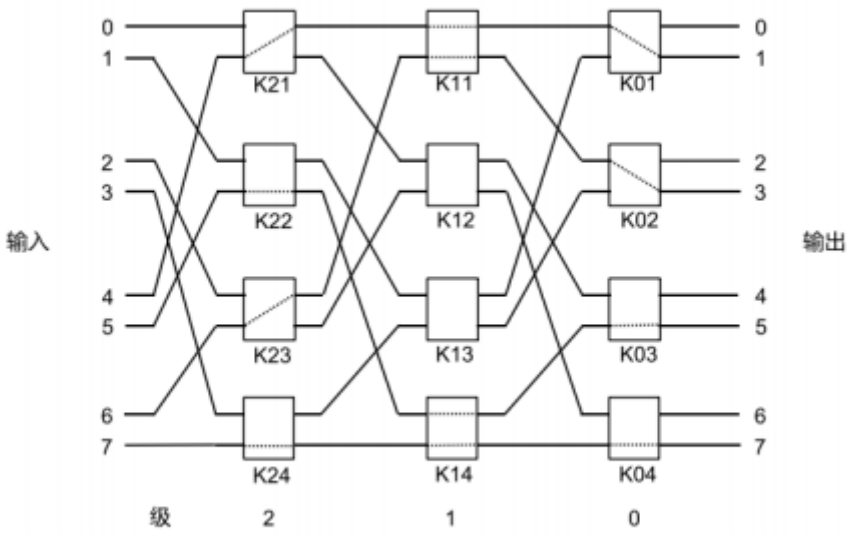
第一次：0 -> 1，1 -> 4，2 -> 2，3 -> 6；

第二次：4 -> 1，5 -> 5，6 -> 3，7 -> 7。

3）两次实现的控制图如下：



第一次



第二次